TOWER_01 / DELHI
28.6139° N · 77.2090° E
TIERS 04 · SITES 020
CHLOROPHYLL MEETS SILICON

THE COMPLETE WALKTHROUGH

PROJECT VERDE



→ WHY IT WAS MADE
→ WHAT IT USES
→ WHAT IT DOESN'T

A file-by-file explanation of the Project Verde website: the problem behind it, every technology and component inside it, the things that were removed or never used, and a verified review of what to fix before the exhibition.

REPOSITORY
UCHIHA-MADARA-ANUJ/
Verde-CBSE

SNAPSHOT
main · 9d5576d
merged 9 Oct 2026

BUILT BY
Anuj Phulera
Aarav Choudhary

PREPARED
18 October 2026
CBSE Science Exhibition

CONTENTS

– Project Verde at a glance 04

PART I · THE WHY

01 Why Verde exists 07
1.1 The problem, in the project's own words 07
1.2 The idea: a tower that looks after itself 07
1.3 The context: a school science exhibition 08
1.4 Why build a website at all? 08
1.5 The design goals, as written in the code 08
1.6 Who the website is for 09

02 The system the website documents 10
2.1 The loop: six steps, forever 10
2.2 Architecture of the physical system 11
2.3 Bill of materials, as listed on the site 11
2.4 The four tiers: the guided tour's story 12
2.5 How decisions are made (as described) 12
2.6 How it was built: the 14-week build log 13
2.7 The team 13

03 How the website was built 14
3.1 The commit timeline 14
3.2 Generation 2: what came before 14
3.3 Generation 3: the maximalist rebuild 15
3.4 Three bugs that were fixed along the way 15
3.5 Claims in the pull request, re-verified 15

PART II · WHAT WAS USED

04 The stack at a glance 18
4.1 Runtime dependencies 18
4.2 Development tooling 18
4.3 Not in package.json but essential 19
4.4 Browser platform features used 19
4.5 Scripts 19

05 Architecture 21
5.1 Repository map 21
5.2 Rendering model 21
5.3 The page composition 22
5.4 The layer stack 23
5.5 The ScenePreloader: the central idea 24
5.6 The message protocol 27
5.7 The boot sequence 28
5.8 Why iframes instead of React Three Fiber? 29
5.9 Global behaviours 29

06 The page, section by section 31
6.0 The page map 31
6.1 Boot curtain 32
6.2 Hero: "We grow the future." 33
6.3 Navigation and the command palette 34
6.4 Velocity marquees 35
6.5 Manifesto: "Soil is running out." 36
6.6 Stats strip 37
6.7 Pillars: "Growing is mostly attention." 38
6.8 The loop deck: "Six steps, forever." 38
6.9 The guided tour: four chapters, one live tower 39
6.10 Live console: "It talks to itself." 40
6.11 Telemetry: "The system is always reporting." 41
6.12 Against the ground: "Dirt loses." 42
6.13 Down at plant level: the seedling pod 43
6.14 The hardware: the exploding board 43
6.15 Subsystems: "Everything inside, in plain terms." 44
6.16 Build log: "Fourteen weeks, in order." 45
6.17 FAQ, Team and Closing 45

07 The three 3D scenes 47
7.1 The shared recipe 47
7.2 The tower · tower.html (443 lines, three r162) 48
7.3 The seedling pod · pod.html (1,602 lines, three r162) 48
7.4 The ESP8266 board · board.html (506 lines, three r128) 49
7.5 Originals versus embedded copies 50
7.6 The plan that was not used: AI-generated GLB models 50

08 The design system 51
8.1 Two layers in one stylesheet 51
8.2 Colour 51
8.3 Typography 52
8.4 Motion language 52
8.5 Signature components 53
8.6 Respecting reduced motion 53

09 API, SEO, config & deployment 54
9.1 The five API routes 54
9.2 SEO and sharing metadata 55
9.3 next.config.mjs, explained 56
9.4 Deployment files 56
9.5 What actually gets downloaded 57

PART III · WHAT WAS NOT USED

10 Not used, and why 59
10.1 Packages that were removed 59
10.2 Files that were removed, and what replaced them 59
10.3 Leftovers: still in the repository, unused today 60
10.4 The README versus reality 61
10.5 Simulated versus real 62
10.6 Technologies deliberately not used 62

PART IV · REVIEW & NEXT STEPS

11 Quality review: verified findings 65
11.1 Method and findings index 65
11.2 The manifesto headline never appears 66
11.3 3D frames show opaque white areas 67
11.4 Pod scene: white fog horizon and a light toolbar 68
11.5 Loop deck: a fixed travel distance that only fits tablets 69
11.6 Tour chapter buttons jump to the wrong chapter 71
11.7 The tower never appears on phones or tablets 71
11.8 Headlines overflow or overlap 72
11.9 Brand leftovers and broken references 72
11.10 Performance: invisible scenes keep rendering 73
11.11 Accessibility 73
11.12 Maintenance and security 74
11.13 Accuracy of technical claims 74

12 Recommendations & exhibition prep 75
12.1 The fix-first plan (about one day) 75
12.2 Roadmap: what would make it outstanding 76
12.3 Exhibition-day checklist 76
12.4 Questions judges may ask, with honest answers 76
12.5 A 90-second demo script 77

APPENDICES

A File-by-file index 78

B Glossary 81

C Verification log 83

D References 85

PROJECT VERDE AT A GLANCE

Project Verde is an autonomous, closed-loop vertical hydroponic tower: four stacked tiers, twenty growing sites, sensors that decide when to water, and cloud alerts on WhatsApp in English and Hindi. It was built by a two-person team for a **CBSE Science Exhibition**. This repository does not contain the tower's firmware or circuit files. It contains the **website** that explains the tower: a cinematic, scroll-driven Next.js site with three interactive 3D models built entirely in code.

This document explains that website completely: **why** it exists, **everything it uses** (and the reason for each choice), **everything it does not use** (including 66 files and 6 packages that were deliberately removed), and an independent **review** of what works and what is broken, with screenshots and measurements as evidence.

31
REACT COMPONENTS
all client components, 3,051 lines

3
LIVE 3D SCENES
tower · seedling pod · ESP8266 board

20
PAGE BLOCKS
17 sections + 3 marquees, 23.6 screens tall

157 kB
FIRST-LOAD JAVASCRIPT
three.js is not in it

0
REQUIRED ENV VARS
builds with an empty environment

5
API ROUTES
all simulated, none used by the UI

8
COMMITS
all on 8 Oct 2026, one evening

3 / 3
CHECKS PASS
tsc · next lint · next build

WHAT IS GENUINELY EXCELLENT

- ✓ **Scene preloader architecture.** Three WebGL iframes are mounted once and glued to placeholders as you scroll, so models never reload.
- ✓ **Procedural 3D.** Tower, pod and board are built from code (no model files) in 26–76 KB of HTML each.
- ✓ **Production hygiene.** Static prerendering, security and caching headers, SEO metadata, and a clean type-check, lint and build.
- ✓ **Motion design.** Velocity-reactive marquees, a pinned horizontal deck, camera flights driven by scroll, and reduced-motion support.
- ✓ **Honest engineering trail.** Clear code comments explain intent, and fixed bugs (hydration, sticky) are documented in the history.

WHAT TO FIX BEFORE ANYONE SEES IT

- ✕ HIGH The manifesto headline **"SOIL IS RUNNING OUT."** never appears (§11.2).
- ✕ HIGH The 3D frames show **opaque white areas** in Chrome, Edge and Firefox because of a color-scheme mismatch (§11.3).
- ✕ MEDIUM The loop deck's travel only fits tablets: on phones the last two cards never appear, and desktops end on an empty screen (§11.5).
- ✕ MEDIUM The tour's chapter buttons jump to the wrong chapter, and the tower never appears on phones (§11.6–11.7).
- ✕ INFO Every number on the site is simulated. Label it or wire real data, and prepare honest answers (§10.5, §12.4).

HOW TO READ THIS DOCUMENT

PART	QUESTION IT ANSWERS	CHAPTERS
I · The why	Why was Verde built, what physical system does the site describe, and how was the site made?	1 – 3
II · What was used	Every library, file, component, 3D scene, design token, API route and config, and why each was chosen.	4 – 9
III · What was not used	Removed packages and files, unused leftovers, technologies deliberately avoided, and what is simulated.	10
IV · Review	Verified bugs with evidence, accessibility, performance, accuracy of claims, and a fix-first plan.	11 – 12
Appendices	File-by-file index of all 73 files, glossary, verification log and references.	A – D

HOW THIS WALKTHROUGH WAS PRODUCED

Every tracked file in the repository was read, including all 31 components, the 602-line stylesheet, the three scene documents (2,551 lines), the original prototypes, configs and docs. The full git history (8 commits) and pull request #1 were inspected. The site was installed with `npm ci`, verified with `tsc --noEmit`, `next lint` and `next build`, and then run with `next start`. Every section was captured in Chromium 131 (headless, software WebGL) at 1440×900 and on a 390×844 phone profile. Every bug claimed in Chapter 11 was reproduced and measured. Where this document *infers* intent rather than quoting it, it says so.

01

PART I

THE WHY

Before any code: the problem Verde answers, the machine the website describes, and how the website itself came to be: built, rebuilt and shipped in a single evening.

→ Chapter 1 · Why Verde exists

→ Chapter 2 · The system the website documents

→ Chapter 3 · How the website was built

WHY VERDE EXISTS

Verde starts from three pressures: shrinking farmland, falling water tables and food that never reaches a plate. The website exists because a machine that answers them cannot explain itself by sitting on a table.

1.1 The problem, in the project's own words

The clearest statement of intent in the repository is the site's manifesto (`app/components/Manifesto.tsx`). It is written as an argument, not a feature list:

> **"SOIL IS RUNNING OUT. ARABLE LAND SHRINKS. WATER TABLES FALL. YET A THIRD OF WHAT WE GROW NEVER REACHES A PLATE."**
>
> MANIFESTO SECTION · HEADLINE + REVEAL PARAGRAPH

It then answers itself: *"VERDE is the counter-argument. A sealed, sensor-driven ecosystem that stacks twenty growing sites into the footprint of a bookshelf, recirculates every drop and decides for itself when to feed."* Three problems are named, and each maps to a design decision in the physical system:

PRESSURE	VERDE'S ANSWER	WHERE THE SITE SAYS IT
Land: arable land shrinks	Grow **vertically**: 4 tiers × 5 net pots = 20 sites in about 0.46 m², versus 4.2 m² for the same 20 plants in soil.	Manifesto, Comparison, Hero ("Four tiers · twenty sites")
Water: tables fall	A **sealed, recirculating** hydroponic loop: runoff drains back to the reservoir, using about 95% less water than an equivalent soil bed.	Loop deck "RECLAIM", Tour "The water comes back", FAQ
Waste: a third never reaches a plate	**Attention by machine**: sensors, a leaf-health camera and alerts catch thirst and disease before a person would.	Pillars ("Growing is mostly attention. Verde pays it for you.")

1.2 The idea: a tower that looks after itself

Verde is a **closed-loop vertical hydroponic tower**. Plants grow in net pots with their roots in a nutrient solution instead of soil. A pump lifts water to the top tier and gravity returns it through all four tiers to a sealed reservoir. An ESP8266 microcontroller reads a dozen sensors, decides when to irrigate (and when *not* to, because rain is forecast), dims a twelve-channel LED array by the hour, mirrors everything to Firebase, and messages its owner on WhatsApp in English or Hindi. The pillar headings put it in three sentences: **"It waters itself. It watches the plants. It tells you."**

The ambition behind the stack is **autonomy without dependence**. The FAQ insists that "nothing stops" when Wi-Fi drops, because decisions are made on the device and the cloud is "a reporting channel, not a dependency". That idea, local decisions with optional cloud, recurs throughout both the hardware story and the website's own engineering. Chapter 4 shows that the site was also built to work without third-party services.

1.3 The context: a school science exhibition

The footer states the setting plainly: `© 2026 Verde · CBSE Science Exhibition`. The repository is literally named **Verde-CBSE**. Everything else is consistent with a student exhibition project built in India:

- **A two-person team.** The Team section reads "Two people, one workbench": Anuj Phulera (software and systems) and Aarav Choudhary (hardware and electronics).
- **Located in Delhi.** The hero HUD prints `LAT: 28.6139° N · LON: 77.2090° E` and `TOWER_01 / DELHI`, the clock runs in `Asia/Kolkata`, and a side label reads "EST. 2025 — DELHI, IN".
- **Built for Indian users.** WhatsApp alerts in Hindi and English, an `en_IN` Open Graph locale, rupee costs in the stats API, and a Delhi weather profile.
- **Hosted close to its audience.** `vercel.json` pins the Mumbai region (`bom1`), "the closest edge to the project's audience" (DEPLOYMENT.md).

1.4 Why build a website at all?

An exhibition stand already has the tower, a poster and the team. The repository suggests five reasons a website was worth this much effort (the first four are inferred from how the site is built, the last is stated in the README):

1. **The interesting parts are invisible.** A visitor can see lettuce on shelves but not the sensor bus, the decision rules or the circuit board inside the enclosure. The site makes the inside visible: an orbitable tower, a seedling with its probe exposed, and a PCB that literally *explodes* into its parts.
2. **Judges have minutes, not hours.** The page is a sequenced argument: problem (Manifesto), claim (Stats), mechanism (Loop, Tour), evidence (Console, Telemetry, Comparison), hardware, build log, team. Scrolling top to bottom is a complete five-minute pitch.
3. **It travels.** A URL or QR code puts the whole project on a visitor's phone after they leave the stand, and on any judge's laptop before the exhibition.
4. **It must survive a school network.** The deployment notes stress "no third-party runtime dependencies" and "works behind a firewall": three.js is self-hosted and fonts are bundled. That is exactly the failure mode of exhibition Wi-Fi.
5. **It is proof of skill.** The project began life as `verde-portfolio` (the old package name), and the README calls it "not just a portfolio — a living technical document… The website itself is an engineering artifact."

1.5 The design goals, as written in the code

Developers leave their intentions in comments. These are quoted verbatim from the source; together they describe the brief better than any README:

COMMENT IN THE CODE	FILE	WHAT IT REVEALS
"Light is used sparingly so the 3D models are the brightest thing on the page."	`app/globals.css`	The dark, near-black stage is deliberate. The products are the heroes.
"Emerald as a precision accent (never a wash), oversized display type, hairline structure."	`app/globals.css`	Restraint in colour; drama comes from type scale.
"the boot sequence is part of the product; never flash it away in 200ms"	`ScenePreloader.tsx`	Loading is treated as storytelling, with a 2.6 s minimum.
"never trap the user behind a stalled frame"	`ScenePreloader.tsx`	An 11-second escape hatch if any 3D scene fails.

COMMENT IN THE CODE	FILE	WHAT IT REVEALS
“the model is never re-created, never re-parsed and never re-warmed on scroll”	`ScenePreloader.tsx`	Performance strategy: mount WebGL once, move it around.
“Pure canvas: no shaders, no WebGL context stolen from the product scenes.”	`NeuralField.tsx`	GPU budget is reserved for the products.
“SSR-safe ticker. The server and the client must agree on the FIRST paint…”	`Hero.tsx`	Correct server rendering was a conscious concern.
“Polling is a local Windows-watcher workaround only… it must never reach production.”	`next.config.mjs`	Developed on Windows; production kept clean.

1.6 Who the website is for

AUDIENCE	WHAT THEY NEED	HOW THE SITE SERVES IT
Judges & teachers	A fast, credible understanding of the system and the students’ own contribution.	Sequenced story, spec explorer, build timeline, team roles, FAQ.
Exhibition visitors	A memorable, hands-on experience in under a minute.	Boot sequence, big type, draggable 3D models, an “explode the board” button.
Recruiters & peers	Evidence of engineering depth.	The site itself: architecture, motion and production configuration.
The team	One place that documents the project.	Subsystem specs, timeline and the repository link in the footer.

THE SYSTEM THE WEBSITE DOCUMENTS

Everything the website teaches is about a physical machine: sensors, a microcontroller, actuators, cloud services and a custom circuit board. This chapter assembles the site's scattered descriptions into one picture.

> **IMPORTANT: WHAT THIS REPOSITORY IS (AND ISN'T)**
>
> **Scope:** this repository contains **only the website**. The firmware ("1,200+ lines of C++"), the KiCad PCB files, the TensorFlow Lite model and the Firebase/Twilio integration code are **not** in it. Everything in this chapter is what the website *says* about the physical system. Chapter 11 checks those claims for technical consistency.

2.1 The loop: six steps, forever

The horizontal "loop deck" (`HorizontalDeck.tsx`) describes the control cycle as six cards. It is the best single summary of how Verde behaves:

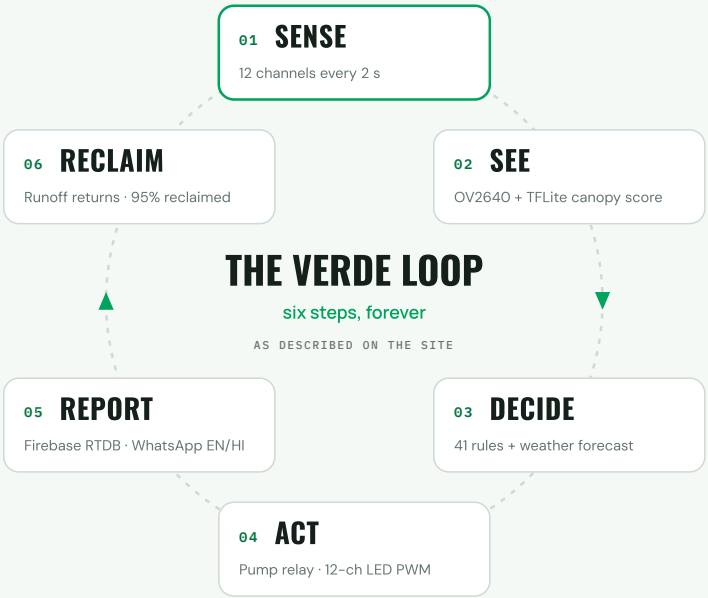


FIGURE 2.1 The control loop as the site describes it. Each step's numbers come from the card's metric chips (for example 12 CH · 0.5 Hz; TFLITE INT8 · 612 KB; 41 RULES; 2.4 L/min; 94 ms sync; 5% loss).

STEP	WHAT HAPPENS	KEY NUMBERS ON THE CARD
01 Sense	Twelve channels sample the bed, the air and the reservoir every two seconds: capacitive soil probes, an RS485 NPK spear, a DHT22 and an ultrasonic level gauge.	12 channels · 0.5 Hz · I²C + RS485
02 See	An OV2640 photographs the canopy hourly; a quantised TensorFlow Lite model scores leaf health, chlorosis and pest damage “on the device itself”.	model v4 · 38 ms · arena 214 KB · 612 KB
03 Decide	A rule engine weighs sensor state against a weather forecast. If rain is likely and the bed is already damp, the cycle is deferred.	41 rules · 0 ms cloud · local fallback
04 Act	An SPDT relay drives the 2.4 L/min pump; twelve MOSFET channels dim the LED array. Every action is logged before it happens.	pump 2.4 L/min · 12-ch PWM · 100% logged
05 Report	State mirrors to Firebase Realtime Database; anything urgent goes out on WhatsApp via Twilio, in English or Hindi, with a photograph.	sync 94 ms · EN/HI · JPEG
06 Reclaim	Runoff drains through all four tiers back to the reservoir. The loop is sealed, so the tower uses about a twentieth of the water a soil bed needs.	5% loss · sealed · 4 tiers

2.2 Architecture of the physical system

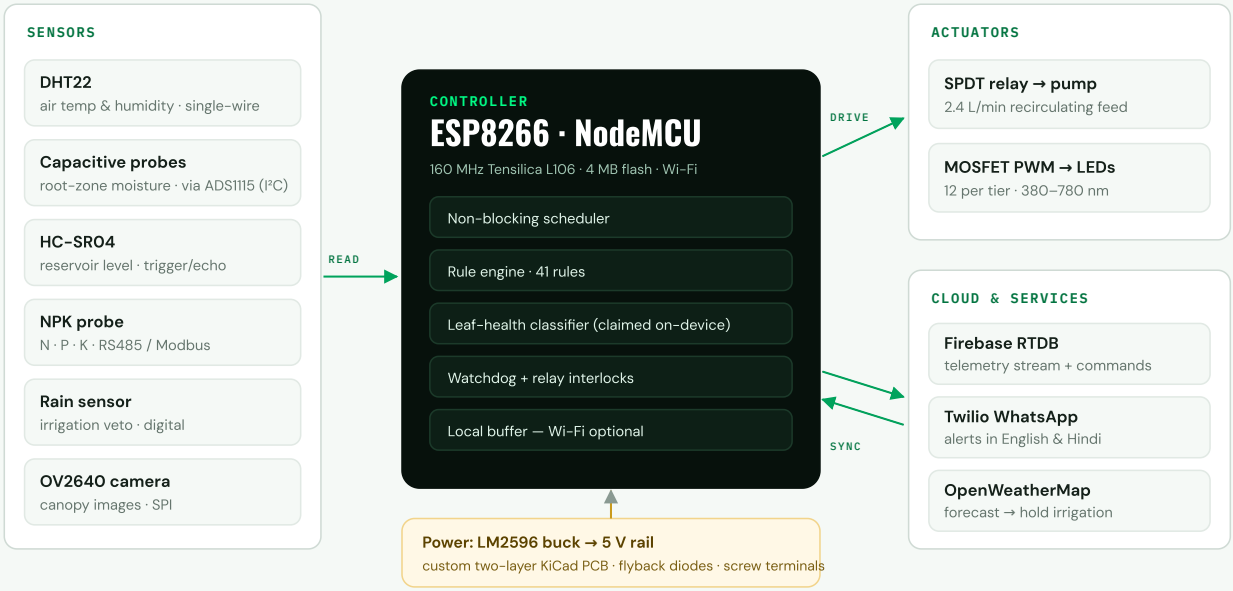


FIGURE 2.2 Physical architecture reconstructed from the Spec Explorer, the boot log, the loop deck and the FAQ. The controller block is dark to show that every decision happens on the ESP8266; the cloud is optional.

2.3 Bill of materials, as listed on the site

The tabbed “Subsystems” explorer (`SpecExplorer.tsx`) lists 18 parts in four groups. Reproduced here with each part’s role:

GROUP	PART	SPECIFICATION ON THE SITE	INTERFACE
Sensing	**DHT22**	Air temperature -40–80 °C ±0.5 °C, RH 0–100 %	“1-Wire”
	Capacitive probe	Root-zone moisture, per growing site	Analog (via ADS1115)
	HC-SR04	Reservoir level 2–400 cm, ±3 mm	Trigger / echo

GROUP	PART	SPECIFICATION ON THE SITE	INTERFACE
	NPK RS485	Nitrogen, phosphorus, potassium	Modbus
	Rain sensor	Suppresses irrigation before rainfall	Digital
	OV2640	2 MP canopy camera for inference	SPI
Compute	**ESP8266**	160 MHz Tensilica L106, 4 MB flash	Controller
	Firmware	1,200+ lines C++, non-blocking scheduler	Embedded
	TensorFlow Lite	Quantised leaf-disease classifier	On-device
	Watchdog	Hardware reset on a hung sensor read	Safety
Power	**LM2596**	Buck converter, 5 V DC rail	Regulation
	Relay K1	SPDT, 1.5 A max, flyback protected	Switching
	LED array	12× per tier, 380–780 nm	MOSFET PWM
	Pump	2.4 L/min recirculating feed	Fluidics
Cloud	**Firebase RTDB**	Telemetry stream + command channel	Realtime
	Twilio	WhatsApp alerts, English and Hindi	Messaging
	OpenWeatherMap	Predictive irrigation hold	Forecast
	Next.js	"This dashboard and documentation", i.e. the website itself	Frontend

2.4 The four tiers: the guided tour's story

The pinned "Guided tour" (`TowerStory.tsx`) flies the 3D camera to a different part of the tower for each chapter. Its four chapters describe the physical layout from top to bottom:

CHAPTER	HEADLINE	WHAT IT EXPLAINS	STAT
Tier 04: Canopy	Light, measured in photons.	A 12-LED array per shelf, MOSFET PWM across 380–780 nm, trimmed hourly so the top tier never scorches and the bottom never stretches.	12× LED per tier
Tier 03: Canopy health	A camera that knows what wilt looks like.	OV2640 + on-device TensorFlow Lite; early chlorosis and leaf-spot markers surface days before they are visible.	0 cloud calls to infer
Tier 02: Root zone	Every site measured on its own.	Capacitive probes and an NPK RS485 bus report per plant; the pump answers to the driest site; the reservoir is tracked to ±3 mm.	±3 mm reservoir
Base: Closed loop	The water comes back.	2.4 L/min through an SPDT relay, recirculated through all four tiers to a sealed reservoir.	95% less water

2.5 How decisions are made (as described)

- **Local first.** "Irrigation and lighting decisions are made on the ESP8266 itself… Readings are buffered locally and flushed once the uplink returns." (FAQ)
- **Weather-aware.** The controller pulls a short-range forecast and holds irrigation if meaningful rain is expected. A physical rain sensor is the fallback "if the API is unreachable". The (simulated) weather API encodes rules such as *pre-irrigate at 22:00 if > 38 °C* and *suspend on rain*.
- **Per-plant.** "The pump answers to the driest site", not to a timer.
- **Safe by design.** A watchdog resets a hung sensor read "so a hung read can't leave the pump on"; relay interlocks; flyback diodes on the relay (Timeline, Spec Explorer).

- **Private vision.** “No plant imagery ever has to leave the unit” because inference runs next to the camera (FAQ).

2.6 How it was built: the 14-week build log

#	PHASE	WEEKS	WHAT WAS DONE
01	**Breadboard**	1–2	DHT22 and a capacitive probe on a NodeMCU; serial output only, “proving the reads were stable before trusting them”.
02	**Firmware**	3–5	Non-blocking C++ scheduler, debounced sampling, relay safety interlocks, watchdog.
03	**The board**	6–8	Schematic and two-layer KiCad layout: LM2596 buck, flyback diodes, screw terminals.
04	**Cloud + alerts**	9–11	Firebase RTDB telemetry, Twilio WhatsApp alerts (EN/HI), OpenWeatherMap irrigation hold.
05	**Vision**	12–14	Small TensorFlow Lite classifier for leaf-spot and chlorosis, quantised for on-device use.
06	**The tower**	15+	Four tiers, twenty net pots, recirculating feed line, controller mounted within reach.

2.7 The team

MEMBER	ROLE ON THE SITE	OWNS	SKILLS LISTED
Anuj Phulera	Software & systems (README: Project Lead / Software Architect)	Firmware, cloud pipeline, the vision model, and this website	Next.js · Firebase · Embedded C++ · TensorFlow · Python
Aarav Choudhary	Hardware & electronics (README: Hardware Node / PCB Designer)	Schematic, board layout, power delivery and the physical build	PCB design · KiCad · Soldering · ESP8266 · Power electronics

HOW THE WEBSITE WAS BUILT

The repository's history is short but revealing: an earlier, light-themed generation of the site was imported, rebuilt into today's dark "maximalist" edition, debugged and hardened for deployment, all in about eight hours with an AI coding agent as co-author.

3.1 The commit timeline

The full history has 8 commits (the clone was shallow and was un-shallowed for this review). Times are UTC on 8 October 2026; add 5 h 30 m for IST (so the work ran from late evening into the early hours of 9 October in India).

TIME	COMMIT	WHAT CHANGED	SIZE
15:03	`0387c7e`	Initial commit by *arena-ai-coding-agent[bot]*: a one-line README (`# Verde-CBSE`).	1 file
22:04	`0f613fd`	**"Maximalist rebuild"**: imports the whole previous site *and* the new v3 components: Oswald mega type, HUD hero, brutalist grids, manifesto, custom cursor. Also brings the three scenes, vendored three.js, API routes and configs.	124 files, +86,391
22:09	`1717579`	Cinematic boot console, glitch logotype, mega marquees, pinned horizontal loop deck, live streaming terminal, scramble text.	9 files, +581
22:26	`a67983b`	**Sticky fix** (`overflow-x: clip`), and the loop deck rebuilt with active-card focus, parallax, ghost numerals, metric chips and step pips.	2 files, +207
22:31	`5f346d9`	Velocity-reactive marquees, word-by-word TextReveal, live section rail, soil-vs-Verde comparison, loaded hero HUD, colossal footer.	8 files, +463
22:34	`a343053`	**Hydration fix**: `useTicker` no longer generates random values during server render.	1 file
22:38	`592fce4`	**Deployment readiness**: delete 66 unreachable files and 6 unused packages; production next.config; metadata, robots and sitemap; vercel.json; DEPLOYMENT.md.	76 files, −14,460
22:42	`9d5576d`	Pull request #1 "Project Verde v3 — maximalist rebuild, Vercel deployment ready" merged into `main` .	merge

> TOOLING NOTE
>
> Commits 2–7 carry the trailer `Co-authored-by: arena-agent` , and the first commit was made by `arena-ai-coding-agent[bot]` . The website was developed **with an AI coding agent**, directed by the project lead. That is worth stating openly if asked. What matters for a viva is that the team can explain every decision, which is what this document is for.

3.2 Generation 2: what came before

The big import commit shows the earlier site that v3 replaced. It was a light-themed portfolio (package `verde-portfolio` , theme colour `#e9ebe8` , title "A little space. A lot of life.") organised into 20 page sections and 40 components:

KIND	FILES IN GENERATION 2 (ALL REMOVED IN V3)
Sections (20)	About, Awards, BootSequence, CTA, Code, Contact, Dashboard, Footer, Gallery, Hardware, Hero, Infographic, Intelligence, Modes, PlantLibrary, Projects, Skills, Team, Testimonials, UseCase
Notable components	**Scene3D** and **VerdeTower** (React Three Fiber; VerdeTower was a port of model1.html), **KioskMode** (`?kiosk=true` auto-tour for an unattended stand), **EasterEggs** (Konami code, CRT/typewriter modes), **QRCodeSection** (a placeholder, not a real QR encoder), **ThemeToggle**, DNAHelix, ParticleCanvas, LivingCircuitBoard, DataPacketFlow, PlantGrowthAnimation…
Hooks (5)	useKeyboardNavigation, useMagneticEffect, usePerformanceTier (low/medium/high), useReducedMotion, useScrollProgress

The old Awards and Testimonials sections contained exhibition-style content (prizes, quotes from named mentors). None of it survives in v3, and this document makes no claim about whether it was real.

3.3 Generation 3: the maximalist rebuild

Version 3.0.0 (the current `main`) keeps the best engineering from generation 2: the three iframe scenes, the scene preloader and the API routes. It rebuilds everything visible in a new register: **black + acid green (#00FF87) with lime (#CCFF00) as a second voice**, Oswald display type at up to 17rem, HUD chrome, brutalist one-pixel grids, a cinematic boot sequence and heavy scroll choreography. The pull request describes the hero as "*3D-free*… carried by typography and motion", so the products are revealed later, in context.

3.4 Three bugs that were fixed along the way

These fixes are excellent viva material because each teaches a real web-platform lesson.

A · HYDRATION MISMATCH (COMMIT A343053)

Next.js renders the page to HTML on the server, then React "hydrates" it in the browser and expects the first client render to produce the *same* markup. The hero's ticker called `useState(fn)`, which ran `Math.random()` and `new Date()` during the server render, so the server and browser disagreed and React threw a hydration error. The fix renders a stable placeholder (`"2.34"`, `"--:--:--"`) and starts generating values only after mount, inside `useEffect`.

B · `position: sticky` SILENTLY BROKEN (COMMIT A67983B)

The pinned sections (loop deck and tour) use `position: sticky`. They did not stick because `body { overflow-x: hidden }` turns `body` into a scroll container, so sticky children stick inside `body`, not the viewport. Replacing it with `overflow-x: clip` clips horizontal overflow *without* creating a scroll container. The pull request also records that `scroll-behavior: smooth` was removed because it fought Lenis.

C · DEPLOYMENT HARDENING (COMMIT 592FCE4)

Not a bug as such, but the moment the project became production-grade: dead code removed, dev-only webpack polling fenced off from production builds, cache and security headers added, and the viewport colour corrected "from the dead light palette to #000 / dark".

3.5 Claims in the pull request, re-verified

PR #1 SAYS	RE-CHECKED ON 18 OCT 2026	RESULT
`tsc --noEmit` clean	Ran with TypeScript 5.3.3 after `npm ci`	✓ exit 0
`next lint` zero warnings	ESLint 8.57 + next/core-web-vitals	✓ "No ESLint warnings or errors"

PR #1 SAYS	RE-CHECKED ON 18 OCT 2026	RESULT
`next build` succeeds, 11/11 static pages	Next 14.2.5 production build (≈18 s)	✓ 11/11 generated
`/` prerendered at 157 kB first load	Build route table	✓ 70.1 kB route + 87 kB shared = 157 kB
`/robots.txt`, `/sitemap.xml`, `/api/health`, `/scenes/*`, `/vendor/*` all 200 with correct headers	curl against `next start`	✓ all 200; headers match next.config
“Pruned 70 unreachable files”	`git show --diff-filter=D 592fce4`	≈ 66 files deleted (40 components, 20 sections, 5 hooks, 1 util)
“No environment variables required”	Built and ran with an empty environment	✓ true

02

PART II

WHAT WAS USED

Every library, every file, every component and every scene: what it does, how it works and why it was chosen over the alternatives.

→ Chapter 4 · The stack at a glance
→ Chapter 5 · Architecture
→ Chapter 6 · The page, section by section
→ Chapter 7 · The three 3D scenes
→ Chapter 8 · The design system
→ Chapter 9 · API, SEO, config & deployment

THE STACK AT A GLANCE

Ten runtime packages, ten development tools, a vendored 3D engine and the browser platform itself. Each choice below is paired with the reason it fits this project.

4.1 Runtime dependencies

PACKAGE	VERSION	WHAT IT DOES IN VERDE	WHY THIS CHOICE
next	14.2.5	Framework: App Router, prerendering, route handlers (`/api/*`), Metadata API, `robots.ts` / `sitemap.ts`, production server.	The whole page can be **prerendered to static HTML** and served from a CDN; SEO metadata is first-class; deploys to Vercel with zero config.
react · **react-dom**	18.2.0	Component model, hooks, context (the scene "bus").	Required by Next; hooks keep every effect self-contained.
framer-motion	11.0.0	All animation: scroll-linked transforms, springs, scroll velocity, in-view triggers, AnimatePresence, a shared-layout tab pill.	The most complete motion library for React. Its scroll primitives (`useScroll`, `useVelocity`) power most of the page.
lenis	1.3.23	Smooth, eased wheel scrolling and anchor navigation with a 70 px offset.	Gives scroll-driven animation a cinematic feel; small and framework-free.
lucide-react	0.400.0	Exactly **one** icon: `ArrowUpRight` in the closing CTA.	Tree-shaken, so only that icon ships. (Generation 2 used many more.)
@fontsource-variable/dm-sans	^5.3.0	Body text (variable weight).	Self-hosted fonts: no Google request, no layout shift from late fonts, works offline.
@fontsource-variable/manrope	^5.3.0	Headings in the refined layer, the nav wordmark.	Geometric, legible at small sizes; also used in the original scene prototypes.
@fontsource-variable/oswald	^5.3.0	The colossal condensed display type (`.font-display-xl`).	Condensed caps allow 11–17rem headlines without overflowing (mostly; see §11.8).
@fontsource/ibm-plex-mono	^5.3.0 (400, 500)	HUD readouts, eyebrows, the terminal, code-like labels.	A monospace "instrument" voice that sells the hardware theme.

4.2 Development tooling

PACKAGE	VERSION	ROLE
typescript	5.3.3	Strict type-checking (`"strict": true`, target ES2017, `moduleResolution: bundler`). A `@/*` path alias is configured but never used.
tailwindcss + **postcss** + **autoprefixer**	3.4.0 · 8.4.33 · 10.4.17	Utility CSS, used heavily with arbitrary values such as `text-[clamp(44px,9vw,9rem)]`. The custom theme in `tailwind.config.ts` is unused (§10.3).
eslint + **eslint-config-next**	8.57.0 · 14.2.5	`next/core-web-vitals` rules; two deliberate `exhaustive-deps` suppressions (Hero, ScrambleText).
@types/node, **@types/react**, **@types/react-dom**	20.11 · 18.2	Type definitions.

PACKAGE	VERSION	ROLE
rimraf	^6.1.3	`npm run dev` deletes `.next` first: a clean dev start, useful on Windows where the file watcher misbehaves.

4.3 Not in package.json but essential

TECHNOLOGY	WHERE	NOTES
three.js r162 (ES module build)	`public/vendor/three.module.js` + 4 addons	Used by the tower and pod scenes through an import map. Addons: OrbitControls, RoomEnvironment, RoundedBoxGeometry, BufferGeometryUtils. 1.29 MB (261 KB gzipped).
three.js r128 (legacy UMD build)	`public/vendor/legacy/three.min.js` + OrbitControls	Used only by the board scene, which was written against the old global- `THREE` API. 603 KB.
Vercel	`vercel.json`	Hosting, CDN, serverless functions; region `bom1` (Mumbai).
Node.js 20	`.nvmrc`, `engines ≥ 18.18`	Build/runtime version pin.
Google Search Console	`public/google280c2ec99007341d.html`	An HTML verification file proves site ownership to Google.

4.4 Browser platform features used

Much of the site's character comes straight from the web platform rather than libraries:

- **WebGL** via three.js, inside three iframes.
- **Canvas 2D**: the hero's particle lattice (NeuralField), plus every texture painted at runtime inside the scenes (PCB traces, leaves, labels, the tower display).
- **SVG**: sparklines, the reservoir gauge, the leaf logo, the film-grain filter (`feTurbulence`).
- **window.postMessage**: the bridge between the page and each scene.
- **requestAnimationFrame**: the scene glue loop, Lenis, canvas loops, marquees.
- **IntersectionObserver**: in-view triggers, the side rail's active section.
- **matchMedia**: `prefers-reduced-motion` , `(hover: hover) and (pointer: fine)` .
- **Modern CSS**: `clamp()` , `svh` units, `overflow: clip` , `text-wrap: balance/pretty` , `-webkit-text-stroke` outline type, `mix-blend-mode` , `backdrop-filter` , `writing-mode` , `mask-image` .
- **Intl / Date**: live IST clocks via `toLocaleTimeString(…, {timeZone: "Asia/Kolkata"})` .

4.5 Scripts

COMMAND	RUNS	PURPOSE
`npm run dev`	`rimraf .next && next dev`	Clean development server on :3000.
`npm run build`	`next build`	Production build (prerenders 11 routes).
`npm start`	`next start`	Serve the production build.
`npm run lint`	`next lint`	ESLint.
`npm run typecheck`	`tsc --noEmit`	Type errors only, no output.
`npm run verify`	typecheck → lint → build	The pre-push gate described in DEPLOYMENT.md.

WHY IT WAS DONE THIS WAY

Every dependency serves something visible on the page. Nothing is installed “just in case”: the deployment pass removed six packages that had stopped earning their place (Chapter 10). The heaviest asset, three.js, is deliberately kept *out* of the JavaScript bundle and cached for a year, so the page itself stays at 157 kB.

ARCHITECTURE

One static page, thirty-one client components, three isolated WebGL documents and a tiny message protocol between them. The central idea is the scene preloader, and it is worth understanding in detail.

5.1 Repository map

Repository tree (73 tracked files) annotated

```
Verde-CBSE/
├─ app/                          Next.js App Router
│  ├─ layout.tsx                 <html>, fonts, SEO metadata, viewport (server component)
│  ├─ page.tsx                   section order + global layers (server component)
│  ├─ globals.css                602 lines: design tokens, components, effects
│  ├─ robots.ts · sitemap.ts     generated /robots.txt and /sitemap.xml
│  ├─ api/{contact,health,stats,telemetry,weather}/route.ts   5 simulated endpoints
│  ├─ components/                29 client components (+ site/Nav, site/Footer)
│  └─ google280c2ec99007341d.html   duplicate of the verification file (not served)
├─ public/
│  ├─ scenes/{tower,pod,board}.html  the three WebGL scenes (2,551 lines)
│  ├─ vendor/three.module.js + addons/ + legacy/   vendored three.js r162 + r128
│  ├─ favicon.svg · manifest.json · google…html
├─ model1.html · model2.html · model3.html   original light-theme prototypes (not served)
├─ docs/3d-model-prompts.md      prompts for AI-generated GLB models (never used)
├─ README.md · DEPLOYMENT.md
└─ package.json · tsconfig.json · next.config.mjs · tailwind.config.ts · postcss.config.mjs
   .eslintrc.json · vercel.json · .nvmrc · .env.example · .gitignore
```

5.2 Rendering model

`app/layout.tsx` and `app/page.tsx` are **server components**: they contain no state and run only at build time. Every one of the 31 components they render starts with `"use client"`, because every one uses hooks, the DOM or animation. At build time Next renders the whole tree to static HTML (the `○ /` in the build report), and the browser receives a complete page that React then **hydrates**, attaching behaviour without re-rendering it. The 3D scenes are not React at all: they are static HTML documents in `public/scenes`, loaded by iframes.

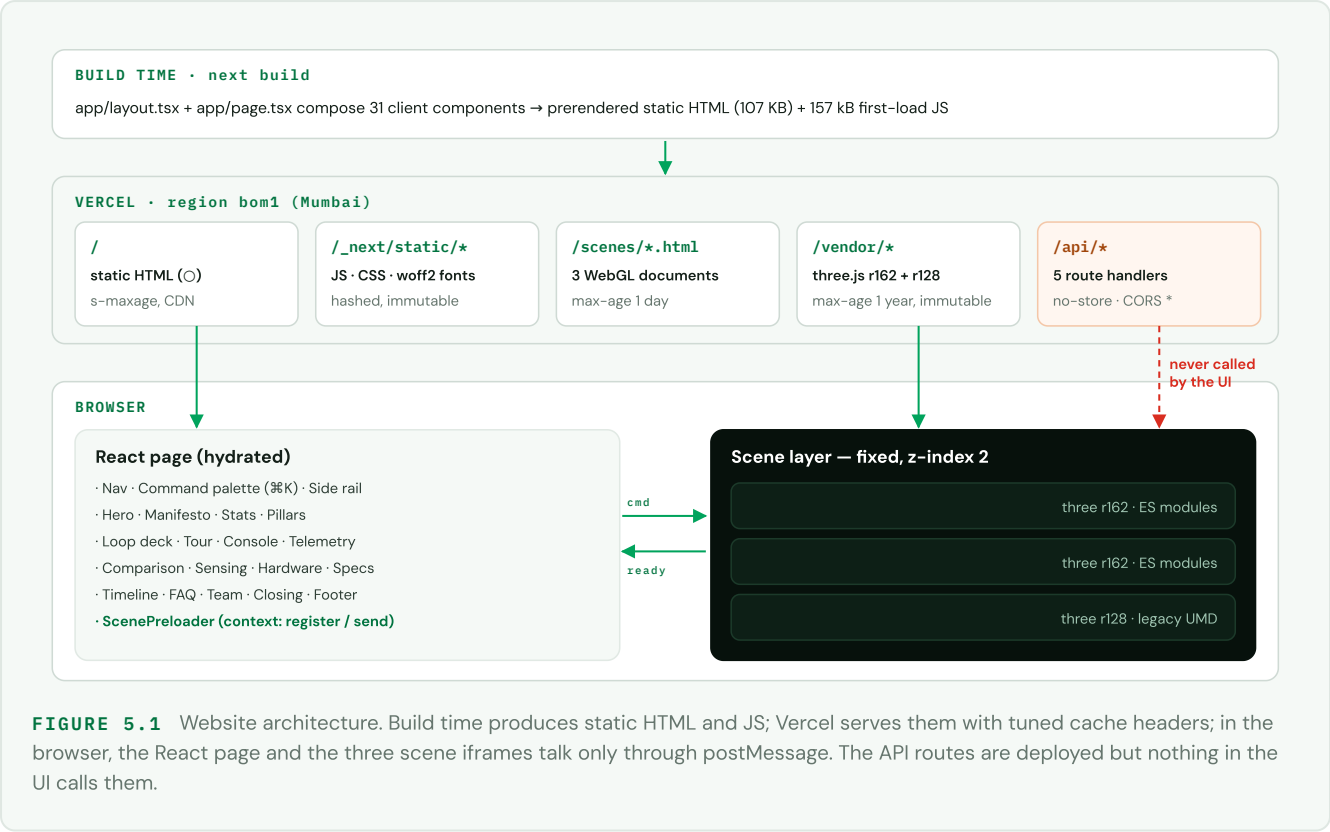


FIGURE 5.1 Website architecture. Build time produces static HTML and JS; Vercel serves them with tuned cache headers; in the browser, the React page and the three scene iframes talk only through postMessage. The API routes are deployed but nothing in the UI calls them.

5.3 The page composition

`app/page.tsx` is the table of contents of the website. It wraps everything in `<ScenePreloader>`, mounts seven global pieces (smooth scroll, progress bar, cursor, side rail, noise, scanline, nav), then lists 20 page blocks in order:

app/page.tsx lines 26–82 whole render tree

```tsx
export default function Home() {
  return (
    <ScenePreloader>
      <SmoothScroll />
      <ScrollProgress />
      <CustomCursor />
      <SideRail />
      <div className="bg-noise" />
      <div className="scanline-fx" />
      <Nav />

      <main id="main" className="relative z-10">
        <Hero />

        <VelocityMarquee
          items={["AUTONOMOUS", "SEALED LOOP", "ON-DEVICE AI", "ZERO SOIL"]}
          baseVelocity={2.2}
        />

        <Manifesto />
        <StatsStrip />
        <Pillars />

        <VelocityMarquee
          items={["95% LESS WATER", "20 SITES", "4 TIERS", "1 FOOTPRINT"]}
          baseVelocity={-1.8}
        />

        <HorizontalDeck />
        <TowerStory />
        <LiveTerminal />
        <Telemetry />
        <Comparison />
        <SensingSection />
        <BoardSection />

        <VelocityMarquee
          items={["ESP8266", "TENSORFLOW LITE", "FIREBASE", "RS485 NPK", "KICAD PCB"]}
          baseVelocity={2.8}
        />

        <SpecExplorer />
        <Timeline />
        <Faq />
        <TeamSection />
        <Closing />
      </main>

      <Footer />

      {/* keeps the tower warm for the hero-less opening; never visible */}
      <div className="sr-only" aria-hidden>
        <SceneSlot id="tower" className="w-0 h-0" />
      </div>
    </ScenePreloader>
  );
}
```

5.4 The layer stack

Many effects depend on stacking order. The most important (and least obvious) fact: **the 3D scenes live *below* the content**, in a fixed layer at `z-index: 2`, while `<main>` is at `z-index: 10`. A model is visible only because the section above it is transparent where its slot is. Overlays such as noise, scanline and cursor sit at the very top with `pointer-events: none` so they never block clicks.



FIGURE 5.2 Every fixed or stacked layer on the page, from the emerald horizon painted on body::before (bottom) to the custom cursor (top).

5.5 The ScenePreloader: the central idea

The problem it solves: a WebGL scene is expensive to start. It must download and parse three.js, build geometry, compile shaders and allocate GPU memory. If each section simply rendered its own `<iframe>`, scrolling would repeatedly create and destroy those contexts, or keep three of them alive in the wrong places. **The solution:** mount all three scenes *once*, at load time, in a fixed layer, and physically move each iframe over whichever placeholder (“slot”) currently needs it.



FIGURE 5.3 Slot claiming. Sections render empty SceneSlot placeholders; on every frame the preloader moves each scene's holder over the slot nearest the viewport centre, or hides it when that slot is far off-screen.

The heart of it is a `requestAnimationFrame` loop that runs for the lifetime of the page:

app/components/ScenePreloader.tsx lines 87–124 — the glue loop

```tsx
/* ---- keep each live iframe glued to whichever slot claimed it ---- */
useEffect(() => {
  let raf = 0;
  const sync = () => {
    (Object.keys(holders.current) as SceneId[]).forEach((id) => {
      const holder = holders.current[id];
      const list = (slots.current[id] ?? []).filter((n) => n.isConnected);
      let slot: HTMLElement | null = null;
      let best = Infinity;
      const mid = window.innerHeight / 2;
      for (const n of list) {
        const r = n.getBoundingClientRect();
        if (r.width < 2 || r.height < 2) continue;
        const d = Math.abs(r.top + r.height / 2 - mid);
        if (d < best) {
          best = d;
          slot = n;
        }
      }
      if (!holder) return;
      if (!slot) {
        holder.style.opacity = "0";
        holder.style.pointerEvents = "none";
        return;
      }
      const r = slot.getBoundingClientRect();
      const offscreen = r.bottom < -400 || r.top > window.innerHeight + 400;
      holder.style.transform = `translate3d(${r.left}px, ${r.top}px, 0)`;
      holder.style.width = `${r.width}px`;
      holder.style.height = `${r.height}px`;
      holder.style.opacity = offscreen ? "0" : "1";
      holder.style.pointerEvents = offscreen ? "none" : "auto";
    });
    raf = requestAnimationFrame(sync);
  };
  raf = requestAnimationFrame(sync);
  return () => cancelAnimationFrame(raf);
}, []);
```

SLOTS REGISTER THEMSELVES

`SceneSlot` is just an empty `<div>` that calls `register(id, node)` on mount and `unregister` on unmount, through React context. A scene can have several slots; the one nearest the viewport centre wins.

WHY IT IS CLEVER

- **Zero re-initialisation:** scrolling never reloads a model, so state (rotation, exploded view) survives.
- **Layout stays React's job:** sections size their slot with ordinary CSS such as `h-[86vh]`; the preloader just follows.
- **Cheap per frame:** one `getBoundingClientRect` per slot and a `translate3d` (compositor-only).
- **Graceful:** slots smaller than 2×2 px are ignored, and far-off scenes become invisible and stop receiving pointer events.

> **LIMITATION**
>
> Hiding a scene sets `opacity: 0`, but the scene keeps rendering every frame. See §11.10 for the cost and the fix.

app/components/ScenePreloader.tsx lines 183–201

```tsx
/** A placeholder in the layout that a preloaded
scene gets positioned over. */
export function SceneSlot({
  id,
  className = "",
}: {
  id: SceneId;
  className?: string;
}) {
  const { register, unregister } = useScenes();
  const ref = useRef<HTMLDivElement>(null);

  useEffect(() => {
    const node = ref.current;
    register(id, node);
    return () => unregister(id, node);
  }, [id, register, unregister]);

  return <div ref={ref} className={className} />;
}
```

5.6 The message protocol

Because each scene is a separate document, the page cannot call its functions directly. Instead both sides exchange tiny objects with `window.postMessage`, tagged with `verde` so unrelated messages are ignored.

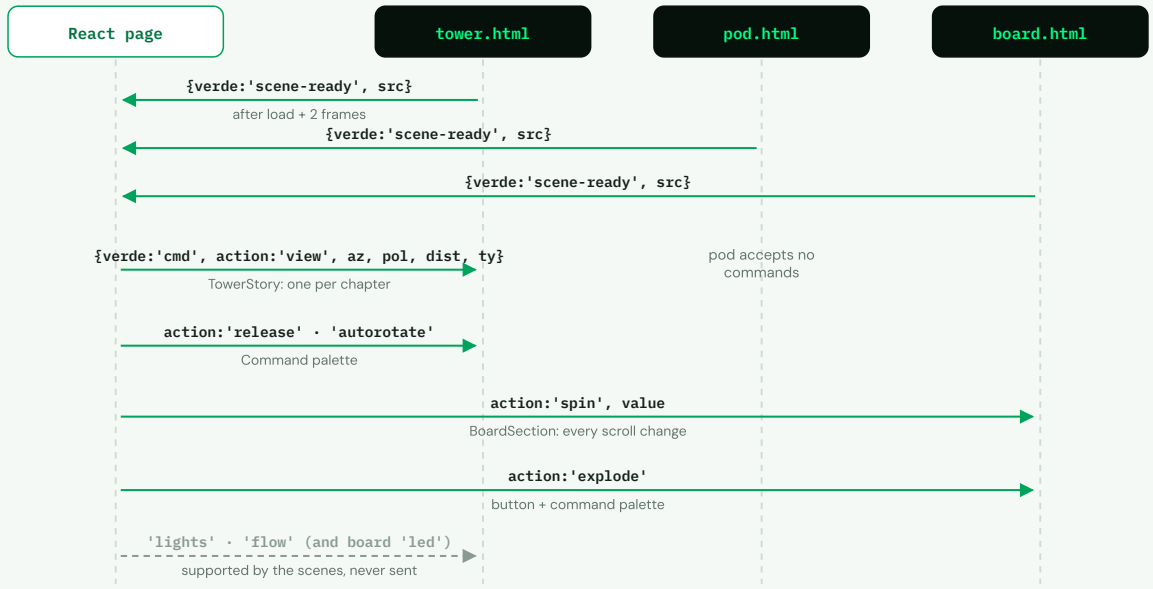


FIGURE 5.4 Every message in the system. Scenes announce readiness; the page sends commands. The grey message shows commands the scenes support that the page never sends.

SCENE	COMMANDS IT ACCEPTS	SENT BY
tower.html	`view {az, pol, dist, ty}` fly the camera (spherical coordinates); `release` return control to the user; `autorotate {value}` ; `lights {value}` ; `flow {value}`	TowerStory (view), CommandPalette (release + autorotate). *lights/flow: never sent.*
pod.html	none (it only posts `scene-ready`)	—

SCENE	COMMANDS IT ACCEPTS	SENT BY
board.html	`spin {value}` rotate the board; `explode` toggle the exploded view; `led` toggle the LED	BoardSection (spin on scroll, explode button), CommandPalette (explode). *led: never sent.*

Inside the tower, the remote control uses **exponential smoothing**: each frame moves the camera 8.5% of the remaining distance toward the requested view, which produces the soft "flight" between tour chapters:

public/scenes/tower.html lines 381–429 remote-control API

```
/* ---- Verde remote control API (driven by the React app) ---- */
(function(){
  const want = { az: Math.atan2(startPosition.x, startPosition.z), pol: 1.08, dist: 12.2, ty: 2.13 };
  const cur  = Object.assign({}, want);
  let driven = false;
  const sph = new THREE.Spherical();
  const tmp = new THREE.Vector3();

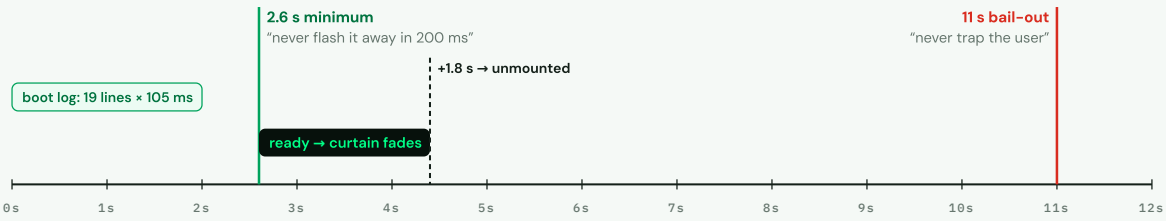
  function apply(){
    controls.target.y = cur.ty;
    sph.set(cur.dist, cur.pol, cur.az);
    tmp.setFromSpherical(sph).add(new THREE.Vector3(0, cur.ty, 0));
    camera.position.copy(tmp);
    camera.lookAt(0, cur.ty, 0);
  }

  function tick(){
    if (driven){
      const k = 0.085;
      cur.az  += (want.az  - cur.az)  * k;
      cur.pol += (want.pol - cur.pol) * k;
      cur.dist+= (want.dist- cur.dist)* k;
      cur.ty  += (want.ty  - cur.ty)  * k;
      apply();
    }
    requestAnimationFrame(tick);
  }
  requestAnimationFrame(tick);

  addEventListener('message', function(e){
    const d = e.data;
    if (!d || d.verde !== 'cmd') return;
    if (d.action === 'view'){
      driven = true;
      controls.autoRotate = false;
      controls.enabled = false;
      if (typeof d.az === 'number')   want.az = d.az;
      if (typeof d.pol === 'number')  want.pol = d.pol;
      if (typeof d.dist === 'number') want.dist = d.dist;
      if (typeof d.ty === 'number')   want.ty = d.ty;
    }
    if (d.action === 'release'){ driven = false; controls.enabled = true; }
    if (d.action === 'autorotate'){ controls.autoRotate = !!d.value; }
    if (d.action === 'lights'){
      lightsOn = !!d.value; updateDisplay(lightsOn, flowOn);
    }
    if (d.action === 'flow'){ flowOn = !!d.value; updateDisplay(lightsOn, flowOn); }
  });
})();
```

5.7 The boot sequence

While the scenes load, `BootCurtain.tsx` covers the page with a black "operating system boot": a glitching **VERDE** logotype, a three-digit percentage, a `MEM_ALLOC` readout that changes every 55 ms, and a 19-line console log streamed every 105 ms ("mount /dev/esp8266 @ 160MHz … OK"). The rules that decide when it lifts are simple and well-reasoned:




```
ready = (all 3 scenes posted "scene-ready" AND ≥ 2.6 s elapsed) OR 11 s elapsed
counter = smoothed % that may climb only to 92% of real progress until ready, then to 100%
```

FIGURE 5.5 When the curtain lifts. The percentage is "honest-ish": it is capped at 92% of real progress until every scene has reported, so it never reaches 100% early, and it never sits at 0.

5.8 Why iframes instead of React Three Fiber?

Generation 2 *did* try React Three Fiber: `VerdeTower.tsx` was a 579-line R3F port of the tower prototype, and `Scene3D.tsx` used `@react-three/drei`. Both were deleted, along with the `three`, `@react-three/fiber` and `@react-three/drei` packages. Weighing the two approaches explains why:

	IFRAMES + VENDORED THREE.JS (CHOSEN)	REACT THREE FIBER IN THE BUNDLE
Bundle size	**0 KB** added to the page JS; three.js downloads in parallel and is cached for a year	three.js + R3F + drei inside the app bundle
Isolation	Each scene has its own JS realm and WebGL context, so a crash or slow shader cannot break the page	One error can take down the React tree
Versions	Scenes can use different three.js versions (r162 and r128 coexist)	One version for everything
Workflow	Each scene is a standalone HTML "product viewer", openable directly (/scenes/tower.html)	Must run inside the app
Porting cost	Prototypes reused almost verbatim (Chapter 7 shows the diff)	Rewrite into JSX components
Costs	Three contexts and two copies of three.js (≈1.9 MB raw, cached); communication only via messages; iframe pitfalls (§11.3)	Shared state is easy; one context possible

WHY IT WAS DONE THIS WAY

For a showcase with three self-contained "product viewers" and a performance budget for the page itself, iframes are the pragmatic choice. They kept the beautiful prototypes intact and made the page load as fast as a text page. The only thing lost is easy two-way state, which the small postMessage protocol covers.

5.9 Global behaviours

COMPONENT	WHAT IT DOES	DETAIL WORTH KNOWING
`SmoothScroll.tsx`	Creates a Lenis instance (duration 1.15 s, exponential ease-out, touch multiplier 1.6) and routes in-page `#anchor` clicks through it with a -70 px offset for the fixed nav.	Disabled completely under `prefers-reduced-motion`.

COMPONENT	WHAT IT DOES	DETAIL WORTH KNOWING
`ScrollProgress.tsx`	A 2 px gradient bar across the top, scaled by scroll progress through a spring.	16 lines: `useScroll` → `useSpring` → `scaleX`.
`CustomCursor.tsx`	A 6 px white dot (`mix-blend-difference`) and a 28 px spring-following ring that grows to 52 px and turns green over links, buttons and the 3D iframes.	Only on `(hover: hover) and (pointer: fine)`, so touch devices keep the native cursor.
`SideRail.tsx`	A fixed index of 11 sections on the left (≥ 1280 px wide) that highlights the section in the middle 10% of the screen.	IntersectionObserver with `rootMargin: -45% 0px -45% 0px`.
`site/Nav.tsx`	Fixed header that turns frosted (`backdrop-blur-xl`) after 24 px of scroll, with 8 anchor links, a ⌘K search chip and a two-line mobile menu.	The logo is an inline SVG leaf (the same drawing appears in the original tower prototype's header). The favicon is a different mark (§11.9).
`CommandPalette.tsx`	⌘K / Ctrl+K palette with 10 commands: jump to sections, toggle tower rotation, explode the board, open GitHub.	Arrow-key navigation, Enter to run, Escape to close, live filtering.

THE PAGE, SECTION BY SECTION

Twenty blocks, top to bottom, exactly as a visitor meets them. For each: what you see, how it works, and the file that does it. All screenshots are real captures of the production build at 1440×900.

6.0 The page map

Measured in the browser, the page is **21,278 px tall at 1440×900, or 23.6 screens**. Two pinned (sticky) sections account for 8.2 screens, **35% of all scrolling**, because they turn vertical scroll into horizontal travel and camera motion. Three sections host live 3D, and three marquee strips (lime) act as chapter breaks. The section IDs double as navigation targets: the nav bar links to 8 of them, the side rail to 11, the command palette to 6 and the footer to 11.

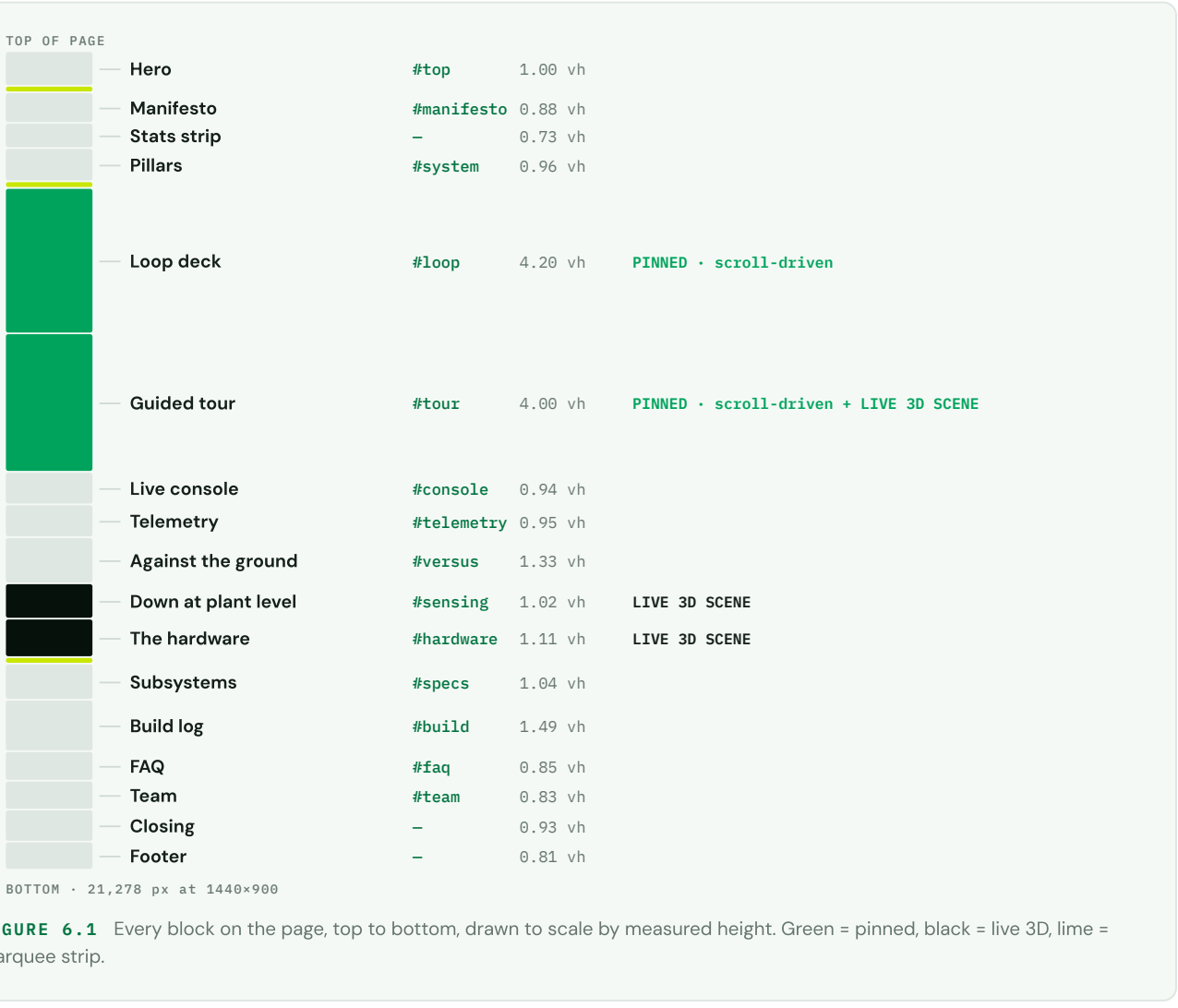


FIGURE 6.1 Every block on the page, top to bottom, drawn to scale by measured height. Green = pinned, black = live 3D, lime = marquee strip.

6.1 Boot curtain

File **BootCurtain.tsx · 127 lines** Shown **until all 3 scenes report (min 2.6 s, max 11 s)**

Tech **setInterval, CSS glitch keyframes**

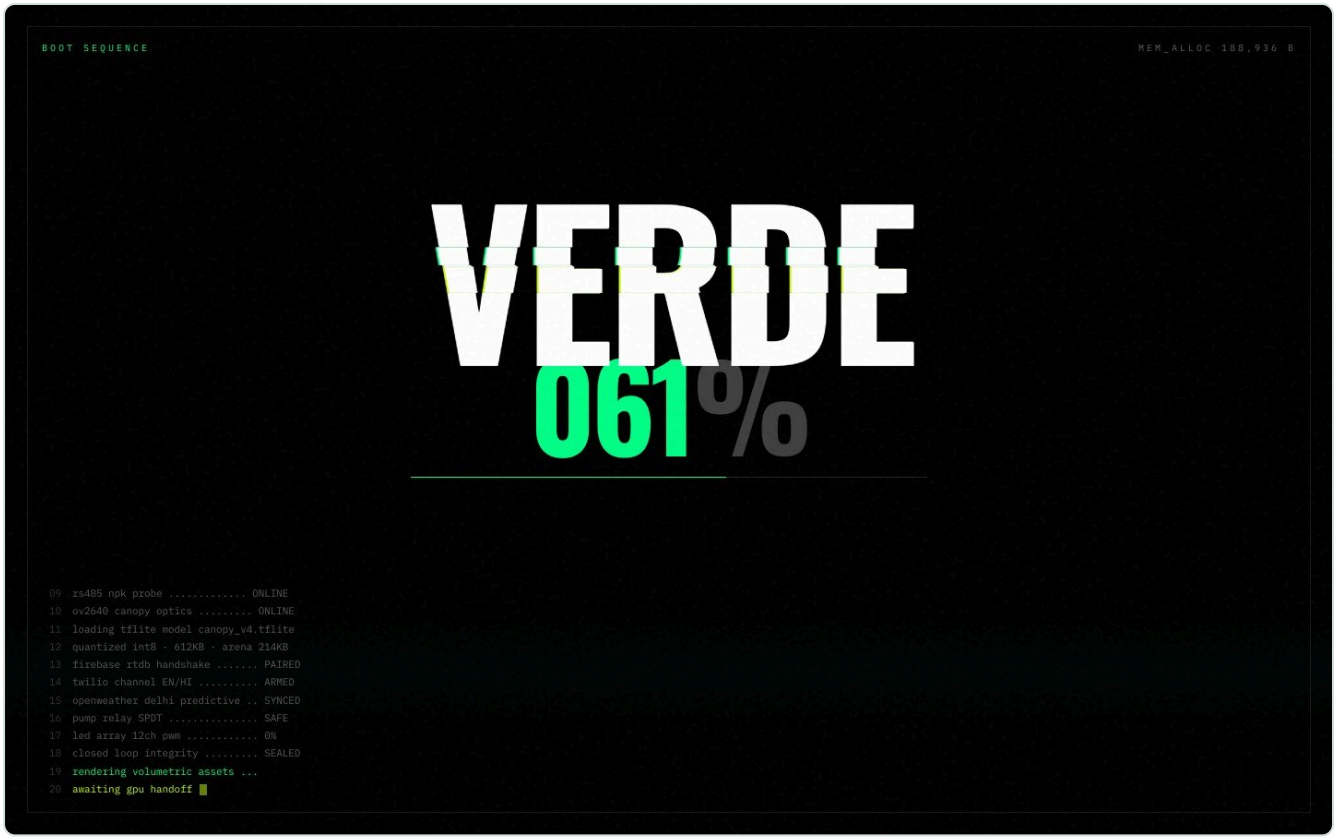


FIGURE 6.2 The boot curtain mid-sequence. The percentage waits at ~61% because one scene has not yet reported (it may only reach 92% of real progress until all three are ready).

What you see: a black screen inside a thin HUD frame, “BOOT SEQUENCE” top-left, a flickering `MEM_ALLOC 241,539 B` top-right, a huge glitching **VERDE**, a green counter with a hairline progress bar, and a console that types out a fictional firmware boot (“dht22 temp/rh … ONLINE”, “quantized int8 · 612KB · arena 214KB”, “twilio channel EN/HI … ARMED”) ending in a blinking “awaiting gpu handoff”.

How it works: the log is a constant array revealed one line every 105 ms. The counter is smoothed every 55 ms by a random step of up to 3.4 toward a *ceiling*: 92% of the real scene progress until ready, then 100%. The glitch is pure CSS: two pseudo-elements with `content: attr(data-text)` offset by ±2 px, tinted lime and green, and clipped by animated `clip-path: inset()` keyframes. The curtain fades out once ready and the counter passes 99, then unmounts 1.8 s after ready.

> **WHY IT WAS DONE THIS WAY**
>
> Three WebGL scenes need time to warm up. Rather than show a blank page or pop-in, the wait becomes part of the story: the site “boots” like the device it describes. The minimum duration exists so it never looks like a glitch on fast connections.

6.2 Hero: “We grow the future.”

Files **Hero.tsx 187 · NeuralField.tsx 142 · ScrambleText.tsx 87 · Magnetic.tsx 40** Height **100svh**

Tech **Canvas 2D, framer-motion scroll transforms**

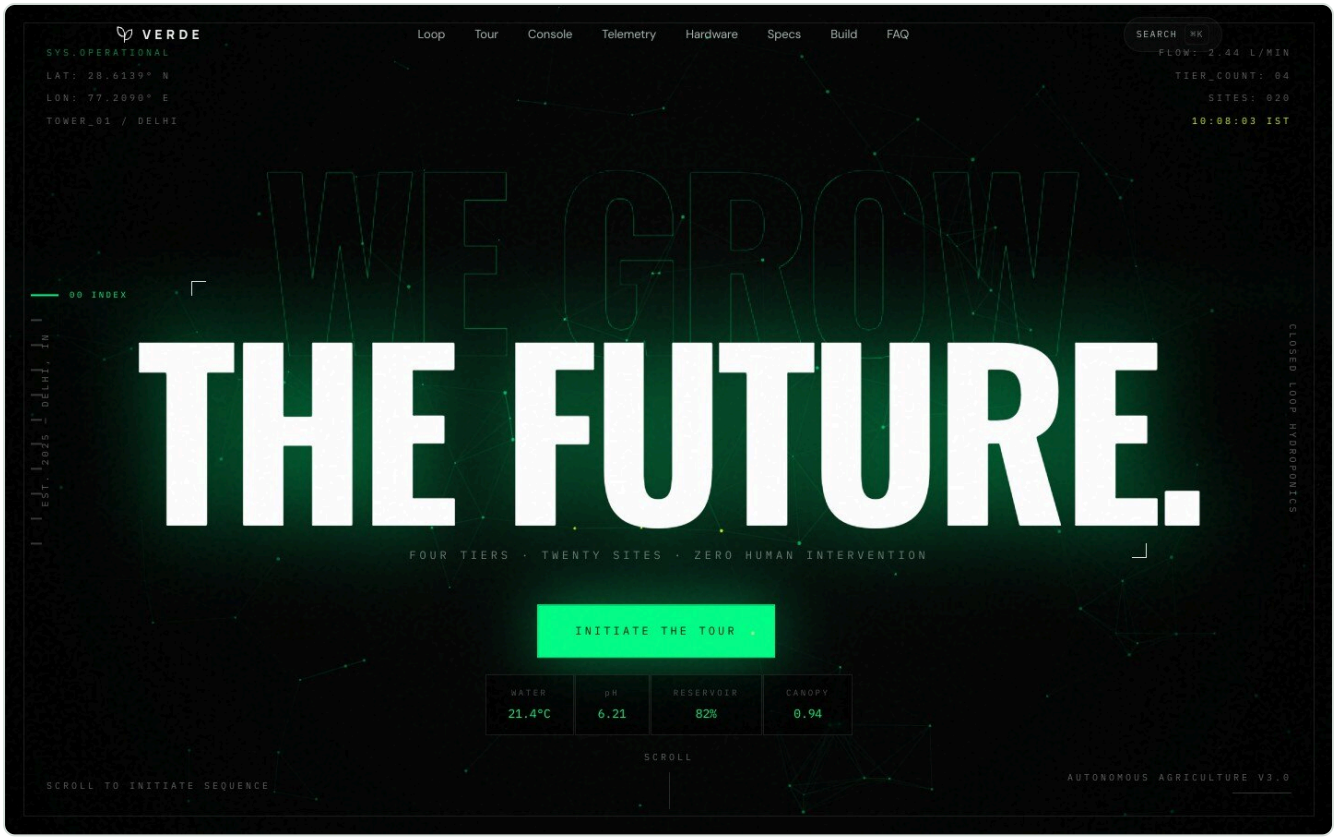


FIGURE 6.3 The hero on desktop: outlined “WE GROW” over a solid glowing “THE FUTURE.”, a HUD frame with live readouts, the magnetic CTA (here hovered, so the custom cursor ring has grown and turned green) and four telemetry cards.

What you see: the largest type on the site (`clamp(64px, 17vw, 17rem)`) over a drifting green “mycelium” network. The HUD frame lists `SYS.OPERATIONAL`, Delhi’s coordinates, a live `FLOW: 2.44 L/MIN` that jitters every 0.9 s, `TIER_COUNT: 04 · SITES: 020`, and a ticking IST clock. Below: the line “Four tiers · twenty sites · zero human intervention”, an “Initiate the tour” button, four cards (water 21.4 °C, pH 6.21, reservoir 82%, canopy 0.94) and a scrambling “SCROLL” cue.

How it works: as you scroll away, the headline block moves down 160 px, fades and blurs to 10 px (`useScroll` on the section → `useTransform`). **NeuralField** scatters 50–190 particles (density 0.00011 per px²), links every pair closer than 132 px with a line whose alpha falls with distance, and pushes particles away from the cursor. Particles near the pointer turn lime. Under reduced motion it draws one frame and stops.

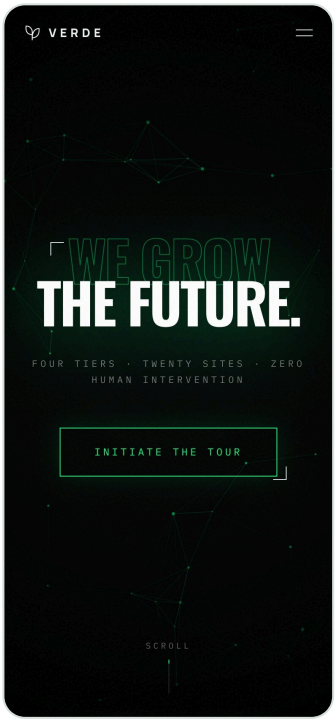


FIGURE 6.4 Same hero on a 390×844 phone: HUD frame and cards hidden below the md breakpoint.

app/components/Hero.tsx lines 9–24 the hydration-safe ticker (see §3.4)

```tsx
/**
 * SSR-safe ticker. The server and the client must agree on the FIRST paint, so
 * we render a fixed placeholder and only start generating values after mount.
 */
function useTicker(fn: () => string, ms = 1000, initial = "—") {
  const [v, setV] = useState(initial);

  useEffect(() => {
    setV(fn());
    const id = setInterval(() => setV(fn()), ms);
    return () => clearInterval(id);
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, [ms]);

  return v;
}
```

6.3 Navigation and the command palette

Files **site/Nav.tsx 96 · CommandPalette.tsx 176** Shortcut **⌘K / Ctrl+K** Tech **AnimatePresence, keyboard events**

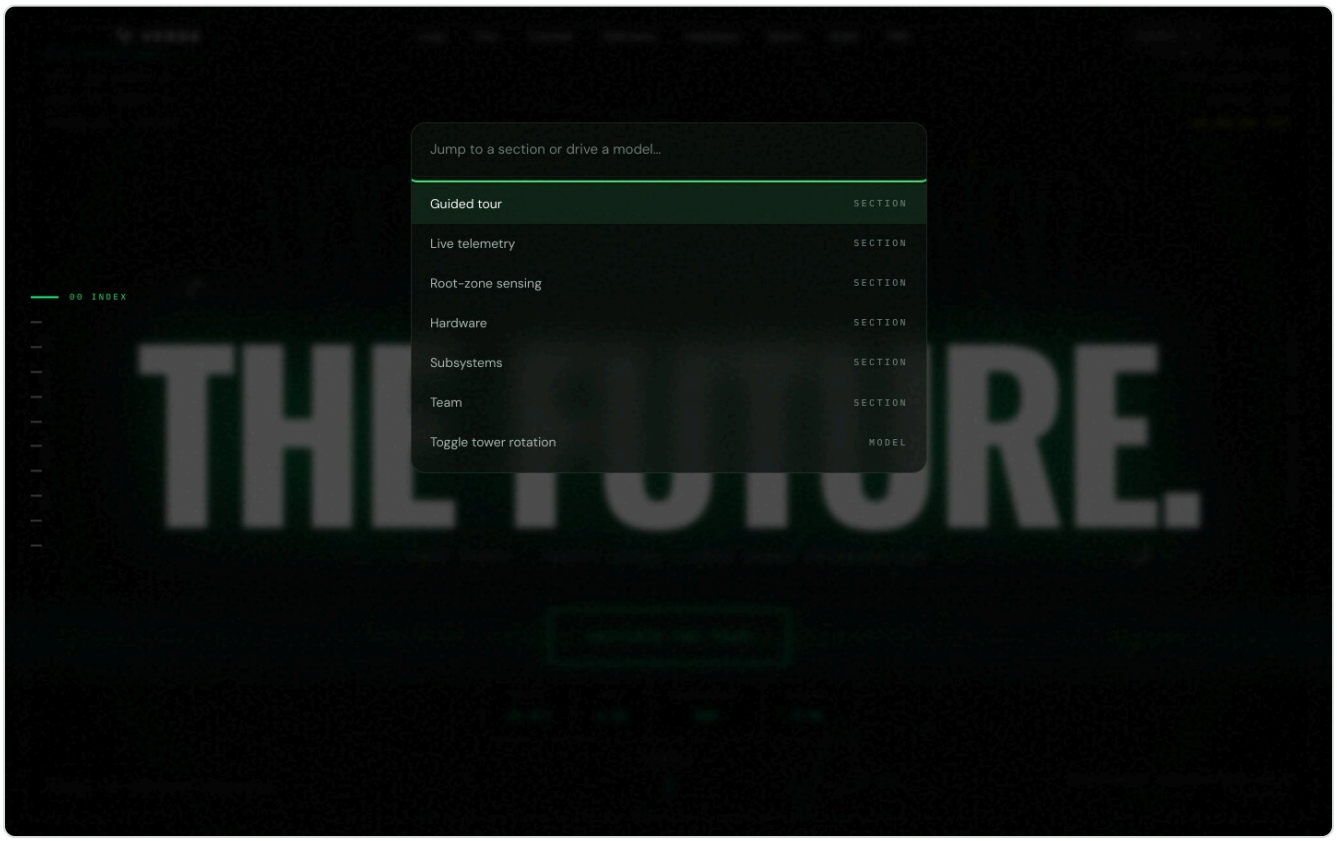


FIGURE 6.5 The command palette over the hero. The page behind is dimmed and blurred; the first command is pre-selected; the hint column shows each command's type.

The header becomes frosted glass after 24 px of scroll. Its search chip (or ⌘K anywhere) opens a palette with ten commands: six "Section" jumps, two "Model" actions that drive the 3D scenes over postMessage (*Toggle tower rotation* sends `release` + `autorotate`; *Explode the controller board* sends `explode` and scrolls to it), *Back to top*, and *Open the source on GitHub*. Typing filters by label and hint, the arrow keys move the selection and Enter runs it. It is a small feature that signals "power-user" polish to technical judges.

6.4 Velocity marquees

File **VelocityMarquee.tsx · 71 lines** Used **3× (after hero, after pillars, after hardware)**

Tech **useVelocity, useAnimationFrame, wrap**



FIGURE 6.6 Marquee 1: "AUTONOMOUS / SEALED LOOP / ON-DEVICE AI / ZERO SOIL" alternating outline and solid green, with lime slashes.

Each strip drifts slowly on its own, but **scrolling makes it react**: scroll velocity (smoothed by a spring) multiplies the speed by up to 4×, reverses its direction when you scroll up, and skews the text up to 8° in the direction of travel. The items are repeated four times and the position is wrapped between -25% and -50%, so the loop is seamless.

app/components/VelocityMarquee.tsx lines 28–45

```tsx
  const baseX = useMotionValue(0);
  const { scrollY } = useScroll();
  const scrollVelocity = useVelocity(scrollY);
  const smooth = useSpring(scrollVelocity, { damping: 50, stiffness: 400 });
  const factor = useTransform(smooth, [0, 1000], [0, 4], { clamp: false });
  const skew = useTransform(smooth, [-2000, 0, 2000], [8, 0, -8], { clamp: true });
  const skewSpring = useSpring(skew, { damping: 30, stiffness: 220 });

  const x = useTransform(baseX, (v) => `${wrap(-25, -50, v)}%`);
  const dir = useRef(1);

  useAnimationFrame((_t, delta) => {
    let move = dir.current * baseVelocity * (delta / 1000);
    if (factor.get() < 0) dir.current = -1;
    else if (factor.get() > 0) dir.current = 1;
    move += dir.current * move * factor.get();
    baseX.set(baseX.get() + move);
  });
```

6.5 Manifesto: “Soil is running out.”

Files **Manifesto.tsx 68 · TextReveal.tsx 43 · Magnetic.tsx** ID **#manifesto**

Tech **3D text flip, scroll-linked word reveal**

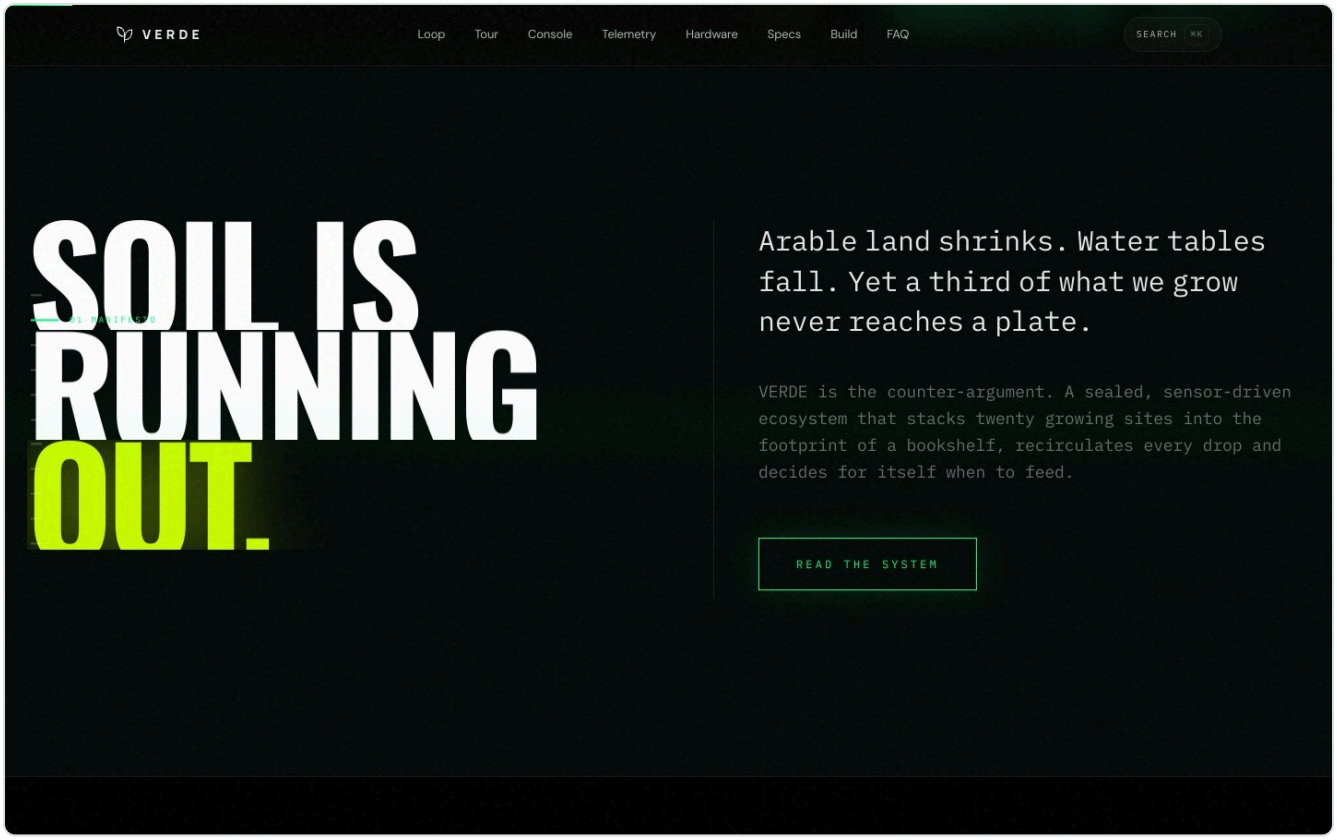


FIGURE 6.7 The manifesto **AS DESIGNED** : three lines flip up into view, with “OUT.” in lime. **SIMULATED FIX:** in the shipped build these lines never appear (see §11.2 for the actual screenshot and the cause).

On the left, three lines are meant to rotate up from −90° (`rotateX`) with a skew, staggered 0.14 s apart. On the right, **TextReveal** splits a sentence into words and maps scroll progress to each word’s opacity (0.12 → 1), lift (8 px → 0) and blur (5 px → 0), so the sentence “lights up” as you read it. A second paragraph and a magnetic “Read the system” button follow.

6.6 Stats strip

File **StatsStrip.tsx · 63 lines** Tech **animate() count-up, CSS variables for a cursor spotlight**

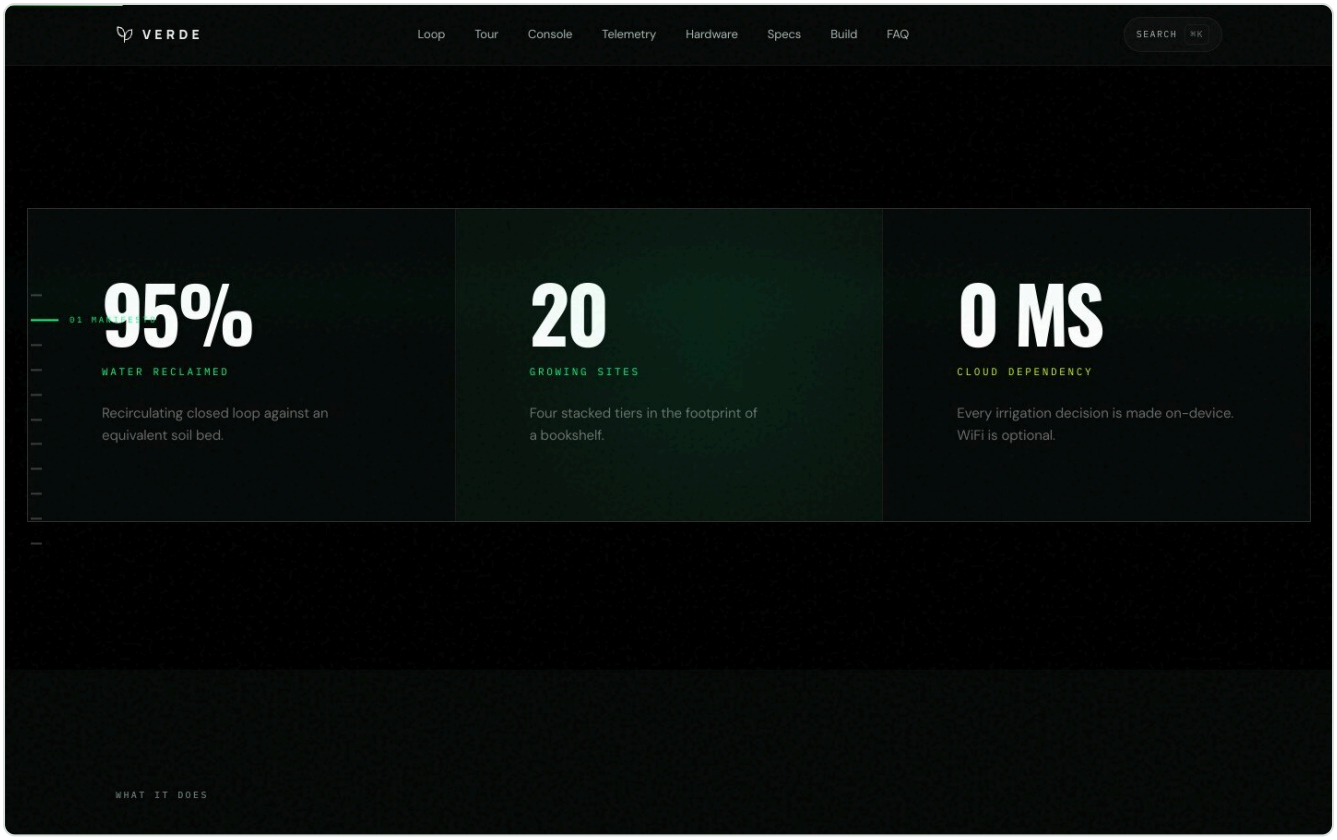


FIGURE 6.8 Three “brutalist” cells separated by 1 px lines: 95% water reclaimed, 20 growing sites, 0 ms cloud dependency. The middle cell shows the hover spotlight and the green sweep line.

Numbers count up from 0 when scrolled into view (1.8 s, signature easing), writing straight to `textContent` to avoid React re-renders. Moving the mouse sets `--mx/--my` CSS variables that position a radial glow (`.cell::after`), and a 1 px gradient line sweeps across the top on hover (`.cell-sweep`).

6.7 Pillars: “Growing is mostly attention.”

Files **Pillars.tsx 85 · SplitText.tsx 60** ID **#system** Tech **pointer-driven 3D tilt, staggered word reveal**

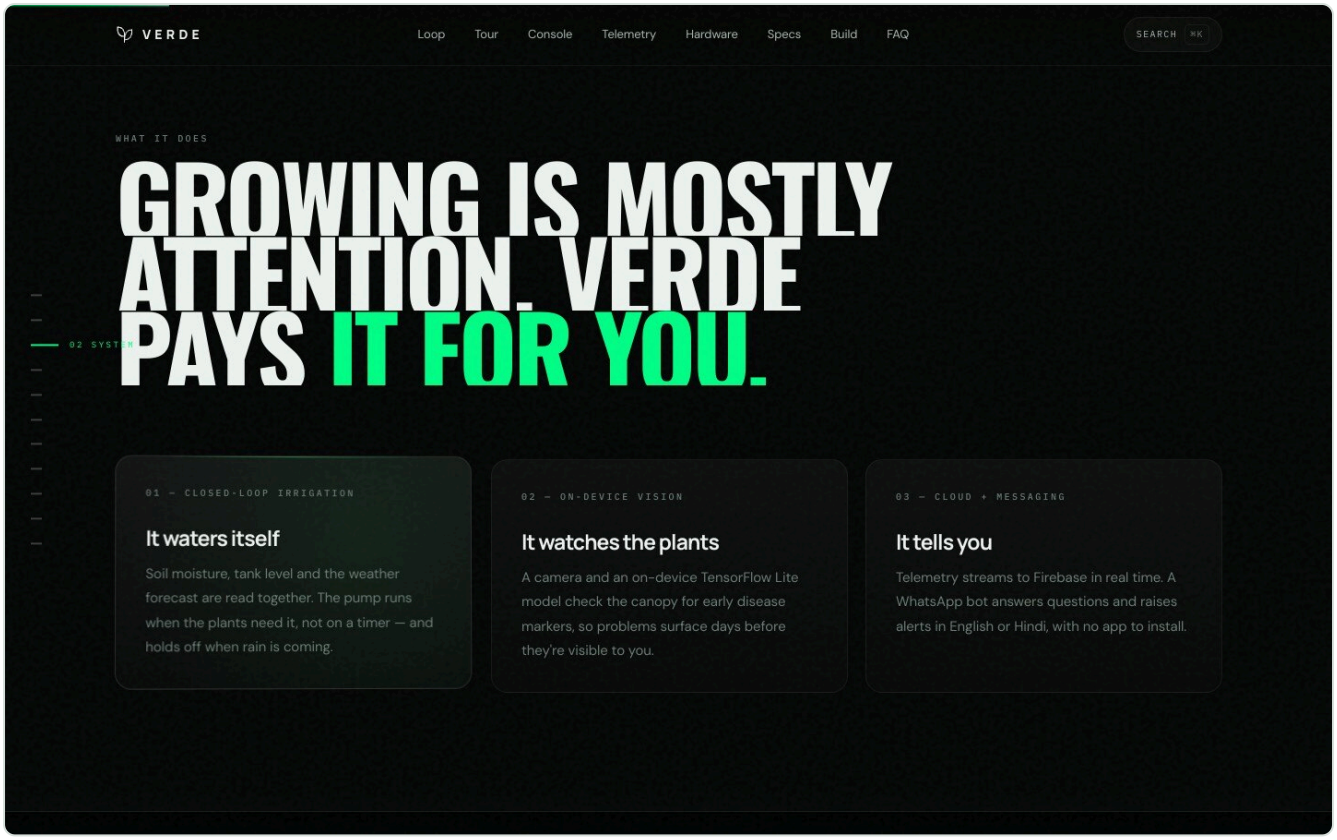


FIGURE 6.9 Three pillar cards (closed-loop irrigation, on-device vision, cloud + messaging). The first card is hovered: it tilts toward the cursor and a soft green glow follows the pointer.

The headline uses **SplitText**: each word rises and un-rotates in sequence, and the last three words (“Verde pays it for you.”) are painted green. Each card’s tilt is computed from the pointer position (up to ±2.5° around X and ±3° around Y, lifted 4 px) and written directly to `style.transform`, which is cheaper than React state on every mouse move.

6.8 The loop deck: “Six steps, forever.”

File **HorizontalDeck.tsx · 234 lines (largest component)** ID **#loop · 420vh pinned**

Tech **sticky + scroll-to-horizontal mapping, spring smoothing**

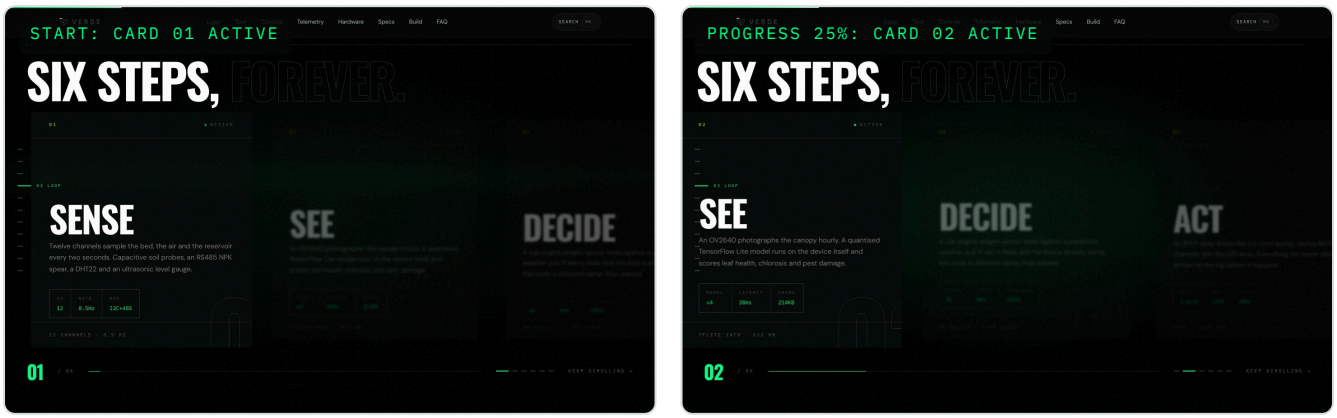


FIGURE 6.10 Vertical scrolling drives a horizontal track of six cards. The active card is sharp; the others shrink to 93%, dim to 34% opacity and blur 2.5 px. A footer rail shows 01–06, a progress line and step pips.

The section is 6 × 70vh = 420vh tall, with a sticky inner frame. Scroll progress (spring-smoothed) maps to the track's `translateX`, holding still for the first and last 6%, plus three layers of parallax: the header drifts left, each card's content drifts vertically at its own rate, and a giant outlined numeral behind each card moves fastest. The active index is `round(progress × 5)`.

> **VERIFIED BUG**
>
> The travel distance is a fixed constant (`-182vw`). It only fits tablet widths: on phones the deck stops at card 3 (card 4 barely peeks in; cards 5–6 never appear), and on desktops the deck ends with two-thirds of the screen empty. Measured evidence and a fix are in §11.5.

6.9 The guided tour: four chapters, one live tower

Files **TowerStory.tsx 146 · scenes/tower.html** ID **#tour · 400vh pinned**

Tech **scroll → chapter → postMessage camera flight**

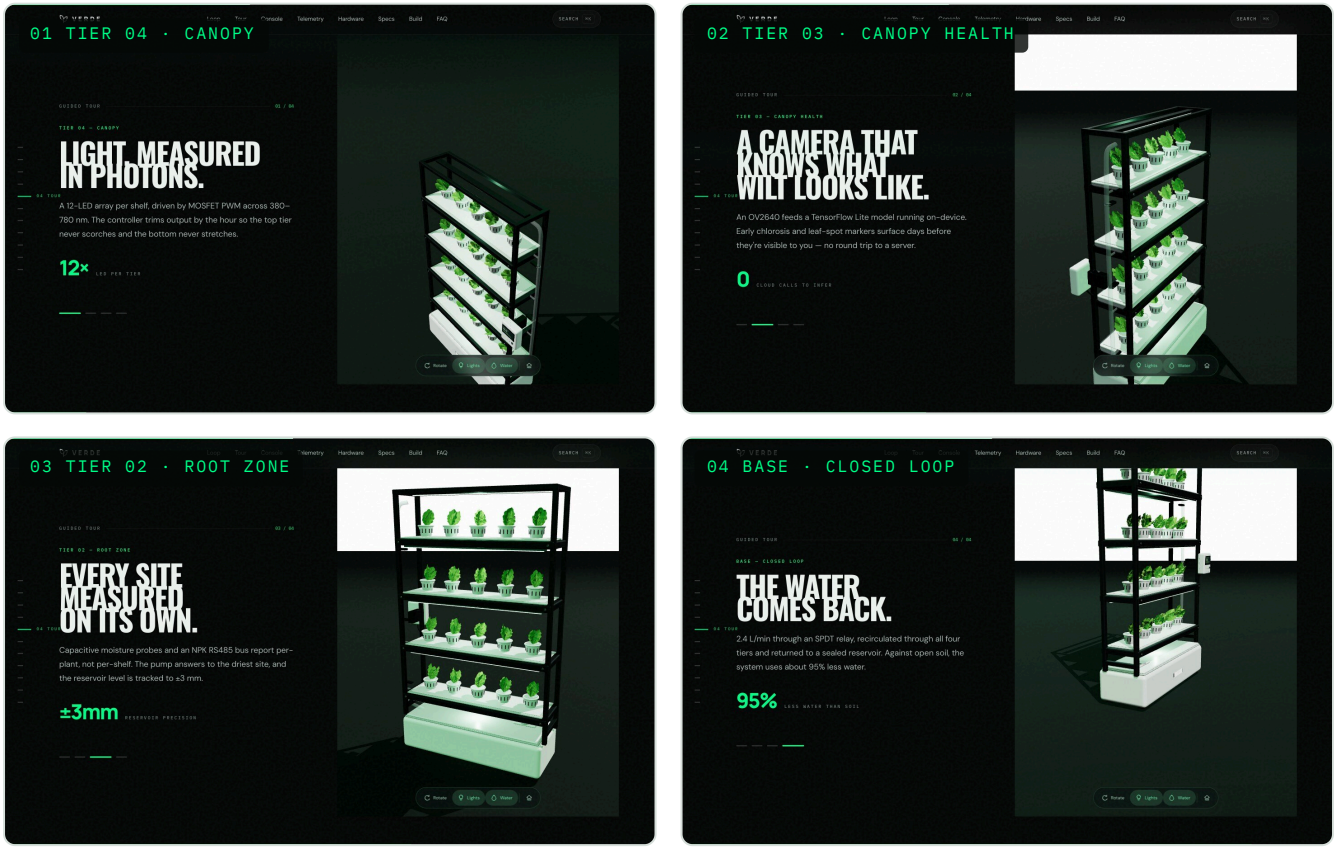


FIGURE 6.11 The pinned tour. Scrolling switches chapters (copy animates out and in with blur) and flies the real 3D camera to a new viewpoint. The white areas behind the tower in chapters 2–4 are the color-scheme bug analysed in §11.3.

The section is 400vh tall. Scroll progress is split into four equal bands; entering a band sets the active chapter, which sends one command to the tower: `{action:'view', az, pol, dist, ty}`, a camera position in spherical coordinates (azimuth, polar angle, distance, target height). The tower then eases its camera there (§5.6). A thin green line along the bottom tracks progress, and four pips act as chapter buttons. On screens narrower than 1024 px the tower slot is hidden (§11.7).

app/components/TowerStory.tsx lines 57–66 chapter → camera

```
57  useEffect(() => {
58    return scrollYProgress.on("change", (v) => {
59      const i = Math.min(chapters.length - 1, Math.max(0, Math.floor(v * chapters.length + 0.001)));
60      setActive(i);
61    });
62  }, [scrollYProgress]);
63
64  useEffect(() => {
65    send("tower", { action: "view", ...chapters[active].view });
66  }, [active, send]);
```

6.10 Live console: “It talks to itself.”

File **LiveTerminal.tsx · 135 lines** ID **#console** Tech **interval-driven log, auto-scroll**

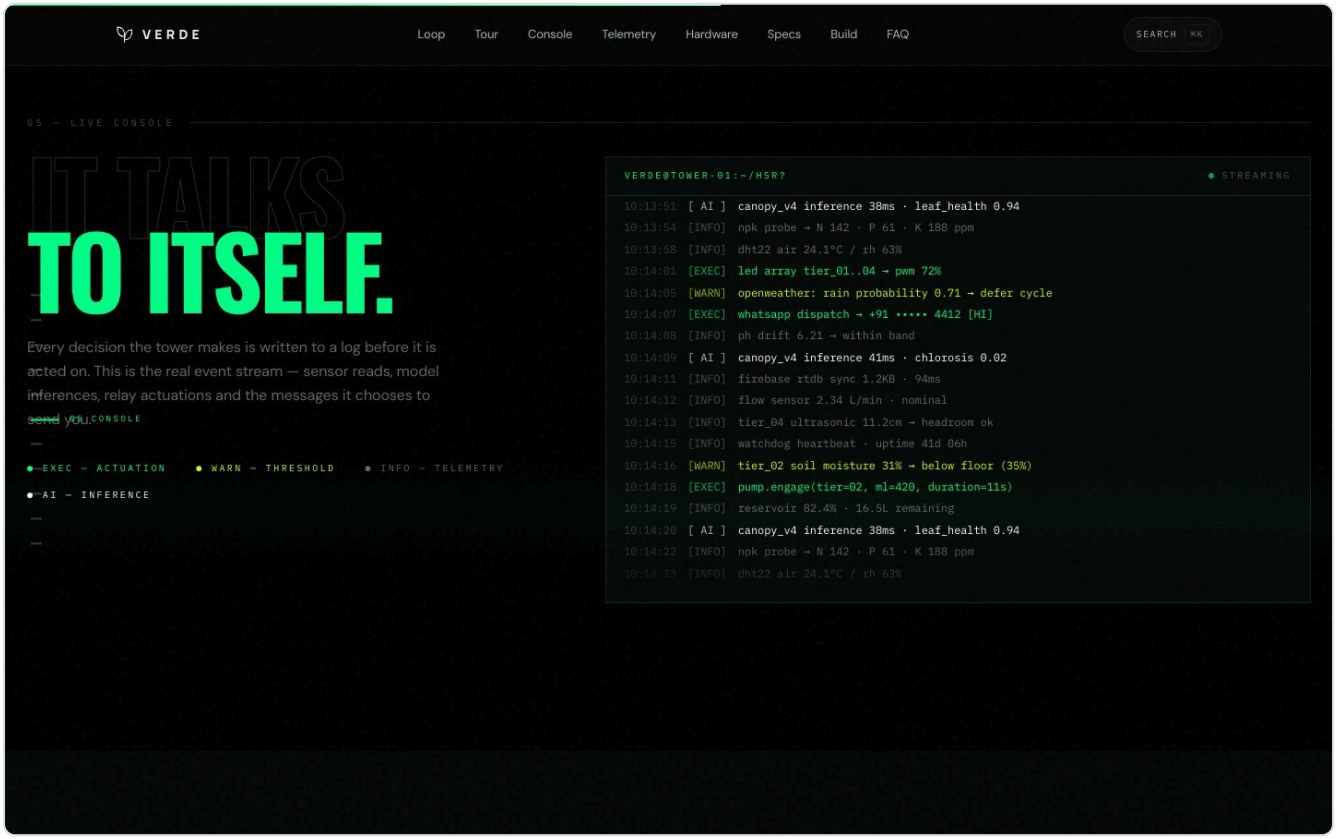


FIGURE 6.12 A terminal window (“verde@tower-01:~/core”, streaming) printing colour-coded events: EXEC actuation (green), WARN threshold (lime), INFO telemetry (grey) and AI inference (white).

Every 1.4 s the next message from a pool of 15 is appended with a real IST timestamp (keeping the last 29 lines) and the box scrolls smoothly to the bottom. The messages tell a coherent story: moisture falls below the floor, the pump engages for 11 s, the canopy model scores leaf health, a rain forecast defers a cycle, a Hindi WhatsApp alert is dispatched, Firebase syncs.

> **FACT CHECK**
>
> The copy says “This is the real event stream”, but it is a scripted loop of 15 messages generated in the browser. See §10.5 for wording that stays impressive and accurate.

6.11 Telemetry: “The system is always reporting.”

File **Telemetry.tsx · 207 lines** ID **#telemetry**

Tech **mean-reverting random walk, SVG sparklines, stroke-dashoffset gauge**

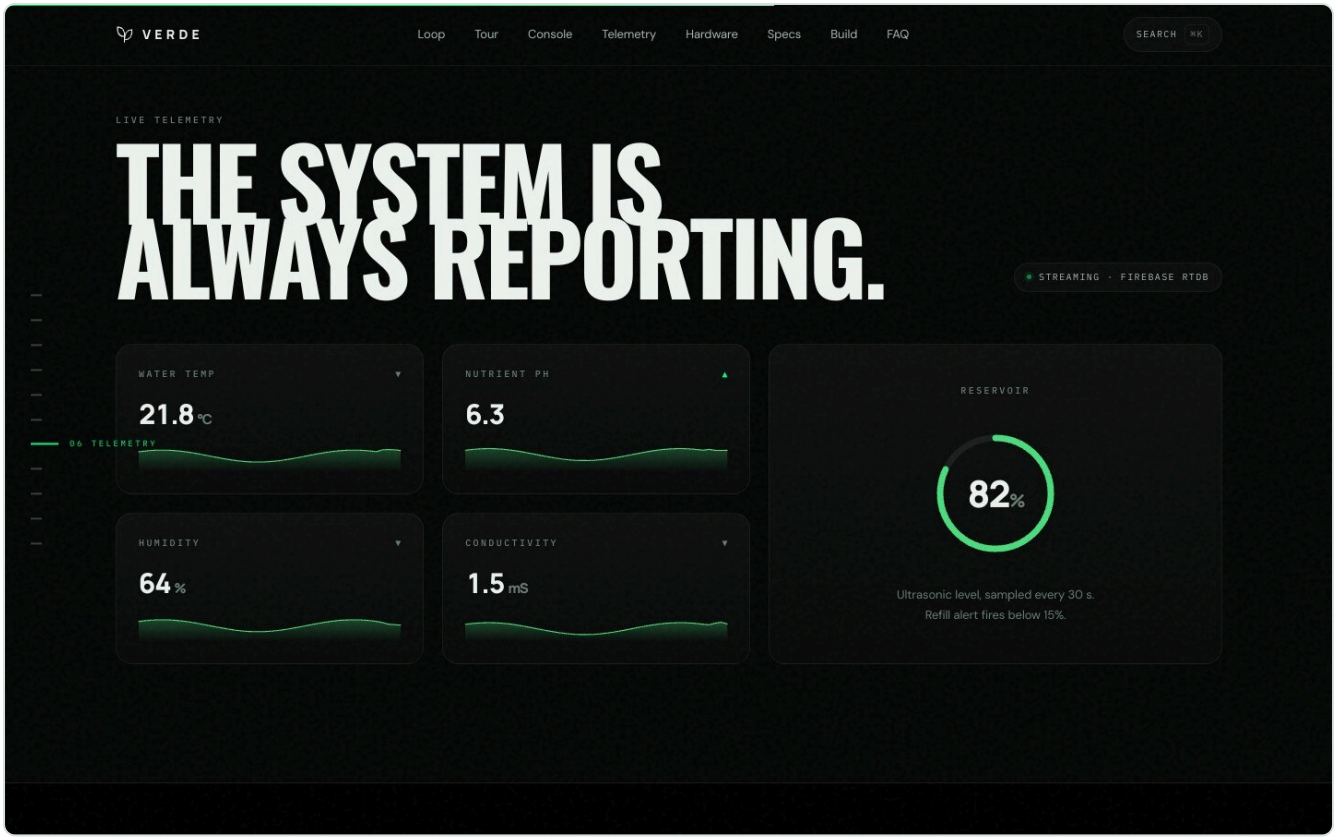


FIGURE 6.13 Four live cards (water temp, nutrient pH, humidity, conductivity) with trend arrows and sparklines, beside an animated reservoir gauge at 82%.

Each series is a **random walk with a pull back toward its seed value**, clamped to a realistic band (pH 5.9–6.5, EC 1.1–1.9 mS…). That is why the lines wander believably instead of jumping. Updates only run while the panel is on screen (`useInView`), a thoughtful touch. The gauge animates `strokeDashoffset` on an SVG circle.

app/components/Telemetry.tsx lines 18–38

```tsx
function useWalk(s: Series, live: boolean, points = 44) {
  const [data, setData] = useState<number[]>(() =>
    Array.from({ length: points }, (_, i) => s.seed + Math.sin(i / 5) * (s.max - s.min) * 0.18)
  );

  useEffect(() => {
    if (!live) return;
    const id = setInterval(() => {
      setData((prev) => {
        const last = prev[prev.length - 1];
        const drift = (Math.random() - 0.5) * (s.max - s.min) * 0.14;
        const pull = (s.seed - last) * 0.08;
        const next = Math.min(s.max, Math.max(s.min, last + drift + pull));
        return [...prev.slice(1), next];
      });
    }, 1400);
    return () => clearInterval(id);
  }, [live, s]);

  return data;
}
```

6.12 Against the ground: “Dirt loses.”

File **Comparison.tsx · 91 lines** ID **#versus** Tech **scaleX bar animation, TextReveal**



FIGURE 6.14 Five paired bars, soil (grey) versus Verde (green, glowing), each animating from zero when scrolled into view.

METRIC	SOIL	VERDE	VERDE AS % OF SOIL
Water per cycle	18.0 L	0.9 L	5%
Floor area / 20 plants	4.2 m²	0.46 m²	11%
Time to harvest	61 days	38 days	62%
Human interventions / week	14	0.4	3%
Nutrient runoff lost	100%	0%	0% (bar kept at 0.6% so it stays visible)

6.13 Down at plant level: the seedling pod

Files **SensingSection.tsx 59 · scenes/pod.html 1,602** ID **#sensing** Tech **SceneSlot "pod", drag to orbit**

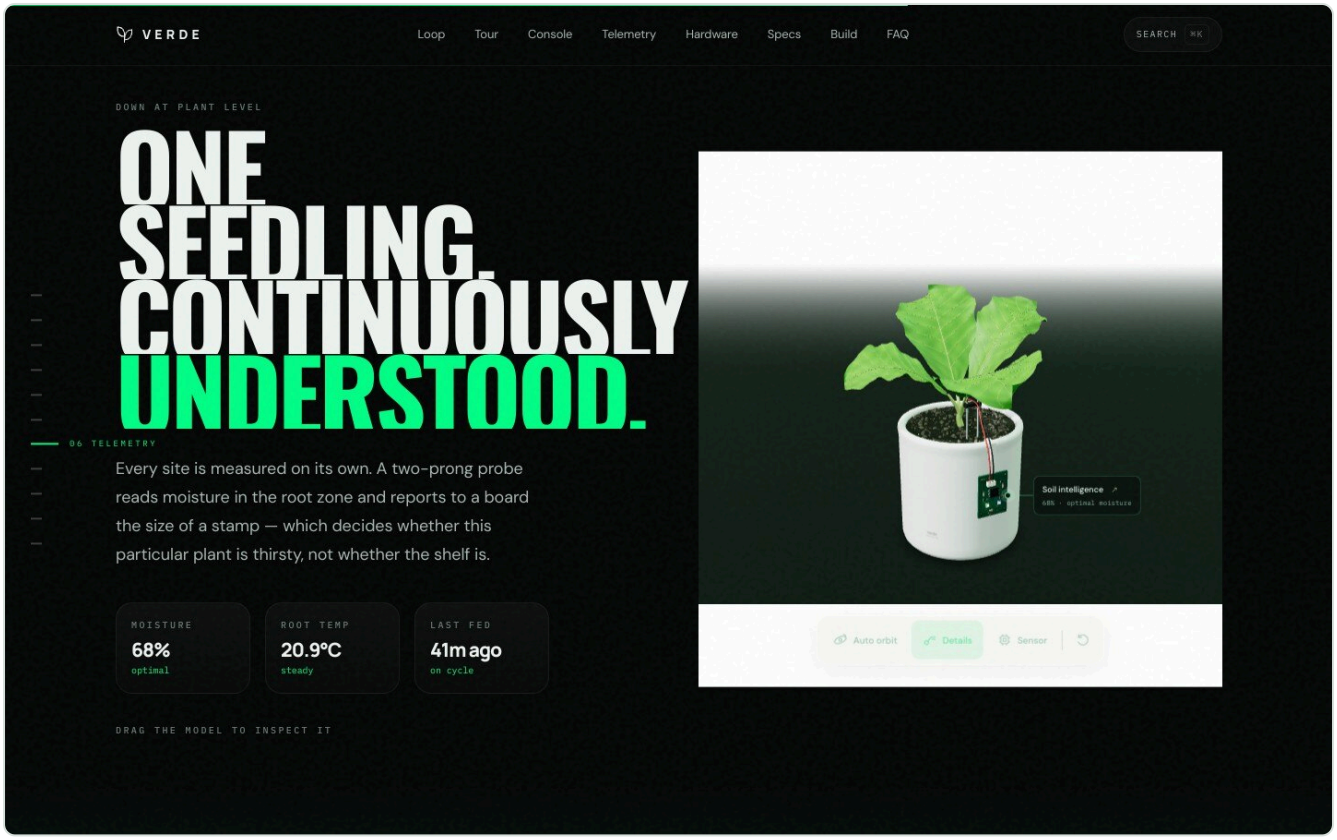
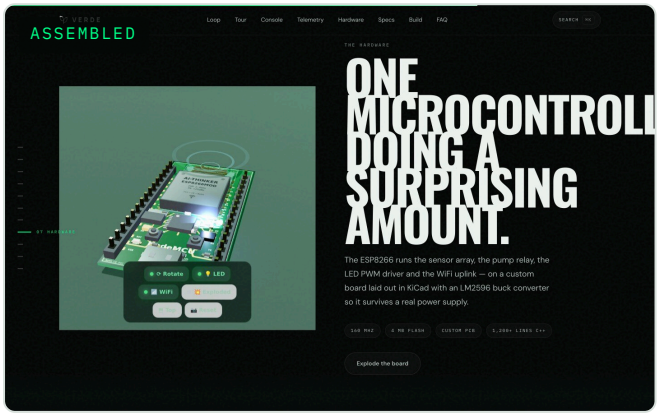


FIGURE 6.15 Headline, reading cards (moisture 68%, root temp 20.9 °C, last fed 41 min ago) and the live pod scene with its "Soil intelligence" annotation. The white bands inside the 3D frame are bugs §11.3 and §11.4.

The most detailed model on the site: a lettuce seedling in a white pod with a two-prong moisture probe wired to a tiny circuit board. The visitor can drag to orbit it, toggle annotations and fly the camera to the sensor. Chapter 7 explains how it is built.

6.14 The hardware: the exploding board

Files **BoardSection.tsx 56 · scenes/board.html 506** ID **#hardware** Tech **scroll-driven rotation, explode command**



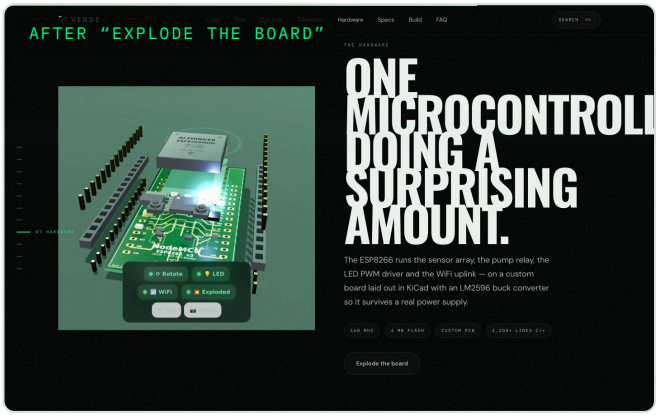


FIGURE 6.16 The NodeMCU ESP8266 model spins as you scroll past (scroll progress -0.9 → 1.5 rad is sent continuously). The button sends {action:"explode"} and the parts fly apart: headers, shield can, module, USB, buttons, chips.

app/components/BoardSection.tsx lines 7–16 scroll drives the real model

```tsx
/** The board spins as you scroll past it — scroll position drives the real 3D model. */
export default function BoardSection() {
  const ref = useRef<HTMLDivElement>(null);
  const { send } = useScenes();
  const { scrollYProgress } = useScroll({ target: ref, offset: ["start end", "end start"] });
  const spin = useTransform(scrollYProgress, [0, 1], [-0.9, 1.5]);

  useEffect(() => {
    return spin.on("change", (v) => send("board", { action: "spin", value: v }));
  }, [spin, send]);
```

6.15 Subsystems: “Everything inside, in plain terms.”

File **SpecExplorer.tsx · 100 lines** ID **#specs** Tech **shared-layout tab indicator (layoutId), AnimatePresence**

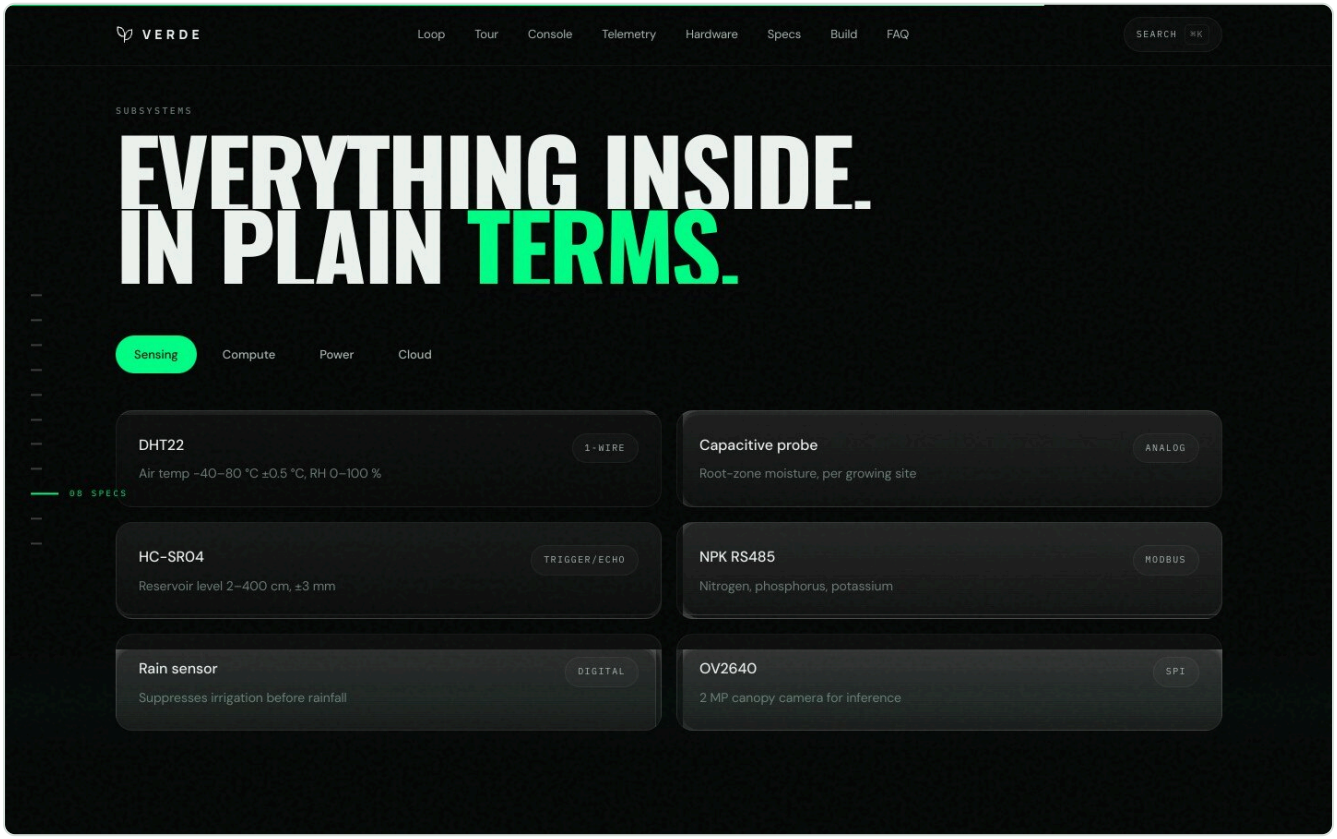


FIGURE 6.17 Four tabs (Sensing, Compute, Power, Cloud). The green pill slides between tabs (framer-motion layoutId) and the cards cross-fade.

The full table of parts is reproduced in §2.3. The sliding pill is a single `motion.span` with `layoutId="specTab"`: framer-motion animates it from the old tab’s box to the new one automatically, with a spring (stiffness 340, damping 32).

6.16 Build log: “Fourteen weeks, in order.”

File **Timeline.tsx · 62 lines** ID **#build** Tech **scroll-drawn rail**

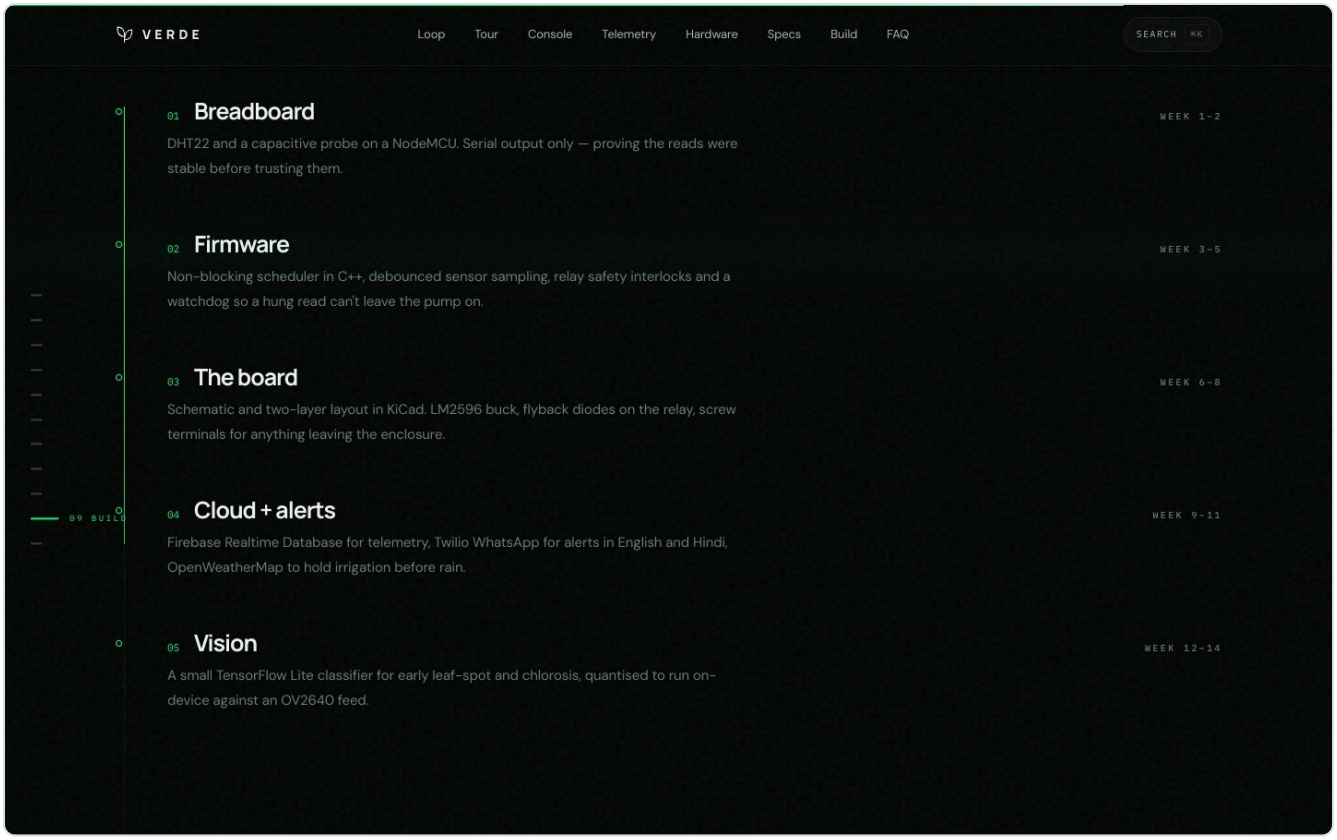
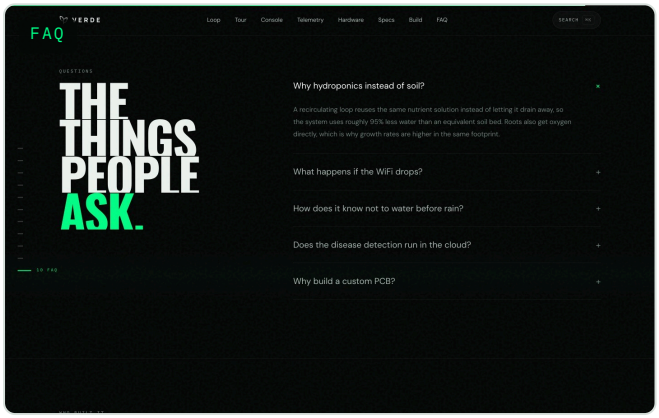


FIGURE 6.18 Six phases from breadboard to tower. The green rail on the left “draws” downward as you scroll (scroll progress → spring → height).

6.17 FAQ, Team and Closing



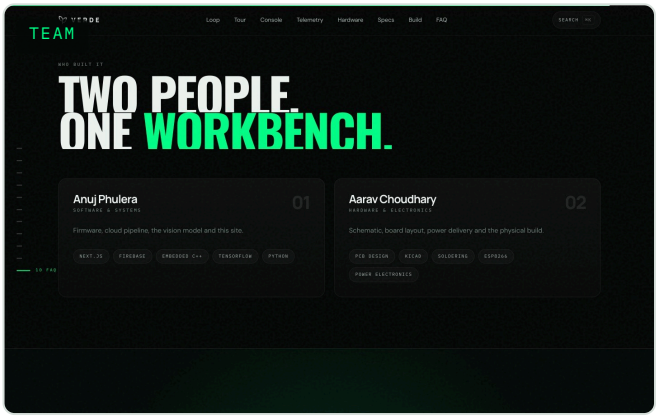


FIGURE 6.19 Left: a five-question accordion (first open by default; height animates to “auto”; the “+” rotates 45° into a “×”; aria-expanded is set). Right: two team cards with role, contribution and skill chips.

The FAQ answers the questions judges actually ask: why hydroponics, what happens offline, how rain is predicted, whether disease detection uses the cloud, and why a custom PCB. The closing section is a giant outlined **CULTIVATE.** above a glowing “Initialize Access” button (magnetic, strength 0.6) that links back to the tour. The one `lucide-react` icon on the site is its arrow.

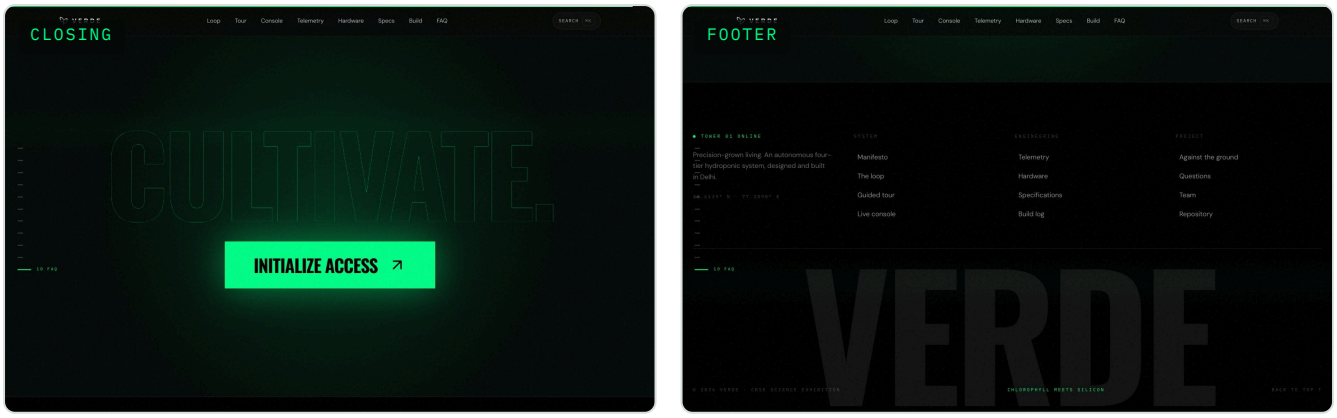


FIGURE 6.20 Left: the closing call to action. Right: the footer, with "● Tower 01 online", three link columns (including the GitHub repository), a colossal faint VERDE wordmark that rises into place, and "© 2026 Verde · CBSE Science Exhibition · Chlorophyll meets silicon".

THE THREE 3D SCENES

A grow tower, a seedling with its sensor and a microcontroller board. None of them loads a 3D model file: every shelf, leaf, solder pad and gold trace is generated by JavaScript at runtime.

7.1 The shared recipe

Each scene began life as a standalone, light-themed "product viewer" page (`model1.html`, `model2.html`, `model3.html` at the repository root). To embed them, each was copied into `public/scenes/` and changed in the same six ways:

#	CHANGE	HOW	WHY
1	Self-hosted three.js	Import map points `three` → `/vendor/three.module.js` (board: `/vendor/legacy/three.min.js`) instead of jsDelivr/cdnjs	No CDN at runtime, works behind a firewall, cached for a year
2	Transparent canvas	`alpha: true`, `setClearColor(0x000000, 0)`, `scene.background = null`	The page's black stage shows through, so the model "floats"
3	Re-lit for a dark page	Light studio colours → dark floor `#080d0b`, dark fog, a green rim light `#4ade80`	Matches the site palette
4	Hide the chrome	A `<style id="verde-embed">` block hides headers, panels and loaders, and re-skins toolbars	Only the model and its essential controls remain
5	Readiness signal	After `load` + two animation frames: `parent.postMessage({verde:"scene-ready", src})`	Feeds the boot curtain's progress
6	Remote control	A `message` listener for `{verde:"cmd", …}` (tower and board only)	Lets scroll position and buttons on the page drive the model

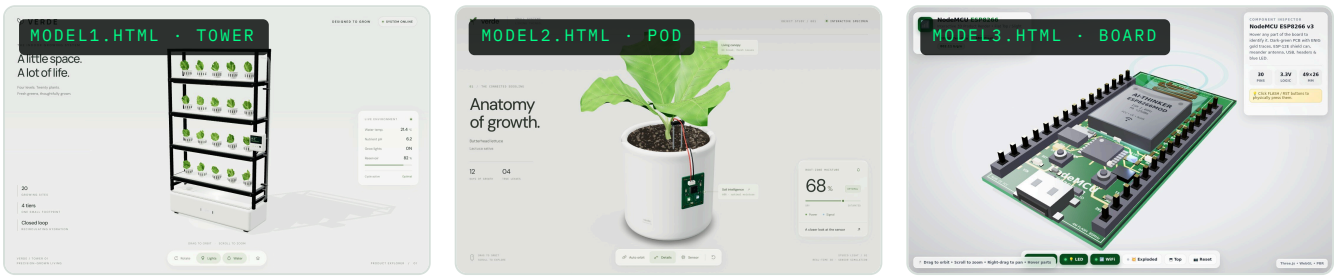


FIGURE 7.1 The three original prototypes in their own light-themed product pages (captured with the CDN requests rerouted to the vendored copies). Note the original brands: "VERD", "verde / Small systems. Living things." and "NodeMCU ESP8266, Interactive 3D Product Render".

7.2 The tower · `tower.html` (443 lines, three r162)

Rendering setup: antialiased WebGL with soft PCF shadows (2048² map), sRGB output, ACES filmic tone mapping (exposure 1.16), pixel ratio capped at 2. A 34° camera (44° on narrow screens) orbits the target `(0, 2.13, 0)` with damped OrbitControls, limited to 6–16 units and above the floor, with no panning.

Lighting: a hemisphere light, a shadow-casting key light, a cool fill, a **green rim light** from behind, plus a point light under every shelf and above the canopy so the grow-light strips actually illuminate the plants.

Built entirely in code:

- Reservoir, frame, four shelves (y = 0.99, 1.88, 2.77, 3.66) from `RoundedBoxGeometry` helpers (`box` , `cyl` , `tube`).
- 20 net pots, each a cylinder, a torus rim and **12 cut slots**.
- 20 lettuces × 10 leaves = **200 leaves** with custom curled geometry and vein tubes. A seeded linear-congruential RNG (`seed = 12`) makes every visitor see the identical plant.
- A glass feed pipe (`MeshPhysicalMaterial` , transmission 0.35) along a Catmull-Rom curve, with **14 droplets** travelling along it.
- A control box whose screen is a live `CanvasTexture` ("VERDE / GROW SYSTEM · 21.4° · pH 6.2 FLOW OK") that redraws when lights or water are toggled.

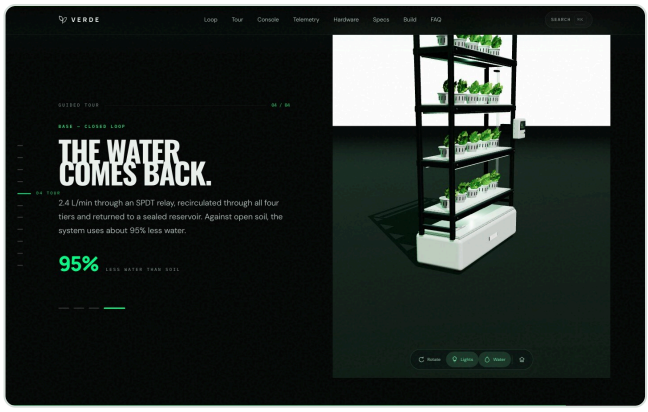


FIGURE 7.2 The tower in the tour's fourth chapter, embedded and re-lit.

Interaction: Rotate / Lights / Water / Reset buttons. Lights fade with `MathUtils.damp` rather than snapping. Clicking the control box (raycast, ignored if you dragged more than 6 px) toggles the info panel. Leaves sway gently, and under reduced motion the water and sway stop. Rendering skips frames while the tab is hidden.

> **LEFTOVER**
>
> The reservoir's printed label still reads **"V E R D · T O W E R O 1"**, the prototype's old brand (line 181). See §11.9.

7.3 The seedling pod · `pod.html` (1,602 lines, three r162)

By far the most sophisticated scene: it is closer to a product render than a web widget.

- **Image-based lighting:** a `RoomEnvironment` is baked through `PMREMGenerator` into an environment map, so plastic, steel and solder reflect a believable studio.
- **Physical materials:** clear-coated plastic, sheen on the leaves (`sheen 0.28`), clear-coated wires and LED lenses.
- **The pod:** a `LatheGeometry` (160 segments) spun from a 2D profile, with a moulded seam, an inner lip and a printed logo.
- **Soil:** canvas-generated textures and a displaced height-field surface, scattered with **440 soil clumps and 22 mineral grains** drawn as `InstancedMesh` (one draw call each), plus 32 root fibres merged into a single mesh with `mergeGeometries` .
- **Leaves:** textures painted on canvas, vein by vein, plus tapered stems and petioles from a custom `taperedTube` generator.
- **Electronics:** a PCB with a painted texture, SMD resistors and capacitors, header pins, two LEDs with additive glow
- **A baked contact shadow:** rendered *once* at start-up. A custom GLSL shader writes height-based darkness (`exp(-h × 1.65)`) from an orthographic camera below the model into a render target. Two passes of a separable 9-tap Gaussian blur at two radii soften it, and the result is laid on the floor as a texture. Real-time shadow maps are also frozen (`shadowMap.autoUpdate = false`). The effect looks expensive but costs nothing per frame.
- **Framing maths:** camera distances are computed from the field of view and aspect ratio so the seedling always fits, whatever the slot size.
- **Two views + annotations:** "plant" and "sensor" camera flights; HTML callouts pinned to 3D points by projecting them to screen space every frame.
- **Keyboard:** Space (auto-orbit), D (details), S (switch view), R/Esc (reset).
- **Graceful failure:** if WebGL fails, a message ("The studio couldn't open.") and a Try again button replace the scene.

sprites, a two-prong steel probe and red/black wires along curves.

- **Determinism:** a mulberry32 RNG (seed 18453) makes the plant identical on every load.

public/scenes/pod.html lines 1279–1310 baked contact shadow (excerpt)

```
function bakeContactShadow() {
  const size = 512, extent = 4.8;
  const target = new THREE.WebGLRenderTarget(size, size, { depthBuffer: true });
  const blurTarget = new THREE.WebGLRenderTarget(size, size, { depthBuffer: false });
  const shadowCamera = new THREE.OrthographicCamera(-extent / 2, extent / 2, extent / 2, -extent / 2, .01, 4.6);
  shadowCamera.position.set(0, -.035, 0);
  shadowCamera.up.set(0, 0, 1);
  shadowCamera.lookAt(0, 2, 0);

  const heightMaterial = new THREE.ShaderMaterial({
    side: THREE.DoubleSide,
    vertexShader: `
      varying float vHeight;
      void main() {
        vec4 p = vec4(position, 1.0);
        #ifdef USE_INSTANCING
          p = instanceMatrix * p;
        #endif
        vec4 world = modelMatrix * p;
        vHeight = world.y;
        gl_Position = projectionMatrix * viewMatrix * world;
      }`,
    fragmentShader: `
      varying float vHeight;
      void main() {
        float opacity = exp(-max(vHeight, 0.0) * 1.65);
        gl_FragColor = vec4(0.0, 0.0, 0.0, opacity);
      }`
  });
  const blurMaterial = new THREE.ShaderMaterial({
    uniforms: { map: { value: null }, direction: { value: new THREE.Vector2() } },
    depthTest: false, depthWrite: false,
```

7.4 The ESP8266 board · `board.html` (506 lines, three r128)

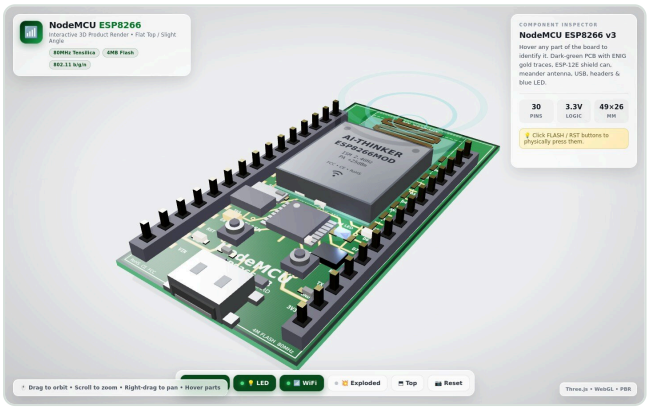


FIGURE 7.3 The original board viewer with its component inspector panel.

Written against the legacy global `THREE` API (r128 from cdnjs), it is the only scene that needs the second, legacy copy of three.js. Upgrading it to r162 would remove 603 KB of vendor code (§12.2).

- **A hand-painted PCB:** a 1024×2048 canvas becomes the board's top texture: gradient FR-4 green, grain noise, gold traces drawn with three stroke passes (shadow, body, highlight), 28 vias, 30 header pads, the real NodeMCU pin labels (A0…VIN, D0…3V3), an antenna keep-out box, and silkscreen text.
- **Components as groups:** ESP-12E module, RF shield can (its own canvas texture: "AI-THINKER ESP8266MOD"), meander antenna, CH340G USB bridge, AMS1117 regulator, 12 MHz crystal, passives, USB port, FLASH/RST buttons, blue LED.
- **Inspector:** hovering raycasts to the nearest group that carries a `title` / `desc`, draws a `Box3Helper` around it and shows a tooltip ("CH340G USB-Serial Chip: USB to UART bridge…").
- **Exploded view:** 16 parts each have an offset vector; a factor eases between 0 and 1.
- **Life:** the LED blinks in a double-pulse pattern, Wi-Fi rings radiate from the antenna, and clicking FLASH or RST physically presses the button.
- **Keyboard:** R, L, W, E toggle rotate, LED, Wi-Fi and explode.

7.5 Originals versus embedded copies

	MODEL1 → TOWER	MODEL2 → POD	MODEL3 → BOARD
Lines (original → embedded)	380 → 443	1,591 → 1,602	474 → 506
three.js source	jsDelivr 0.164.1 → vendored r162	jsDelivr 0.160.0 → vendored r162	cdnjs r128 → vendored legacy r128
Background	#e9ebe8 → transparent	#d2d3d0 → transparent (`null`)	grey gradient → transparent
Lighting	warm studio → dark + green rim	warm studio → dark + green rim	neutral → dark + green rim
Remote API	view, release, autorotate, lights, flow	none	spin, explode, led
Embed CSS hides	header, intro, specs, panel, hint, bottom	topbar, intro, telemetry, notes, loader…	header, credit, hint, chips, loader, vignette

> NOTE
>
> The embedding left two subtle defects that only appear on the dark page: the pod's fog became **white** (`new THREE.Fog(null, …)` defaults to white) and its toolbar keeps the light theme (the embed CSS targets class names that do not exist in pod.html). Both are covered in §11.4.

7.6 The plan that was not used: AI-generated GLB models

`docs/3d-model-prompts.md` is a detailed brief for generating the three models with text-to-3D and image-to-3D tools (Meshy, Tripo, Rodin, Luma Genie, Hunyuan3D, Sloyd): prompts, negative prompts, polycounts (150k–300k for the tower), and export rules ("GLB, PBR materials on, Y-up, 2K textures, under 10 MB"). It even ranks priorities: tower essential, pod high, board nice-to-have.

None of it was used. There is no `.glb`, `.gltf` or loader anywhere in the repository; the scenes are procedural. That turned out to be the better outcome:

	PROCEDURAL SCENES (USED)	AI-GENERATED GLB (NOT USED)
Download size	26–76 KB of HTML per scene	Up to 10 MB per model (the brief's limit)
Animatable parts	Every part is its own object: board explodes, droplets flow, leaves sway, LEDs blink	One fused mesh, hard to animate
Accuracy	Real pin labels, real component names, exact shelf count	Generators "guess" (the brief warns about uneven shelves and fake pins)
Editability	Change a number, reload	Regenerate and re-export

THE DESIGN SYSTEM

Near-black, one electric green, a second voice in lime, and type so large it becomes architecture. All of it lives in one 602-line stylesheet and a handful of reusable classes.

8.1 Two layers in one stylesheet

`app/globals.css` contains two clearly labelled registers that coexist:

LAYER	HEADER COMMENT	CONTAINS
Dark edition (lines 1–330)	"Near-black stage, emerald as a precision accent (never a wash), oversized display type, hairline structure."	Tokens, base typography, `.wrap .eyebrow .card .chip .btn .panel`, scene frame, scrollbars, reduced motion, aurora/grid backdrops
Maximalist layer (lines 331–602)	"brutalist / cinematic register. Oswald display at absurd sizes, outline type, HUD chrome, hard grids."	`.font-display-xl`, the type scale, outline text, HUD, noise and scanline, brutalist grid, `.link-brut`, glitch, section index

8.2 Colour

--bg
#050706
page background

--void
#040a07
manifesto & closing

.cell
#050907
brutalist cells, terminal

--ink
#edf3ef
primary text

--ink-soft
#aab8b1
secondary text

--muted
#6c7d75
eyebrows, captions

--grow / --green
#00ff87
the signature green

--green-deep
#00c96a
gradients

--acid
#ccff00
second voice: warnings, "OUT."

glow green
#4ade80
card glows, sparklines, rim light

mint
#a7f3d0
end of the scroll-progress gradient

--line
rgba(255,255,255,.09)
hairlines (white at 5.5–10%)

The palette is disciplined: one hue family (green) at different intensities, lime used sparingly as a "second voice" (warnings, the word "OUT.", the IST clock), and structure drawn with translucent white hairlines rather than grey boxes. The same `#00ff87` appears in the CSS, the Tailwind arbitrary values, the canvas particles, the cursor ring and the boot log.

8.3 Typography

Oswald
DISPLAY · 700 · UPPERCASE
.FONT-DISPLAY-XL

WE GROW THE FUTURE.

Manrope
HEADINGS · 700–800
H1–H4 · .DISPLAY

One seedling, continuously understood.

DM Sans
BODY · 400–600
16PX / 1.65

Every site is measured on its own. A two-prong probe reads moisture in the root zone and reports to a board the size of a stamp.

IBM Plex Mono
HUD · LABELS · 400/500
.MONO .EYEBROW .HUD-READ

SYS.OPERATIONAL · FLOW: 2.44 L/MIN · 10:08:03 IST

CLASS	SIZE	USED FOR
`.t-mega`	`clamp(64px, 17vw, 17rem)`	Hero headline, “CULTIVATE.”
`.t-giant`	`clamp(48px, 11vw, 11rem)`	Manifesto
`.t-big`	`clamp(38px, 7.5vw, 7.5rem)`	Most section headlines (11 uses)
`.font-display-xl`	Oswald 700, uppercase, tracking -0.035em, **line-height 0.75**	All mega type (24 uses); the tight leading causes overlaps on wrapped lines (§11.8)

8.4 Motion language

One easing curve is used almost everywhere: `cubic-bezier(0.16, 1, 0.3, 1)`, an “expo-out” that starts fast and settles slowly, which reads as confident and physical. It appears **35 times** in the components, 3 times in the CSS and once as the `--ease` token. Continuous motion uses springs, each tuned to its job:

WHERE	SPRING (STIFFNESS / DAMPING)	FEEL
Cursor ring	450 / 34 (mass 0.35)	Snappy follow, no wobble
Spec tab pill	340 / 32	Quick slide
Magnetic buttons	220 / 18 (mass 0.4)	Slightly elastic pull
Scroll progress bar	140 / 30	Smooth, no jitter
Loop deck · tour bar · timeline rail	110 / 30 · 120 / 28 · 90 / 26	Heavy, cinematic
Marquee velocity · skew	400 / 50 · 220 / 30	Responsive yet damped

8.5 Signature components

CLASS	WHAT IT LOOKS LIKE	TRICK
`.card`	Rounded glass card with a 1 px emerald line that appears on its top edge on hover	`::before` gradient line; hover lifts 4 px
`.cell` + `.cell-sweep`	Hard-edged brutalist cell with a cursor-following glow and a green sweep line	CSS variables `--mx/--my` set from JS; `translateX(-100% → 100%)`
`.grid-brut`	1 px grid lines between cells	A `gap: 1px` grid over a 10%-white background, so the gaps read as lines
`.link-brut`	Outlined green button that fills from the bottom on hover	`::before` slides from `translateY(101%)` to 0
`.text-outline`	Hollow letters that fill with glowing green on hover	`-webkit-text-stroke` + transparent fill
`.hud-frame` / `.hud-read`	Inset hairline frame and spaced-out mono readouts	Absolute inset 20 px; 0.3em tracking
`.glitch`	RGB-split jitter on the boot logotype	Two pseudo-elements + animated `clip-path`
`.bg-noise` / `.scanline-fx`	Film grain over everything; a faint green band sweeping down every 9 s	Inline SVG `feTurbulence` data URI; fixed, pointer-events none
`.sec-index`	“04 — The loop” labels with a trailing rule	`::after` flex line

8.6 Respecting reduced motion

Users who ask their OS for less motion get a calmer site: Lenis is not started, the particle field draws a single still frame, CSS animations and transitions drop to 0.01 ms, the scanline, glitch layers and backdrop animations are removed, and the tower and pod scenes pause their water and sway. One gap remains: framer-motion animations (reveals, marquees, parallax) still run, because no `<MotionConfig reducedMotion="user">` wraps the app (§11.11).

API, SEO, CONFIG & DEPLOYMENT

The invisible half of the project: five route handlers, a complete metadata setup, security and caching headers, and a one-click Vercel deployment.

9.1 The five API routes

All five live under `app/api/` (177 lines in total). They describe the tower convincingly, but **all of them are simulated**, and **no component on the page calls any of them** (searching the components for `fetch(` or `/api/` finds nothing).

ROUTE	METHODS	RENDERING	WHAT IT RETURNS
`/api/health`	GET	○ static	A health report (Firebase RTDB 8–16 ms, ML model "tflite_v1.2.0" with 340 ms inference and 91.3% accuracy, a "mesh" of 6 nodes…) with random latencies. **Frozen at build time**: the timestamp and "random" values never change (verified: identical across requests).
`/api/stats`	GET	○ static	Project statistics: monthly water cost ₹180 vs a ₹310 "manual equivalent" (42% saving), `waterSaved: "42%"`, sensor reads at 1 Hz, and invented usage figures (142 "active devices", 89 "registered users", top cities).
`/api/weather`	GET	○ static	A fixed 5-day Delhi forecast and irrigation rules (pre-irrigate at 22:00 above 38 °C; suspend when rain is expected).
`/api/telemetry`	GET, POST	ƒ dynamic	GET: fresh random sensor values for a "Tulsi" plant profile on every request. POST: echoes the payload back.
`/api/contact`	GET, POST	ƒ dynamic	POST validates name, email and message (with an email regex) and returns a fake transmission ID such as `VERDE-YE9F-QS10`. **The message is not stored or sent anywhere.**

app/api/contact/route.ts lines 3–32 validation, then a fake ID; nothing is delivered

```ts
export async function POST(request: NextRequest) {
  try {
    const body = await request.json();
    const { name, email, message, subject } = body;

    if (!name || !email || !message) {
      return NextResponse.json(
        { error: "Missing required fields", required: ["name", "email", "message"] },
        { status: 400 }
      );
    }

    const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
    if (!emailRegex.test(email)) {
      return NextResponse.json(
        { error: "Invalid email format" },
        { status: 400 }
      );
    }

    const transmissionId = `VERDE-${Math.random().toString(36).substring(2,
6).toUpperCase()}-${Date.now().toString(36).substring(4).toUpperCase()}`;

    return NextResponse.json({
      success: true,
      message: "Transmission received. Our neural mesh will process your inquiry.",
      transmissionId,
      timestamp: new Date().toISOString(),
      protocol: "HTTPS // TLS 1.3",
      estimatedResponse: "< 24 hours",
    }, { status: 200 });
```

NOT USED

These routes are a holdover from generation 2, which had a Dashboard and a Contact section that may have used them. In v3 they are deployed, documented in the README and DEPLOYMENT.md, and reachable by anyone, but unused by the site itself. Chapter 12 suggests either wiring them up honestly or removing them.

9.2 SEO and sharing metadata

`app/layout.tsx` uses the Next.js Metadata API to emit every tag a search engine or chat app needs:

FIELD	VALUE
Title	“Project Verde — We grow the future.” with template `%s · Project Verde`
Description	“An autonomous, closed-loop vertical growing system. Four tiers, twenty growing sites, on-device AI and 95% less water than soil.”
Keywords / authors	Project Verde, vertical farming, hydroponics, IoT, ESP8266, Firebase, smart irrigation · Anuj Phulera, Aarav Choudhary
Open Graph	title, description, url, siteName, type website, locale `en_IN`
Twitter	`summary_large_image` card. *No image is provided*, so shares show no preview picture (§12.2).
Robots	index + follow; Googlebot `max-image-preview: large`
Canonical / base URL	`alternates.canonical "/"`, `metadataBase` from the SITE chain below
Viewport	theme colour `#000000`, colour scheme `dark`
Icons	`/favicon.svg`: a green (#22c55e) “A”-shaped glyph on black, *not* the leaf logo used in the nav (§11.9)

app/layout.tsx lines 9–15 resolving the canonical site URL

```
9    const SITE =
10     process.env.NEXT_PUBLIC_SITE_URL ??
11     (process.env.VERCEL_PROJECT_PRODUCTION_URL
12       ? `https://${process.env.VERCEL_PROJECT_PRODUCTION_URL}`
13       : process.env.VERCEL_URL
14         ? `https://${process.env.VERCEL_URL}`
15         : "http://localhost:3000");
```

The chain prefers an explicit custom domain, then Vercel's production URL, then the per-deployment URL, and finally localhost. `app/robots.ts` (allow all, disallow `/api/`, link the sitemap) and `app/sitemap.ts` (one URL, monthly) use a shorter version of the same chain without `VERCEL_URL`. The Google verification file proves ownership in Search Console.

9.3 next.config.mjs, explained

SETTING	VALUE	WHY
`reactStrictMode`	true	Double-invokes effects in development to surface bugs.
`poweredByHeader`	false	Do not advertise "X-Powered-By: Next.js".
`compress`	true	gzip responses from `next start`.
`images.unoptimized`	true	"Keeping it off avoids a needless serverless function on Vercel". The site has no raster images anyway.
`compiler.removeConsole`	production: keep error/warn	Strip `console.log` noise from shipped bundles.
`/api/:path*` headers	CORS `*`, GET/POST/OPTIONS, `Cache-Control: no-store`	Callable from anywhere; never cached by the browser.
`/vendor/:path*`	`public, max-age=31536000, immutable`	"~2 MB of vendored three.js — immutable, never revalidate".
`/scenes/:path*`	`max-age=86400, must-revalidate` + `X-Frame-Options: SAMEORIGIN`	Scenes can change between deploys; only this site may frame them.
`/:path*`	`nosniff`, `strict-origin-when-cross-origin`, DNS prefetch on	Baseline security headers.
`webpack` (dev only)	poll every 300 ms	"A local Windows-watcher workaround only… must never reach production."

VERIFIED

Verified against the running server with `curl -I`: `/vendor/three.module.js → Cache-Control: public, max-age=31536000, immutable · X-Content-Type-Options: nosniff`; `/scenes/tower.html → max-age=86400, must-revalidate · X-Frame-Options: SAMEORIGIN`; `/api/health → Access-Control-Allow-Origin: * · Cache-Control: no-store`. No `X-Powered-By` header is sent.

9.4 Deployment files

FILE	CONTENT	PURPOSE
`vercel.json`	framework nextjs, build `next build`, install `npm ci`, regions `["bom1"]`, github silent	Reproducible installs from the lockfile; Mumbai edge; no bot comments on commits

FILE	CONTENT	PURPOSE
`.nvmrc`	`20`	Node version for local development and CI
`package.json` engines	`node >= 18.18.0`	The minimum required by Next 14
`.env.example`	only `NEXT_PUBLIC_SITE_URL` (commented out)	Needed only for a custom domain
`.gitignore`	node_modules, .next, .vercel, env files, `.preview/` ("local model preview harness"), `nul`	`nul` is a Windows artefact, more evidence of a Windows workstation
`DEPLOYMENT.md`	Step-by-step Vercel guide, route rendering table, caching notes	Accurate (re-verified in Appendix C)

9.5 What actually gets downloaded

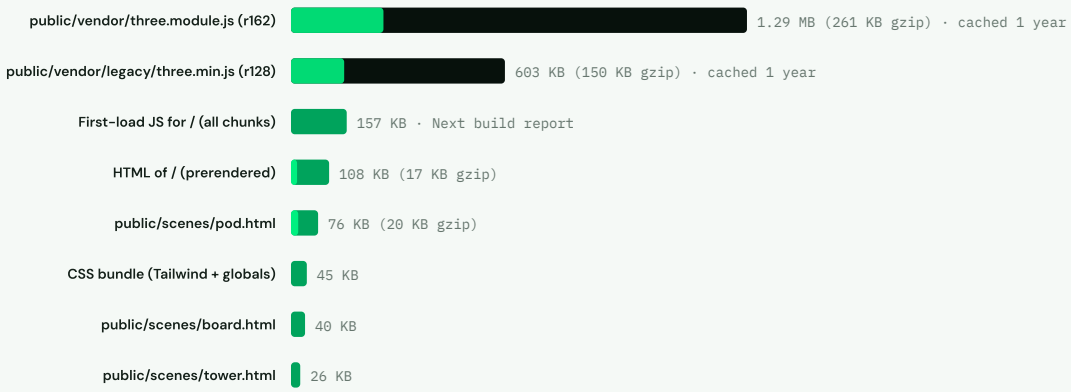


FIGURE 9.1 Asset sizes on disk (dark bars: vendored three.js, cached for a year) with gzip transfer sizes overlaid in bright green where measured. The page's own code is small; the 3D engine dominates but loads in parallel inside the iframes.

The build reports **157 kB of first-load JavaScript** (70.1 kB for the page plus 87 kB shared framework chunks), a **44.5 kB** CSS bundle and self-hosted woff2 fonts. The prerendered HTML is 107 KB, or 17 KB gzipped. The first visit also downloads both three.js builds (≈410 KB gzipped together) and the three scenes, but all of it is cacheable.

03

PART III

WHAT WAS NOT USED

Removed packages, deleted files, leftovers still sitting in the repository, ideas that were planned but never built, technologies deliberately avoided, and the numbers that are simulated rather than measured.

→ Chapter 10 · Not used, and why

→ 10.1 Removed packages

→ 10.2 Removed files & what replaced them

→ 10.3 Leftovers still in the repo

→ 10.4 The README versus reality

→ 10.5 Simulated versus real

NOT USED, AND WHY

A project is defined as much by what it leaves out. Verde's history includes a large, deliberate clean-out, but also leftovers, a stale README and simulated data that a careful reader (or judge) will notice.

10.1 Packages that were removed

The deployment commit (`592fce4`) removed six packages. Each removal has a clear reason:

PACKAGE	VERSION	WHAT IT WAS FOR	WHY IT WENT	REPLACED BY
three	^0.162.0	The 3D engine for the React versions of the scenes	Scenes moved into iframes that load their own vendored copy	`public/vendor/three.module.js`
@react-three/fiber	^8.15.0	React renderer for three.js (VerdeTower.tsx, Scene3D.tsx)	The iframe architecture made it unnecessary (§5.8)	iframes + postMessage
@react-three/drei	^9.88.0	R3F helpers: OrbitControls, Wireframe	Same; only used by the deleted Scene3D	three.js addons in /vendor
clsx	2.1.1	Joining class names in `app/lib/utils.ts → cn()`	No v3 component used `cn()`	Template literals
tailwind-merge	2.3.0	Resolving conflicting Tailwind classes in `cn()`	Same	—
vercel (dev)	^54.10.2	Command-line deploys	Deploys happen through the GitHub integration; the CLI is a very large install	Vercel Git integration

10.2 Files that were removed, and what replaced them

Git records **66 deleted files** in that commit (the message says 70): every generation-2 file that `page.tsx` no longer imported. Many had a direct successor in v3, so the ideas survived and the code was rewritten:

REMOVED (GENERATION 2)	SUCCESSOR IN V3
BootSequence (section)	**BootCurtain.tsx**, driven by real scene readiness
NavBar · SmoothScrollProvider · ScrollProgressRing	**site/Nav.tsx** · **SmoothScroll.tsx** · **ScrollProgress.tsx** (bar instead of ring)
ParticleCanvas	**NeuralField.tsx** (linked "mycelium" lattice, cursor repulsion)
Marquee · MegaMarquee (lived for 29 minutes)	**VelocityMarquee.tsx** (speed, direction and skew follow scroll velocity)
TextScramble · GlitchText · GlitchCorruptionTitle · NeonText	**ScrambleText.tsx** and the CSS `.glitch` / `.glow-grow` classes
MagneticButton · MagneticCursorTrail	**Magnetic.tsx** and **CustomCursor.tsx** (dot + ring, no trail)
TiltCard	The `Tilt` wrapper inside **Pillars.tsx**
Reveal · SectionReveal · SectionTransitionWipe · ParallaxLayer	framer-motion `whileInView`, **SplitText**, **TextReveal**, scroll transforms

REMOVED (GENERATION 2)	SUCCESSOR IN V3
Terminal · TerminalText	**LiveTerminal.tsx**
Sparkline · SVGGauge · Dashboard (section)	Sparkline and gauge rebuilt inside **Telemetry.tsx**
Hardware (section) · HardwareDiagram · LivingCircuitBoard	**BoardSection** (live 3D board) + **SpecExplorer**
NoiseOverlay · Scanline · GridBackground	CSS `.bg-noise`, `.scanline-fx`, `.verde-grid`
Team · Hero · Footer (sections)	**TeamSection** · **Hero** · **site/Footer**
VerdeTower (R3F, 579 lines) · Scene3D (R3F)	The iframe scenes + **ScenePreloader**

And these were dropped with **no successor**, so the ideas themselves were cut:

REMOVED IDEA	WHAT IT DID	WORTH REVIVING?
KioskMode	Auto-advanced through sections when the URL had `?kiosk=true`, for an unattended screen	**Yes, for exhibition day** (§12.3): a laptop on the stand could loop the site on its own.
QRCodeSection	Showed a QR code for visitors (its generator was a placeholder, not a real QR encoder)	Yes, as a printed QR or a real library such as `qrcode`.
EasterEggs	Konami code, CRT and typewriter modes (V and T keys)	Optional fun; the README still advertises it.
ThemeToggle	Light/dark switch	No: v3 is dark by design.
usePerformanceTier	Low/medium/high device tiers to scale effects	Partly: the scenes would benefit from pausing (§11.10).
useKeyboardNavigation · **useReducedMotion**	Keyboard section navigation; a reduced-motion hook	Reduced motion is handled inline now; keyboard nav is partly replaced by ⌘K.
Sections: About, Awards, CTA, Code, Contact, Gallery, Infographic, Intelligence, Modes, PlantLibrary, Projects, Skills, Testimonials, UseCase	Portfolio-style pages: firmware code viewer, plant database (Tulsi, tomato…), awards, mentor quotes, gallery	Selectively: a real **Gallery** of the physical build and a **Code** excerpt of the firmware would add credibility.
Others: DNAHelix, DataPacketFlow, PlantGrowthAnimation, WaterDropletEffect, RippleEffect, ThermalHoverReveal, HeroCopy, InteractiveTimeline	Decorative animations	No: v3's motion is stronger and more coherent.

10.3 Leftovers: still in the repository, unused today

ITEM	EVIDENCE	IMPACT	SUGGESTION
`THEME` Custom Tailwind theme	Colours `primary/verde/accent/dark`, 3 font stacks, **14 animations**, 3 shadows, 2 easings, 2 gradients in `tailwind.config.ts`: **0 uses** in any component	None at runtime (Tailwind only emits used classes), but misleading to readers	Delete the `extend` block or replace it with the real tokens
`CSS` Dead CSS classes	`.reveal`, `.verde-aurora` (+2 keyframes), `.mega-marquee` (+1 keyframe), `.btn-solid`, `.hairline`: defined in globals.css, never used	≈ 80 lines of shipped CSS	Remove
`API` 5 API routes	No `fetch` anywhere in the UI	Deployed endpoints that return fake data	Wire them to real data or delete (§12.2)

ITEM	EVIDENCE	IMPACT	SUGGESTION
FILE `public/manifest.json`	Not linked from the metadata; its icon `/favicon.ico` returns **404**	No install prompt; a broken reference	Link it via `metadata.manifest` and point the icon at `favicon.svg`
FILE `model1/2/3.html` (root)	Outside `public/`, so never served	2,445 lines of reference code	Move to `docs/prototypes/` and say so in the README
FILE Duplicate verification files	`google280c…html` exists at the root, in `app/` and in `public/`; only the `public/` copy is served	Clutter	Keep only `public/`
DOC `docs/3d-model-prompts.md`	No GLB/GLTF or loader in the project	Documents a pipeline that was never adopted	Keep as history; mark it "superseded by procedural scenes"
CODE Hidden `SceneSlot` in page.tsx	0×0 px, and slots under 2×2 px are skipped by the glue loop	None (the comment says it "keeps the tower warm", but the preloader mounts the tower anyway)	Remove
CONFIG `@/*` path alias	Configured in tsconfig; 0 imports use it	None	Use it or drop it
CONFIG Tailwind `./components/**` glob	That folder does not exist	None	Remove the entry
PROP `ScrambleText speed`	Accepted as a prop and listed in effect dependencies, but the algorithm never reads it	None	Implement or remove
CMDS Scene commands	The tower's `lights` / `flow` and the board's `led` are never sent	None	Add them to the command palette, a cheap "wow"
UI Hidden scene UI	Each scene still ships its full product-page UI (panels, intros, loaders), hidden with CSS	Extra bytes in each scene	Fine to keep: the scenes still work standalone
NET Google Fonts in scenes	tower.html and pod.html `@import` fonts.googleapis.com (line 9)	A third-party request, contradicting "no third-party runtime dependencies"	Self-host or drop the imports (§11.12)

10.4 The README versus reality

The README was written for generation 2 and was never updated for v3. Anyone who reads it first (a judge, a teacher, a recruiter) gets the wrong picture:

README CLAIMS	REALITY IN V3.0.0
Fonts: **Space Grotesk + JetBrains Mono**	DM Sans, Manrope, Oswald and IBM Plex Mono
3D: **Three.js + React Three Fiber + Drei**	Vendored three.js inside iframes; R3F and Drei were removed
Structure: `verde-portfolio/` with `hooks/`, `sections/` and components such as DNAHelix, DataPacketFlow, EasterEggs, GlitchText, LivingCircuitBoard, ParticleCanvas, Scene3D, ThemeToggle, TiltCard…	None of these folders or components exist; the real list is in Appendix A
Features: performance-adaptive 3D background, DNA helix, plant-growth animation, living circuit board, data-packet flow, **performance tiers**	All removed; the hero is "3D-free" by design
Accessibility: **keyboard navigation, skip links**	No skip link exists; the keyboard-navigation hook was removed (⌘K partly replaces it)

README CLAIMS	REALITY IN V3.0.0
Easter eggs: **Konami code**, typing **"verde"**	Removed with EasterEggs.tsx
Custom cursor "with circuit trail", "ripple effects on click"	Dot + ring cursor; no trail, no ripples
Theme: "green-on-black neon with **cyan/purple** accents"	Green + lime; cyan and purple are unused
Clone: `git clone …/verde-main-portfolio.git`	The repository is `UCHIHA-MADARA-ANUJ/Verde-CBSE`
Scripts: dev, build, start, lint	Also `typecheck` and `verify`

Still accurate: the system description, the API route list, the team table, the stack badges for Next 14, React 18, TypeScript 5.3, Tailwind 3.4 and Framer Motion 11, and the three.js 0.162 badge (the vendored module is r162).

10.5 Simulated versus real

This is the most important table in Part III for an exhibition. **None of the numbers on the website come from the physical tower.** That is normal for a showcase site, but the copy sometimes implies otherwise:

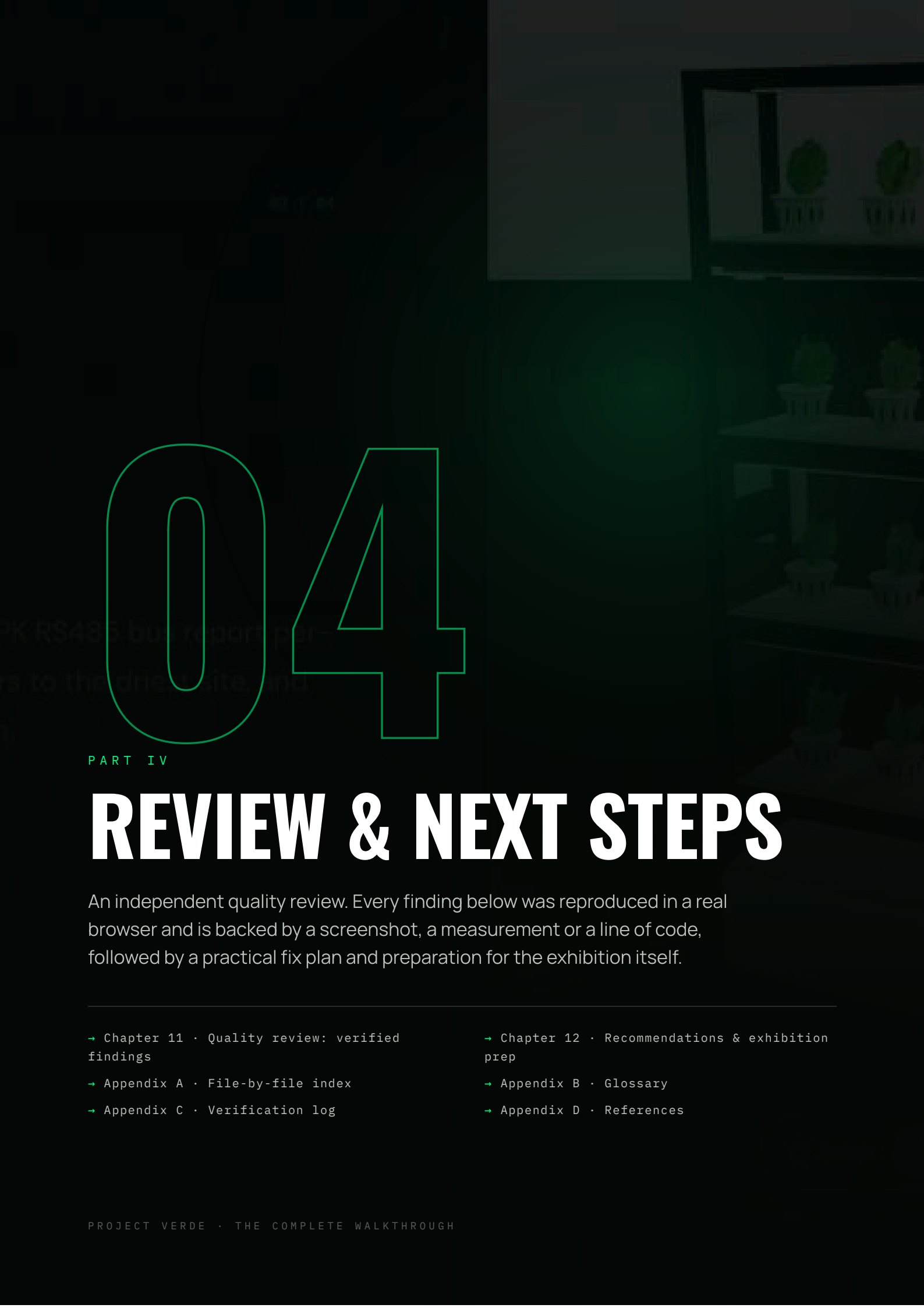
WHERE	WHAT THE VISITOR SEES	WHAT ACTUALLY HAPPENS	SUGGESTED HONEST WORDING
Hero HUD	`FLOW: 2.44 L/MIN` changing; IST clock	Flow is `2.1 + random × 0.5` every 0.9 s; the clock is real	"Simulated readout", or feed it from Firebase
Hero cards	Water 21.4 °C, pH 6.21, reservoir 82%, canopy 0.94	Hard-coded	Fine as illustrative values
Live console	"This is the **real event stream**"	A loop of 15 scripted messages, one every 1.4 s	"A replay of a typical hour on the tower"
Telemetry	"Streaming · **Firebase RTDB**" + live sparklines	Random walks generated in the browser	"Simulated from real sensor ranges", or connect the RTDB
Reservoir gauge	82%, "sampled every 30 s"	Static 82	—
Comparison	"**Measured** … over three full cycles, in the same Delhi conditions"	Static numbers in the component	Keep only if logs/photos exist; otherwise "expected" or "estimated"
Boot log	Firmware boot, MEM_ALLOC counter	Scripted lines; random memory number	Fine: it reads as theatre
3D tower panel, pod "68% moisture"	Live-looking values	Static labels	—
/api/* routes	Health, stats, weather, telemetry	Mock data (health frozen at build time)	Label as "demo API"

> **WHY THIS MATTERS**
>
> Judges respect a clear line between *"this is what the device does"* and *"this is a simulation to show you what it does"*. One small badge such as "SIMULATED DATA" on the console and telemetry panels, or one sentence in the FAQ, removes the risk entirely. Even better: connect the Firebase RTDB read-only and show **one** real number live on exhibition day.

10.6 Technologies deliberately not used

NOT USED	WHY IT ISN'T NEEDED (TODAY)	WHEN YOU WOULD ADD IT
A database / CMS	All content is static and versioned in code	If non-developers need to edit text

NOT USED	WHY IT ISN'T NEEDED (TODAY)	WHEN YOU WOULD ADD IT
Authentication	Nothing is private	For a control panel that sends commands to the tower
State libraries (Redux, Zustand)	One React context (scenes) and local state cover everything	Never, at this size
UI kits (MUI, shadcn) / CSS-in-JS	A bespoke look built with Tailwind and one stylesheet	—
3D model files (GLB/GLTF)	Procedural geometry is smaller and animatable (§7.6)	For photoreal scans of the real tower
Raster images / next/image	Every visual is drawn: CSS, SVG, canvas, WebGL	For photos of the real build (recommended)
Google Fonts / CDNs (main page)	Self-hosted fonts and vendored three.js	—
Analytics / error monitoring	Not required for an exhibition site	If the site is shared widely
Internationalisation	English only (although alerts are EN/HI)	A Hindi version would suit the audience
Tests & CI	No unit, end-to-end or visual tests; no `.github/workflows`	**Now**: a Playwright smoke test would have caught §11.2–11.6
PWA / offline cache	Not configured	For exhibition halls with weak Wi-Fi



04

PART IV

REVIEW & NEXT STEPS

An independent quality review. Every finding below was reproduced in a real browser and is backed by a screenshot, a measurement or a line of code, followed by a practical fix plan and preparation for the exhibition itself.

→ Chapter 11 · Quality review: verified findings

→ Appendix A · File-by-file index

→ Appendix C · Verification log

→ Chapter 12 · Recommendations & exhibition prep

→ Appendix B · Glossary

→ Appendix D · References

QUALITY REVIEW: VERIFIED FINDINGS

The site is ambitious and mostly well built, which is exactly why its few visible defects matter. Two of them hide content or break the look in every major browser; most of the rest take minutes to fix.

11.1 Method and findings index

The production build ran locally and was driven by headless Chromium 131 with software WebGL (SwiftShader). Each issue was reproduced at least twice; layout bugs were measured with `getBoundingClientRect`; rendering bugs were confirmed with an A/B experiment. Severity: **High** = content missing or obviously broken for most visitors; **Medium** = broken in some situations or devices; **Low** = polish; **Info** = maintenance, accuracy or risk.

§	FINDING	SEVERITY	EFFORT TO FIX
11.2	Manifesto headline “SOIL IS RUNNING OUT.” never appears	HIGH	5 minutes
11.3	3D frames show opaque white areas (iframe color-scheme mismatch)	HIGH	1 line per scene
11.4	Pod scene: white fog horizon and a light toolbar on the dark page	MEDIUM	2 lines
11.5	Loop deck: fixed travel distance breaks phones and desktops; active card off-screen	MEDIUM	1–2 hours
11.6	Tour chapter buttons jump to the wrong chapter	MEDIUM	1 line
11.7	The tower (main product) never appears on phones or tablets	MEDIUM	30 minutes
11.8	Headlines overflow or overlap (“MICROCONTROLLER”, “CONTINUOUSLY”)	LOW	15 minutes
11.9	Brand leftovers and broken references (“VERD” label, favicon, manifest, numbering)	LOW	15 minutes
11.10	Performance: all three WebGL scenes render every frame, even when invisible	INFO	1–2 hours
11.11	Accessibility gaps (dialog semantics, contrast, motion preferences, skip link)	INFO	1–2 hours
11.12	Maintenance and security: Next 14.2.5 advisory, Google Fonts in scenes, frozen API, stale README	INFO	30 minutes
11.13	Accuracy of technical claims (for judges)	INFO	wording

11.2 The manifesto headline never appears HIGH

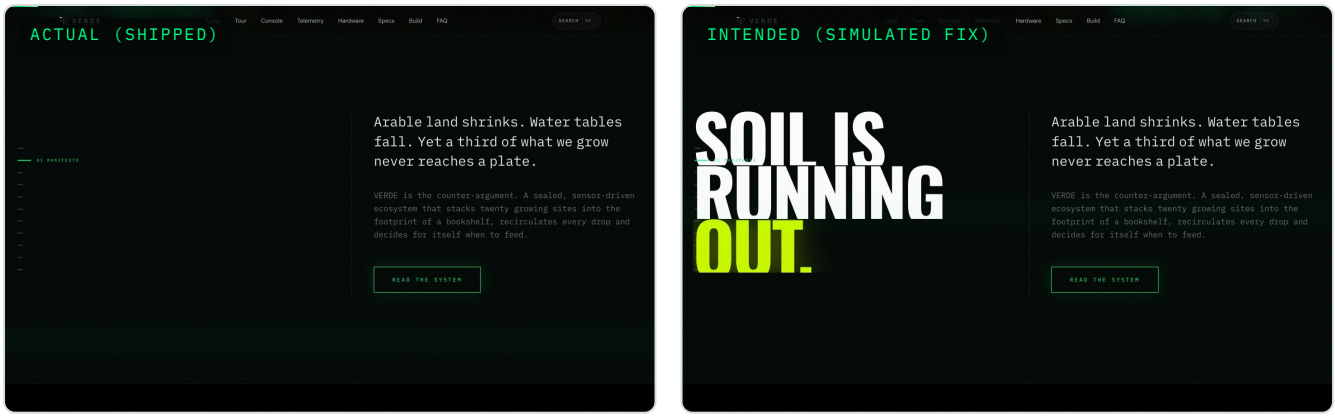


FIGURE 11.1 Left: the manifesto as every visitor sees it. The left column is empty. Right: the same frame with the animation's final state forced, showing "SOIL IS RUNNING OUT." as designed.

Evidence. After scrolling to the section, and separately after 40 real mouse-wheel steps through it, all three lines report `opacity: 0` with their initial transform. The measured rectangles explain why: each animated line has **zero height** (rotated -90° about its bottom edge) and sits **150 px below** its parent, which has `overflow: hidden`.

app/components/Manifesto.tsx lines 21–41 — whileInView is attached to a clipped, zero-height element

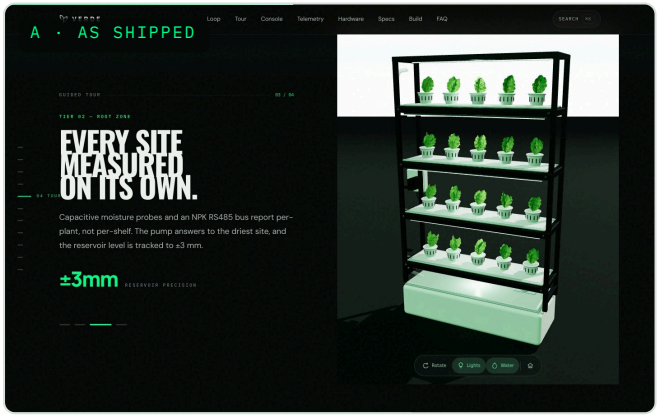
```
21  <h2
22    className="font-display-xl t-giant text-white"
23    style={{ perspective: 1000 }}
24    aria-label="Soil is running out."
25  >
26    {LINES.map((l, i) => (
27      <span key={l} className="block overflow-hidden">
28        <motion.span
29          aria-hidden
30          className={`block ${i === 2 ? "c-acid drop-shadow-[0_0_30px_rgba(204,255,0,0.4)]" : ""}`}
31          style={{ transformStyle: "preserve-3d", transformOrigin: "bottom" }}
32          initial={{ y: 150, opacity: 0, rotateX: -90, skewY: 8 }}
33          whileInView={{ y: 0, opacity: 1, rotateX: 0, skewY: 0 }}
34          viewport={{ once: true, margin: "-20%" }}
35          transition={{ duration: 1.5, delay: i * 0.14, ease: [0.16, 1, 0.3, 1] }}
36        >
37          {l}
38        </motion.span>
39      </span>
40    ))}
41  </h2>
```

Cause. framer-motion's `whileInView` uses an IntersectionObserver on the element it is attached to. Here that element starts outside its `overflow-hidden` parent and flattened to zero height, so it is clipped and **never intersects** the viewport. The animation never starts, and with `once: true` nothing retries. Notably, `SplitText.tsx` uses the correct pattern: the observer is on the *parent* heading and the words animate through variants. That is why every other headline works.

FIX: OBSERVE THE PARENT, ANIMATE THE CHILDREN (THE SPLITTEXT PATTERN)

```
<motion.h2
  className="font-display-xl t-giant text-white"
  style={{ perspective: 1000 }}
  aria-label="Soil is running out."
  initial="hidden"
  whileInView="show"                        // observe the visible parent instead
  viewport={{ once: true, margin: "-20%" }}
>
  {LINES.map((l, i) => (
    <span key={l} className="block overflow-hidden">
      <motion.span
        aria-hidden
        className={`block ${i === 2 ? "c-acid ..." : ""}`}
        style={{ transformStyle: "preserve-3d", transformOrigin: "bottom" }}
        variants={{
          hidden: { y: 150, opacity: 0, rotateX: -90, skewY: 8 },
          show: { y: 0, opacity: 1, rotateX: 0, skewY: 0,
                  transition: { duration: 1.5, delay: i * 0.14, ease: [0.16, 1, 0.3, 1] } },
        }}
      >{l}</motion.span>
    </span>
  ))}
</motion.h2>
```

11.3 3D frames show opaque white areas HIGH



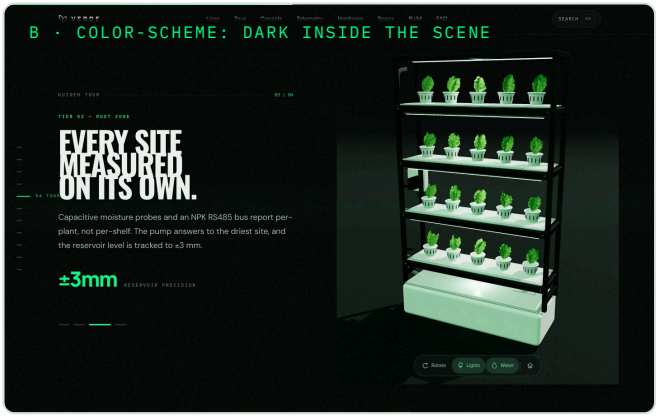


FIGURE 11.2 A/B experiment on the tour (chapter 3). Left: wherever the camera sees empty space, the scene shows an opaque white block. Right: the only change is setting color-scheme: dark on the scene's own document, and the tower floats seamlessly on the page as intended.

Cause. The site declares a dark colour scheme (`viewport.colorScheme: "dark"`) and `globals.css` sets `color-scheme: dark` on the iframes. The scene documents never declare one, so they are "normal" (light). The CSS specification says that when an iframe element's colour scheme differs from its embedded document's, the browser **must paint an opaque canvas** in the document's colour, white for light. Chrome, Edge and Firefox all follow this rule, so the WebGL canvases' transparent pixels show white instead of the page.

app/globals.css lines 234–241 iframe declares dark…

```
234  .scene-frame {
235    width: 100%;
236    height: 100%;
237    border: 0;
238    display: block;
239    background: transparent;
240    color-scheme: dark;
241  }
```

FIX: MAKE BOTH SIDES AGREE

Add one line to the `<head>` of each scene (tower.html, pod.html, board.html):

```
<meta name="color-scheme" content="dark">
```

Equivalent: add `:root{color-scheme:dark}` to each `#verde-embed` style block. (Removing `color-scheme: dark` from `.scene-frame` also works.) Verified: the A/B frames above differ only by this property.

11.4 Pod scene: white fog horizon and a light toolbar MEDIUM

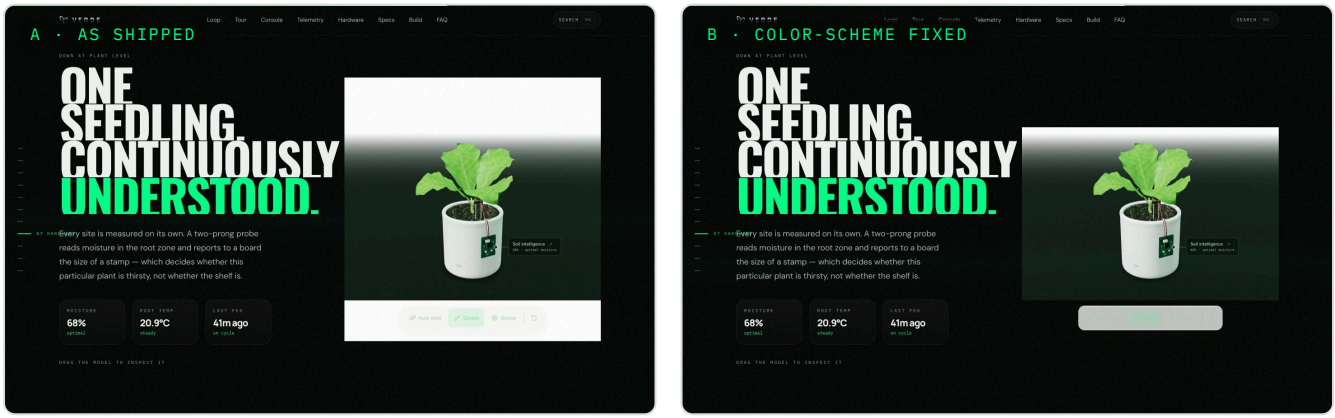


FIGURE 11.3 Even after the §11.3 fix (right), the top of the pod frame fades to white and the viewer toolbar stays light grey.

public/scenes/pod.html lines 413–428 background = null is passed to Fog and setClearColor

```
413  const stage = $('stage');
414  const scene = new THREE.Scene();
415  const background = null;
416  scene.background = background;
417  scene.fog = new THREE.Fog(background, 15, 38);
418
419  const renderer = new THREE.WebGLRenderer({ antialias: true, alpha: true, powerPreference: 'high-performance'
     });
420  renderer.setPixelRatio(Math.min(window.devicePixelRatio, window.innerWidth < 761 ? 1.75 : 2));
421  renderer.outputColorSpace = THREE.SRGBColorSpace;
422  renderer.toneMapping = THREE.ACESFilmicToneMapping;
423  renderer.toneMappingExposure = 1.06;
424  renderer.shadowMap.enabled = true;
425  renderer.shadowMap.type = THREE.PCFSoftShadowMap;
426  renderer.shadowMap.autoUpdate = false;
427  renderer.setClearColor(background, 1);
428  stage.prepend(renderer.domElement);
```

Cause 1: white fog. When the prototype was converted to a transparent background, `background` became `null` and was still passed to `new THREE.Fog(background, 15, 38)`. three.js's `Color` starts as white and `set(null)` changes nothing, so the fog is **white** and the distant floor fades to white. **Cause 2: light toolbar.** The embed stylesheet re-skins `.toolbar, .tools`, but pod.html's toolbar class is `.viewer-tools`, so the light glass pill survives.

FIX

```
scene.fog = new THREE.Fog('#050a08', 15, 38);      // same dark fog as the tower
/* in <style id="verde-embed"> */
.viewer-tools{background:rgba(10,18,15,.72)!important;border-color:rgba(110,231,168,.18)!important}
```

11.5 Loop deck: a fixed travel distance that only fits tablets MEDIUM

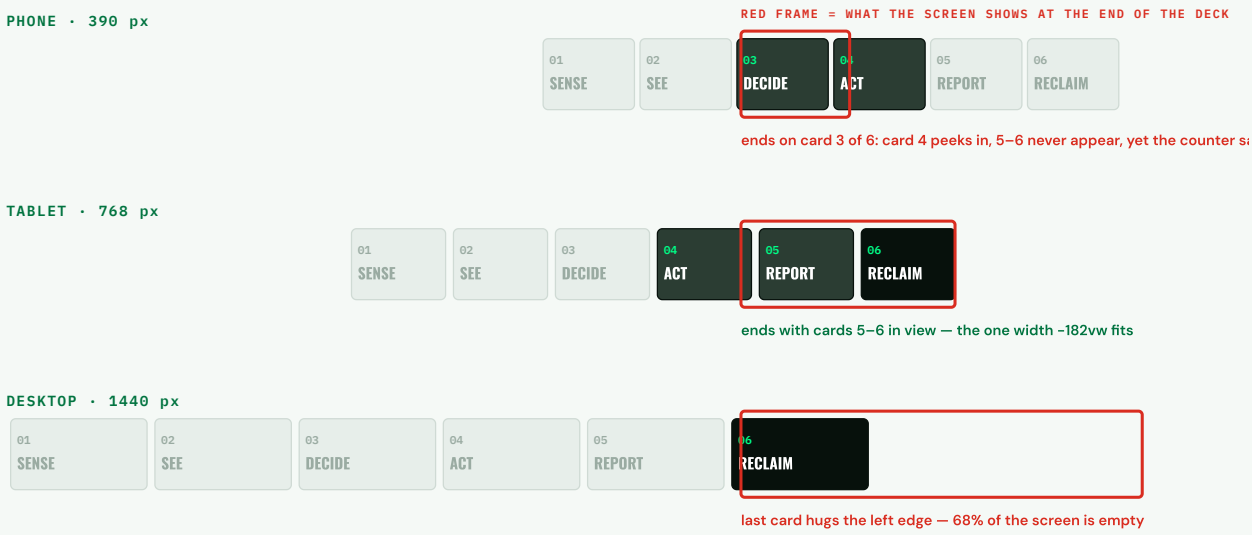


FIGURE 11.4 Where the six-card track ends at three screen widths (drawn to scale from the CSS: card width 84/44/34vw, gaps 20/28 px, travel from 4vw to -182vw). Only the tablet layout ends in the right place.

Evidence (measured). Desktop, 1440 px, end of deck: the active card RECLAIM spans x = -33…457 px and **68% of the screen is empty**. Desktop at 88.5%: the counter says **05** while card 05 is scrolled off the left edge and the blurred card 06 sits mid-screen. Phone, 390 px, end of deck: the counter says **06** but the visible card is **03 DECIDE**; card 04 shows only a 46 px sliver at the right edge, and cards 05–06 sit at x ≥ 692 px and are **never shown**.

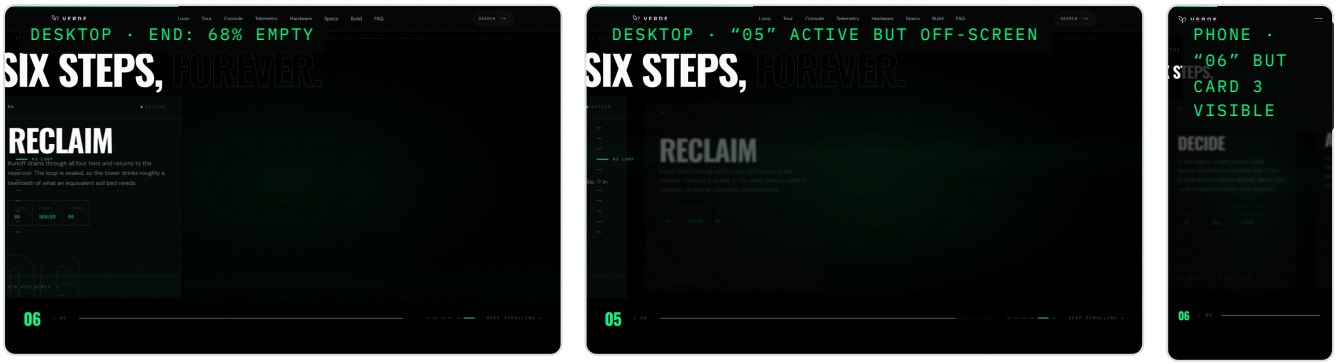


FIGURE 11.5 The three symptoms, captured from the production build.

app/components/HorizontalDeck.tsx lines 158–167 a constant travel, and an index unrelated to geometry

```
158  // travel: hold a beat at each end so the first and last card can breathe
159  const x = useTransform(eased, [0, 0.06, 0.94, 1], ["4vw", "4vw", "-182vw", "-182vw"]);
160  const bar = useTransform(eased, [0, 1], ["0%", "100%"]);
161  const headX = useTransform(eased, [0, 1], [0, -70]);
162  const glow = useTransform(eased, [0, 0.5, 1], [0.05, 0.16, 0.05]);
163
164  useMotionValueEvent(eased, "change", (v) => {
165    const n = Math.min(CARDS.length - 1, Math.max(0, Math.round(v * (CARDS.length - 1))));
166    setIdx((p) => (p === n ? p : n));
167  });
```

FIX

Measure instead of guessing: compute the travel from the real track width, and pick the active card by geometry.

```
const track = useRef<HTMLDivElement>(null);
const [travel, setTravel] = useState(0);
useLayoutEffect(() => {
  const measure = () => setTravel(Math.max(0, track.current!.scrollWidth - window.innerWidth * 0.92));
  measure(); window.addEventListener("resize", measure);
  return () => window.removeEventListener("resize", measure);
}, []);
const x = useTransform(eased, [0, 0.06, 0.94, 1], [0, 0, -travel, -travel]);
// active = the card whose centre is nearest the viewport centre (getBoundingClientRect)
```

11.6 Tour chapter buttons jump to the wrong chapter MEDIUM

Evidence. Clicking the four chapter pips in order lands on chapters `01, 03, 04, 04` (read from the on-screen counter after each click).

Cause. The section is 400vh tall but only 300vh of it is scroll distance (sticky content stays in view for the last screen). Chapter *i* therefore begins at `0.75 × i` screens, while the button scrolls to `i + 0.5` screens.

PIP	SCROLLS TO	PROGRESS	LANDS ON
1	0.5 vh	0.17	01 ✓
2	1.5 vh	0.50	**03 ✗**
3	2.5 vh	0.83	**04 ✗**
4	3.5 vh	>1.0	04 (past the section)

app/components/TowerStory.tsx lines 104–117

```
104  {chapters.map((ch, i) => (
105    <button
106      key={ch.k}
107      aria-label={ch.k}
108      onClick={() =>
109        window.scrollTo({
110          top:
111            (ref.current?.offsetTop ?? 0) +
112            (i + 0.5) * window.innerHeight,
113          behavior: "smooth",
114        })
115      }
116      className="group h-8 flex items-end"
117    >
```

FIX

```
top: (ref.current?.offsetTop ?? 0) +
     ((i + 0.5) / chapters.length) *
     (ref.current!.offsetHeight -
window.innerHeight),
```

11.7 The tower never appears on phones or tablets MEDIUM

The tour's only tower slot is `hidden lg:block`, so below 1024 px it is `display: none`, 0×0 and ignored by the preloader. A comment promises "mobile: model sits behind, dimmed", but that container holds only a gradient and **no SceneSlot**. The hidden slot in page.tsx is also 0×0. On a phone the guided tour is text-only, and **the tower, the project's hero product, is never seen anywhere on the site**. Camera commands are still sent to an invisible scene.

app/components/TowerStory.tsx lines 130–137

```
130      {/* the live model keeps its own column */}
131      <SceneSlot id="tower" className="hidden lg:block h-
         [86vh] -mr-10" />
132    </div>
133
134    {/* mobile: model sits behind, dimmed */}
135    <div className="lg:hidden absolute inset-0 -z-0 opacity-30
       pointer-events-none">
136      <div className="absolute inset-0 bg-gradient-to-t from-
       [var(--bg)] via-transparent to-[var(--bg)]" />
137    </div>
```

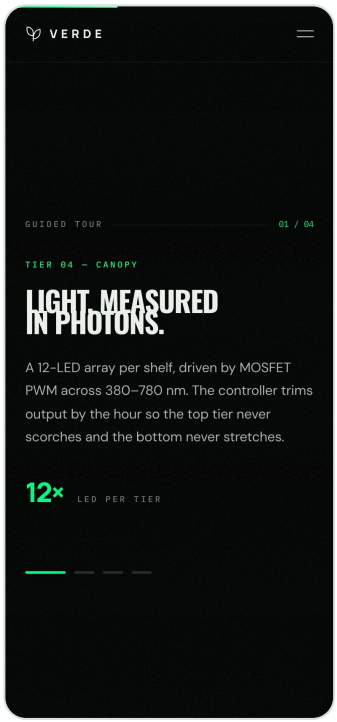


FIGURE 11.6 The tour on a 390 px phone: no tower.

FIX: GIVE MOBILE ITS OWN SLOT

```
<div className="lg:hidden absolute inset-0 opacity-40
pointer-events-none">
  <SceneSlot id="tower" className="absolute inset-0" />
  <div className="absolute inset-0 bg-gradient-to-t from-
[var(--bg)] via-transparent to-[var(--bg)]" />
</div>
```

11.8 Headlines overflow or overlap LOW



FIGURE 11.7 At 1440 px, single long words at .t-big size (7.5vw ≈ 108 px) are wider than their half-width columns. The tight leading of .font-display-xl (line-height 0.75) also makes wrapped lines overlap (also visible in the tour headings).

Fix: for the two long headlines use a smaller clamp (for example `text-[clamp(34px,5.2vw,6rem)]`) or allow hyphenation (`hyphens: auto` with `lang="en"`), and give multi-line display headings `leading-[0.86]`.

11.9 Brand leftovers and broken references LOW

ISSUE	EVIDENCE	FIX
The 3D tower's reservoir label reads **"V E R D · T O W E R 0 1"**	`tower.html` line 181: `labelTexture('V E R D', …)`, inherited from the prototype brand "VERD"	Change to `'V E R D E'`
The favicon is a green **"A"-shaped glyph**, not the Verde leaf	`public/favicon.svg` draws a chevron with a crossbar; the nav logo is a leaf, so browser tabs show an unrelated mark	Reuse the nav's leaf path in favicon.svg
`/favicon.ico` returns **404**	manifest.json lists it as the only icon; only favicon.svg exists	Point the manifest at `/favicon.svg` and link the manifest in metadata
HUD readouts collide with the fixed nav; side-rail labels overlap content at 1280–1440 px	See the crops below ("SEARCH ⌘K" over "FLOW…"; "04 TOUR" over the headline)	Offset the HUD by the nav height; show rail labels only on hover, or only ≥ 1536 px
Section numbers disagree	Content tags say 04 Loop, 05 Console, 07 Against the ground; the side rail says 03 Loop, 05 Console, 07 Hardware	Use one numbering source
"Fourteen weeks, in order."	The last timeline step is "Week 15+"	"Fifteen weeks…" or "Week 13–14"



FIGURE 11.8 Overlap 1 · the fixed nav sits on top of the hero's HUD readouts ("SEARCH ⌘K" over "FLOW: 2.44 L/MIN", the logo over "SYS.OPERATIONAL").



FIGURE 11.9 Overlap 2 · the side rail's active label ("04 TOUR") overlaps the tour headline at 1440 px.

11.10 Performance: invisible scenes keep rendering INFO

The preloader hides an off-screen scene with `opacity: 0`, but each scene's own animation loop keeps rendering every frame. Measured on a CPU renderer (which exaggerates GPU cost but shows the proportions): a tower frame took **≈1.5 s**, a pod frame **≈0.44 s** and a board frame **≈0.24 s**. With all three running, the tower slowed further (0.57 fps versus 0.65 fps alone). On a real GPU that is plenty fast, but it is **continuous work for the whole visit**: battery drain on laptops and phones, and fan noise on an exhibition laptop left running all day.

HOTSPOT	DETAIL	IMPROVEMENT
Three WebGL loops	Always on; the tower has hundreds of separate meshes (200 leaves + veins, 20 pots × 12 slots…)	Send `{action:"pause"}` / `"resume"` from the glue loop when a slot leaves or enters the viewport; merge or instance repeated meshes
NeuralField canvas	Up to 190 particles = 17,955 distance checks per frame, still running after you scroll past the hero	Pause with an IntersectionObserver
LiveTerminal	Appends a line every 1.4 s forever, even off-screen	Gate with `useInView`, as Telemetry already does
Two three.js builds	r162 (1.29 MB) and r128 (0.60 MB) are both downloaded	Port board.html to r162 and delete `vendor/legacy`

11.11 Accessibility INFO

DONE WELL

- ✓ `lang="en"`, semantic sections and heading order.
- ✓ Animated headings carry an `aria-label` with the full sentence; the split words are `aria-hidden`.
- ✓ FAQ buttons expose `aria-expanded`; the side rail sets `aria-current`; icon buttons are labelled.
- ✓ Each scene iframe has a descriptive `title`; each 3D stage an `aria-label`.
- ✓ A visible focus ring (`:focus-visible`, 2 px green).

GAPS

- ✗ The command palette has no `role="dialog"` / `aria-modal`, no focus trap and no listbox semantics.
- ✗ No skip link, although the README promises one.
- ✗ Many 9–11 px labels use white at 20–40% opacity (≈2:1–3.7:1 contrast), below WCAG AA's 4.5:1.
- ✗ framer-motion animations ignore the reduced-motion setting: wrap the app in `<MotionConfig reducedMotion="user">`.
- ✗ `cursor: none !important` hides the system cursor, including enlarged accessibility cursors.

- ✓ Reduced-motion support in CSS, Lenis, NeuralField and two scenes.
- ✓ The custom cursor is limited to fine pointers.
- ✕ On touch screens, OrbitControls sets `touch-action: none`, so a swipe that starts on a 3D model rotates it instead of scrolling the page. This is likely a "scroll trap"; test it on a phone.

11.12 Maintenance and security INFO

ITEM	DETAIL	ACTION
Next.js 14.2.5	npm flags it as vulnerable. The official December 2025 advisory lists 14.x as affected by a React Server Components denial-of-service issue (CVE-2025-55184, completed by CVE-2025-67779), fixed in **14.2.35**. Next 14 also reached end-of-life in October 2025. Practical risk for this mostly static site is low, but the upgrade is trivial.	`npm i next@14.2.35 eslint-config-next@14.2.35`; plan a move to Next 15/16
Google Fonts in scenes	tower.html and pod.html `@import` fonts.googleapis.com on line 9. DEPLOYMENT.md says "no third-party runtime dependencies… works behind a firewall", which is not quite true: blocked networks just fall back to system fonts.	Remove the imports (the text they style is mostly hidden) or self-host
/api/health frozen	Statically prerendered, so its timestamp and "random" latencies were fixed at build time (identical across requests).	`export const dynamic = "force-dynamic"`, or delete it
Contact API	Validates input but discards it; no form uses it.	Remove, or add a real form and delivery (e.g. email service)
README	Describes generation 2 (§10.4).	Rewrite from Chapter 4 and 5 of this document
robots/sitemap	Fall back to `http://localhost:3000` when deployed outside Vercel without `NEXT_PUBLIC_SITE_URL`.	Set the variable on any non-Vercel host

11.13 Accuracy of technical claims INFO

A science exhibition judge may probe the hardware story. These statements on the site deserve a prepared, honest answer, or a small wording change:

CLAIM ON THE SITE	WHY A JUDGE MIGHT QUESTION IT	SUGGESTION
A TensorFlow Lite model "runs on the device itself" (the ESP8266): *612 KB model, 214 KB arena, 38 ms*	Espressif's datasheet gives only **~50 KB of heap + data RAM** available to programs on the ESP8266 in station mode, so a 214 KB tensor arena cannot fit. TensorFlow Lite Micro targets ESP32-class chips, and the OV2640 camera is usually paired with an ESP32-CAM.	State where inference really runs (an ESP32-CAM, a phone or a laptop), or present it as planned work
"160 MHz" in the copy; the 3D board says "80MHz Tensilica"	Both are correct (80 MHz default, 160 MHz overclock), but they look inconsistent side by side	Pick one, or say "80/160 MHz"
DHT22 listed as "1-Wire"	The DHT22 uses its own single-wire protocol, not Dallas 1-Wire	"Single-wire digital"
"Every site measured on its own" (20 sites)	The boot log lists "ads1115 soil x4". Even four ADS1115 chips give 16 channels	Explain the multiplexing, or say "per tier"
Comparison "measured… over three full cycles"	The numbers are constants in the code	Bring the logbook and photos, or call them estimates
Console "the real event stream"; telemetry "Streaming · Firebase RTDB"	Both are simulated in the browser (§10.5)	Label as simulation, or connect Firebase
"95% less water than soil" (site) vs `/api/stats`	The public stats endpoint says `waterSaved: "42%"`, claims 142 active devices and 89 registered users, and /api/health gives 340 ms inference (the site says 38 ms)	Make the API agree with the site, or remove it

RECOMMENDATIONS & EXHIBITION PREP

One focused day turns this from impressive to flawless. Then: a roadmap, a checklist for the exhibition hall, the questions judges are likely to ask, and a ninety-second demo.

12.1 The fix-first plan (about one day)

#	TASK	WHERE	TIME	FIXES
1	Observe the parent heading, animate the lines with variants	Manifesto.tsx	5 min	§11.2 HIGH
2	Add `<meta name="color-scheme" content="dark">` to the three scenes	public/scenes/*.html	5 min	§11.3 HIGH
3	Dark fog in pod.html; add `.viewer-tools` to its embed CSS	pod.html	5 min	§11.4
4	Correct the chapter-pip scroll formula	TowerStory.tsx	5 min	§11.6
5	Add a mobile tower slot behind the tour copy	TowerStory.tsx	30 min	§11.7
6	Shrink the two long headlines; loosen display leading	BoardSection, SensingSection, globals.css	15 min	§11.8
7	"VERD" → "VERDE"; manifest + favicon; one numbering scheme; "Fifteen weeks"	tower.html, manifest.json, layout.tsx…	15 min	§11.9
8	`npm i next@14.2.35 eslint-config-next@14.2.35`, then `npm run verify`	package.json	10 min	§11.12
9	Add "SIMULATED DATA" badges or honest copy to the console, telemetry and comparison	LiveTerminal, Telemetry, Comparison	15 min	§10.5
10	Measure the deck track; pick the active card by geometry	HorizontalDeck.tsx	1–2 h	§11.5
11	Pause off-screen scenes (new `pause` / `resume` commands)	ScenePreloader + 3 scenes	1–2 h	§11.10
12	Rewrite the README from Chapters 4–5; delete leftovers (§10.3)	README.md, tailwind.config.ts, globals.css	1 h	§10.3–10.4

TIP

After step 8, run the same checks this review used:

```
npm ci && npm run verify        # tsc + lint + build must all pass
npm start                       # open http://localhost:3000 and scroll every section
```

12.2 Roadmap: what would make it outstanding

IDEA	WHY	HOW
One real number, live	Turns the strongest claim ("always reporting") into proof	Read the tower's Firebase RTDB path (read-only rules) in Telemetry; fall back to simulation with a visible badge
Photos of the real build	The site never shows the physical tower; judges want to see the actual soldering	A gallery section with `next/image`: tower, PCB, wiring, first harvest
Automated checks	Would have caught §11.2–11.7 automatically	Playwright smoke test (scroll, assert headline visible, screenshot) + GitHub Actions running `npm run verify`
Port board.html to r162	One three.js build; consistent colour management (fixes §11.8's pale board frame)	Replace the global `THREE` calls with module imports; delete `public/vendor/legacy` (−603 KB)
Share image	Links on WhatsApp currently show no preview picture	Add `app/opengraph-image.png` (1200×630), e.g. the hero render
Hindi version	The tower speaks Hindi; the site could too	A simple `/hi` route with translated copy
Offline-ready	Exhibition Wi-Fi is unreliable	Service worker caching the page, scenes and vendor files
Send the unused commands	Free interactivity	Palette entries for tower lights/flow and the board LED

12.3 Exhibition-day checklist

- ✓ **Open the site once** on the stand laptop before doors open: three.js and the scenes are cached for a year after the first visit.
- ✓ **Carry an offline copy:** after `npm ci`, `npm run build && npm start` serves the whole site from the laptop with no internet (the scenes need no CDN).
- ✓ **Plug in power** and disable sleep: three WebGL scenes render continuously.
- ✓ Use **Chrome full-screen** (F11) with one tab open.
- ✓ Print a **QR code** to the deployed URL on the poster, so visitors take the project home.
- ✓ For an unattended loop, revive the deleted **KioskMode** idea (`?kiosk=true` auto-advancing through sections).
- ✓ Rehearse the **"Explode the board"** moment and dragging the tower: these are the crowd-pleasers.
- ✓ Keep this document's **§10.5 and §11.13** in mind when answering questions.

12.4 Questions judges may ask, with honest answers

Q Why hydroponics instead of soil?

A A recirculating loop reuses the same nutrient solution instead of letting it drain away, so the tower needs roughly a twentieth of the water of a soil bed (18 L vs 0.9 L per cycle in our comparison), and stacking four tiers fits twenty plants into about half a square metre.

Q What happens if the Wi-Fi drops?

A Irrigation and lighting decisions are made on the ESP8266 itself; Firebase is only a reporting channel. Readings are buffered and sent when the connection returns, and a physical rain sensor replaces the forecast.

Q Where does the leaf-disease model actually run?

A *Answer precisely.* The ESP8266 has only about 50 KB of usable RAM, so if the model runs on an ESP32-CAM, a phone or a laptop, say so; if it is still in development, say that. (See §11.13.)

Q Is the dashboard on the website live?

A The website shows a simulation built from our real sensor ranges, so visitors can see what the tower reports; the live data goes to Firebase. (Or, after §12.2: "This reading is live from the tower right now.")

Q How did you build the 3D models?

A They are not downloaded files. Every part is generated by JavaScript with three.js: shelves, pots, 200 lettuce leaves, the PCB traces and pin labels. That keeps each scene under 80 KB and lets us animate parts, like exploding the board.

Q Why are the 3D scenes in iframes?

A Isolation and speed: each scene has its own WebGL context, a crash cannot break the page, three.js stays out of the main bundle (the page loads only 157 kB of JavaScript), and the scenes are mounted once and moved around as you scroll so they never reload.

Q What was the hardest bug?

A Good candidates: the hydration mismatch (random values rendered on the server, §3.4), `position: sticky` silently broken by `overflow-x: hidden`, or the colour-scheme rule that made iframes paint white (§11.3).

Q Did you use AI tools?

A Yes. The website was developed with an AI coding agent (credited as a co-author in the git history), directed and reviewed by us. We can explain every part of it, from the scene preloader to the deployment headers.

Q What would you do next?

A Show live data from the tower on the site, add photos of the physical build, port the board scene to the newer three.js, and add automated tests (§12.2).

12.5 A 90-second demo script

TIME	ON SCREEN	SAY
0–10 s	Boot curtain → hero	"This is Verde: an autonomous hydroponic tower. The website boots like the device does."
10–25 s	Manifesto → stats	"Soil is running out and water tables are falling. Verde uses about 95% less water and fits 20 plants in the space of a bookshelf."
25–40 s	Loop deck	"Every two seconds it senses, sees, decides, acts, reports and reclaims the water."
40–60 s	Tour: drag the tower	"Each tier has a job: light, plant health, roots and the closed loop. You can orbit the real model."
60–75 s	Hardware: Explode the board	"One ESP8266 runs all of it, on a custom KiCad board." (click Explode)
75–90 s	Build log → team	"Fourteen weeks, two people: Anuj on software and Aarav on hardware. Questions?"

FILE-BY-FILE INDEX

All 73 tracked files at commit 9d5576d, with size, status and purpose. Status: USED part of the live site (41) · VENDOR third-party code shipped with the site (7) · CONFIG build/tooling (11) · DOC documentation (2) · MOCK deployed but simulated (5) · LEFTOVER present but not used (6) · UNUSED planning material (1).

PATH	LINES	SIZE	STATUS	PURPOSE
.env.example	4	277 B	CONFIG	Documents the only optional env var, NEXT_PUBLIC_SITE_URL
.eslintrc.json	3	40 B	CONFIG	ESLint: extends next/core-web-vitals
.gitignore	40	341 B	CONFIG	Ignores node_modules, .next, .vercel, env files, .preview/, nul
.nvmrc	1	3 B	CONFIG	Node 20
DEPLOYMENT.md	75	2.7 KB	DOC	Vercel deployment guide; accurate
README.md	222	8.6 KB	DOC	Project overview; describes generation 2 (stale, §10.4)
app/api/contact/route.ts	49	1.5 KB	MOCK	POST validation + fake transmission ID; GET status
app/api/health/route.ts	22	1.0 KB	MOCK	Fake health report; frozen at build time
app/api/stats/route.ts	32	953 B	MOCK	Static project statistics (₹180 vs ₹310, 42%)
app/api/telemetry/route.ts	46	2.3 KB	MOCK	Random sensor values (Tulsi profile); POST echo
app/api/weather/route.ts	28	1.7 KB	MOCK	Static Delhi forecast + irrigation rules
app/components/BoardSection.tsx	56	2.2 KB	USED	Hardware section; scroll spins the 3D board; explode button
app/components/BootCurtain.tsx	127	4.1 KB	USED	Boot sequence overlay: glitch logo, counter, streamed log
app/components/Closing.tsx	35	1.5 KB	USED	“CULTIVATE.” + magnetic CTA (the only lucide icon)
app/components/CommandPalette.tsx	176	5.9 KB	USED	⌘K palette: section jumps and model commands
app/components/Comparison.tsx	91	4.2 KB	USED	Soil vs Verde paired bars
app/components/CustomCursor.tsx	65	2.2 KB	USED	Dot + spring ring cursor (fine pointers only)
app/components/Faq.tsx	99	4.3 KB	USED	Five-question accordion
app/components/Hero.tsx	187	6.9 KB	USED	Hero: HUD, headline, cards, SSR-safe tickers
app/components/HorizontalDeck.tsx	234	8.2 KB	USED	Pinned horizontal loop deck (6 cards); travel bug §11.5
app/components/LiveTerminal.tsx	135	5.4 KB	USED	Scripted streaming console
app/components/Magnetic.tsx	40	1.0 KB	USED	Wrapper that pulls children toward the cursor
app/components/Manifesto.tsx	68	2.7 KB	USED	Manifesto; headline never animates in (§11.2)

PATH	LINES	SIZE	STATUS	PURPOSE
app/components/NeuralField.tsx	142	4.0 KB	USED	Canvas 2D particle lattice behind the hero
app/components/Pillars.tsx	85	3.1 KB	USED	Three tilt cards
app/components/ScenePreloader.tsx	201	6.7 KB	USED	Mounts 3 scene iframes once; glues them to slots; message bus
app/components/ScrambleText.tsx	87	2.1 KB	USED	Random-glyph text reveal (hero cue, terminal title)
app/components/ScrollProgress.tsx	16	482 B	USED	2 px top progress bar
app/components/SensingSection.tsx	59	2.2 KB	USED	Plant-level section with the pod scene
app/components/SideRail.tsx	76	2.4 KB	USED	Fixed section index (≥1280 px)
app/components/SmoothScroll.tsx	45	1.2 KB	USED	Lenis smooth scroll + anchor routing
app/components/SpecExplorer.tsx	100	3.9 KB	USED	Tabbed subsystem specs (18 parts)
app/components/SplitText.tsx	60	1.6 KB	USED	Word-by-word heading reveal (correct in-view pattern)
app/components/StatsStrip.tsx	63	2.2 KB	USED	Count-up stats in brutalist cells
app/components/TeamSection.tsx	68	2.3 KB	USED	Two team cards
app/components/Telemetry.tsx	207	6.9 KB	USED	Random-walk sparklines + reservoir gauge
app/components/TextReveal.tsx	43	1.3 KB	USED	Scroll-linked word illumination
app/components/Timeline.tsx	62	3.4 KB	USED	Build log with scroll-drawn rail
app/components/TowerStory.tsx	146	6.0 KB	USED	Pinned 4-chapter tour driving the tower camera
app/components/VelocityMarquee.tsx	71	2.3 KB	USED	Scroll-velocity-reactive marquee (3 uses)
app/components/site/Footer.tsx	111	3.6 KB	USED	Footer: link columns, colossal wordmark
app/components/site/Nav.tsx	96	3.0 KB	USED	Fixed header, links, palette chip, mobile menu
app/globals.css	602	14.9 KB	USED	All custom CSS: tokens, components, effects (some dead classes)
app/google280c2ec99007341d.html	0	53 B	LEFTOVER	Duplicate verification file (not served from app/)
app/layout.tsx	74	2.2 KB	USED	Root layout: fonts, metadata, viewport
app/page.tsx	82	2.6 KB	USED	Home page: section order and global layers
app/robots.ts	14	409 B	USED	Generates /robots.txt
app/sitemap.ts	18	426 B	USED	Generates /sitemap.xml
docs/3d-model-prompts.md	214	8.3 KB	UNUSED	Brief for AI-generated GLB models (never adopted)
google280c2ec99007341d.html	0	53 B	LEFTOVER	Duplicate verification file (repo root, not served)
model1.html	380	23.5 KB	LEFTOVER	Original light-theme tower prototype (three 0.164.1 via CDN)
model2.html	1,591	75.3 KB	LEFTOVER	Original light-theme pod prototype (three 0.160.0 via CDN)
model3.html	474	37.8 KB	LEFTOVER	Original light-theme board prototype (three r128 via CDN)
next.config.mjs	73	2.2 KB	CONFIG	Headers, caching, removeConsole, dev-only polling
package-lock.json	6,437	225.4 KB	CONFIG	Exact dependency tree (408 packages) for npm ci
package.json	40	1.0 KB	CONFIG	Dependencies, scripts, engines

PATH	LINES	SIZE	STATUS	PURPOSE
postcss.config.mjs	3	142 B	CONFIG	Tailwind + autoprefixer
public/favicon.svg	5	348 B	USED	Favicon: a green “A”-shaped glyph (not the leaf logo, §11.9)
public/google280c2ec99007341d.html	0	53 B	USED	Google Search Console verification (served)
public/manifest.json	14	527 B	LEFTOVER	Web manifest; not linked; icon /favicon.ico is 404
public/scenes/board.html	506	39.6 KB	USED	ESP8266 board scene (three r128)
public/scenes/pod.html	1,602	76.4 KB	USED	Seedling pod scene (three r162)
public/scenes/tower.html	443	26.2 KB	USED	Four-tier tower scene (three r162)
public/vendor/addons/controls/OrbitControls.js	1,532	32.3 KB	VENDOR	Camera orbit controls (r162)
public/vendor/addons/environments/RoomEnvironment.js	148	3.7 KB	VENDOR	Procedural studio environment (pod)
public/vendor/addons/geometries/RoundedBoxGeometry.js	155	4.6 KB	VENDOR	Rounded boxes (tower, pod)
public/vendor/addons/utils/BufferGeometryUtils.js	1,371	31.6 KB	VENDOR	mergeGeometries (pod)
public/vendor/legacy/OrbitControls.js	1,045	26.4 KB	VENDOR	Orbit controls for the legacy build (board)
public/vendor/legacy/three.min.js	6	603.4 KB	VENDOR	three.js r128 UMD build (board)
public/vendor/three.module.js	53,411	1.29 MB	VENDOR	three.js r162 ES module build
tailwind.config.ts	134	5.0 KB	CONFIG	Tailwind config; custom theme entirely unused (§10.3)
tsconfig.json	40	666 B	CONFIG	Strict TypeScript; unused @/* alias
vercel.json	8	199 B	CONFIG	Vercel: npm ci, bom1 region

GLOSSARY

TERM	MEANING
App Router	Next.js's folder-based routing (the `app/` directory) where layouts and pages are React Server Components by default.
Server / client component	Server components run only at build or request time; `"use client"` components also run in the browser and can use state and effects.
Prerendering (SSG)	Generating a page's HTML at build time so a CDN can serve it instantly. Verde's home page is fully prerendered (○).
Hydration	React attaching event handlers and state to server-rendered HTML. Server and client output must match exactly.
Route handler	A file `app/api/…/route.ts` exporting GET/POST functions; deployed as a serverless function or a static file.
Metadata API	Next's typed way of declaring titles, Open Graph, Twitter cards, robots and canonical URLs.
WebGL / three.js	WebGL is the browser's GPU drawing API; three.js is a library that builds scenes, cameras, lights and materials on top of it.
Procedural geometry	3D shapes generated by code at runtime instead of loaded from a model file.
PBR material	Physically based rendering: materials described by colour, roughness and metalness so they react realistically to light.
ACES tone mapping	A film-industry curve that compresses bright HDR light into displayable colours.
PMREM / environment map	A pre-filtered image of the surroundings used for realistic reflections (the pod's RoomEnvironment).
Shadow map / contact shadow	Real-time shadows rendered from a light's view; a contact shadow is a soft blob under an object, here baked once.
InstancedMesh	One draw call that renders many copies of the same geometry with different transforms (the pod's 440 soil clumps).
Import map	A `<script type="importmap">` mapping bare module names like `three` to URLs.
iframe	An embedded document with its own JavaScript, DOM and WebGL context.
postMessage	The safe way for two documents (page and iframe) to send each other messages.
color-scheme	CSS telling the browser whether a document is light or dark; mismatches make iframes opaque (§11.3).
requestAnimationFrame	Browser callback once per screen refresh; the basis of every animation loop.
IntersectionObserver	Browser API reporting when an element enters or leaves the viewport; behind "in view" animations.
position: sticky	An element scrolls normally until it reaches an edge, then stays pinned inside its parent (the deck and tour).
overflow: clip vs hidden	Both hide overflow, but only `hidden` creates a scroll container (which breaks sticky).
Spring animation	Motion simulated as a spring (stiffness, damping, mass) instead of a fixed duration.
Easing curve	How speed changes over an animation; Verde's signature is `cubic-bezier(0.16, 1, 0.3, 1)`.
Lenis	A smooth-scrolling library that eases wheel input.
Tailwind CSS	Utility classes (`flex`, `px-6`, `text-[clamp(…)]`) composed in markup.
Variable font	One font file containing every weight; Verde self-hosts four families via Fontsource.
CDN / edge region	Servers that cache files close to users; Verde is pinned to Vercel's bom1 (Mumbai).

TERM	MEANING
Cache-Control: immutable	Tells browsers a file will never change, so it is never revalidated (the vendored three.js).
WCAG contrast	Accessibility guideline: normal text needs at least 4.5:1 contrast against its background.
ESP8266	A low-cost Wi-Fi microcontroller (Tensilica L106, 80/160 MHz, ~50 KB usable RAM).
ADS1115	A 4-channel 16-bit analog-to-digital converter on I²C; adds analog inputs to the ESP8266 (which has one).
I²C / RS485 / Modbus	I²C: a two-wire bus for on-board chips. RS485: robust long-distance wiring. Modbus: the protocol the NPK probe speaks over it.
PWM / MOSFET	Pulse-width modulation dims LEDs by switching them rapidly; a MOSFET is the transistor doing the switching.
SPDT relay	Single-pole double-throw switch operated by an electromagnet; here it switches the pump.
Buck converter (LM2596)	Efficiently steps a higher DC voltage down to 5 V.
Flyback diode	Protects electronics from the voltage spike when a relay coil switches off.
TensorFlow Lite / int8 quantisation	A compact ML runtime; quantisation stores weights as 8-bit integers to shrink models for small devices.
Tensor arena	The RAM block TFLite Micro needs for intermediate values during inference.
Firebase RTDB	Google's realtime JSON database; devices write values and dashboards see updates instantly.
Hydroponics	Growing plants without soil, roots in a nutrient solution; a closed loop recirculates it.

VERIFICATION LOG

The exact commands run for this review on 18 October 2026 (Node 22, Debian 12), with their results. Anyone can repeat them.

Install, type-check, lint, build

```
$ git fetch --unshallow origin          # full history: 8 commits
$ npm ci                                 # added 408 packages
  npm warn deprecated next@14.2.5: This version has a security vulnerability…
$ npx tsc --noEmit                       # exit 0
$ npx next lint                          # ✔ No ESLint warnings or errors
$ npx next build
  ✓ Compiled successfully · ✓ Generating static pages (11/11)
  Route (app)                    Size     First Load JS
  ┌ ○ /                          70.1 kB         157 kB
  ├ ○ /_not-found                871 B          87.9 kB
  ├ ƒ /api/contact               0 B                0 B
  ├ ○ /api/health                0 B                0 B
  ├ ○ /api/stats                 0 B                0 B
  ├ ƒ /api/telemetry             0 B                0 B
  ├ ○ /api/weather               0 B                0 B
  ├ ○ /robots.txt                0 B                0 B
  └ ○ /sitemap.xml               0 B                0 B
  + First Load JS shared by all  87 kB
```

HTTP checks against the production server

```
$ next start -H 0.0.0.0 -p 3000
/                        200  107,637 B  (17,211 B gzip)
/robots.txt              200       83 B
/sitemap.xml             200      256 B
/api/health              200      767 B   identical on every request (static)
/api/telemetry           200      986 B   different on every request (dynamic)
/scenes/tower.html       200   26,155 B
/scenes/pod.html         200   76,415 B   (20,443 B gzip)
/scenes/board.html       200   39,556 B
/vendor/three.module.js  200 1,285,851 B  (260,630 B gzip)  Cache-Control: public, max-age=31536000, immutable
/vendor/legacy/three.min.js 200 603,445 B  (149,992 B gzip)
/manifest.json           200      527 B
/favicon.svg             200      348 B
/favicon.ico             404             referenced by manifest.json
POST /api/contact {email:"bad"}  → 400 {"error":"Missing required fields",…}
POST /api/contact {valid}        → 200 {"success":true,"transmissionId":"VERDE-YE9F-QS10",…}
```

Browser measurements

```
# Chromium 131 headless, SwiftShader WebGL 2.0, 1440×900 unless noted
page height                    21,278 px = 23.6 viewports · 1,131 DOM nodes
manifesto lines after jump     opacity 0.00 ×3 (transform matrix3d… unchanged)
manifesto lines after 40 wheels  opacity 0.00 ×3 · child rect height 0, 150 px below parent
iframe color-scheme            element: dark · scene document: normal · parent meta: dark
A/B colour-scheme override     white regions removed (tower, pod); board unchanged
deck end (1440)                RECLAIM x:-33..457 active · 68% of width empty
deck at 0.885 (1440)           counter "05" · REPORT x:-533..-78 (off-screen)
deck end (390, phone)          counter "06" · DECIDE -3..302 · ACT 344..649 · REPORT 692.. · RECLAIM 1028..
tour pips 1→4                  ["01 / 04","03 / 04","04 / 04","04 / 04"]
frame time (CPU renderer)      tower ≈1.5 s (all 3 running: 0.57 fps) · pod ≈0.44 s · board ≈0.24 s
```

Static analysis

unused Tailwind theme utilities	0 matches (verde-*, accent-*, dark-*, primary, 14 animate-*, shadow-neon/glass, ease-expo-*)
unused CSS classes	.reveal .verde-aurora .mega-marquee .btn-solid .hairline → 0 uses
API calls from UI	grep "fetch(" | "/api/" in app/components → 0
"@/" alias imports	0
files deleted in 592fce4	66 (40 components, 20 sections, 5 hooks, 1 util)
signature easing occurrences	35 in TSX + 3 in CSS

REFERENCES

TOPIC	SOURCE
Repository reviewed	github.com/UCHIHA-MADARA-ANUJ/Verde-CBSE: main @ 9d5576d; pull request #1
Next.js security advisory (14.x fixed in 14.2.35)	nextjs.org/blog/security-update-2025-12-11 · github.com/vercel/next.js/security/advisories/GHSA-5j59-xgg2-r9c4
Next.js 14 end of life (26 Oct 2025)	endoflife.date/nextjs
iframe colour-scheme rule (opaque canvas on mismatch)	W3C, CSS Color Adjustment Module Level 1, §2.2: www.w3.org/TR/css-color-adjust-1/ · Chromium issue 40849375 (“This is per spec”) · fvsch.com/transparent-iframes
ESP8266 usable RAM (~50 KB heap + data in station mode)	Espressif, ESP8266EX Datasheet, §3 Functional Description: documentation.espressif.com/0a-esp8266ex_datasheet_en.pdf
Libraries	nextjs.org/docs · react.dev · motion.dev (Framer Motion) · github.com/darkroomengineering/lenis · threejs.org/docs · tailwindcss.com · fontsource.org · lucide.dev
Browser APIs	developer.mozilla.org: IntersectionObserver, Window.postMessage, requestAnimationFrame, CSS position: sticky, overflow: clip, color-scheme

This walkthrough was prepared on 18 October 2026 by reviewing every file in the repository, the full git history and pull request #1, and by building, running and measuring the production site. Screenshots are real captures (headless Chromium 131, software WebGL; 3D frames therefore look slightly softer than on a GPU). One figure is explicitly labelled as a simulated fix. Statements about intent are quoted from the code where possible and marked as inference otherwise.